

Lecture 21

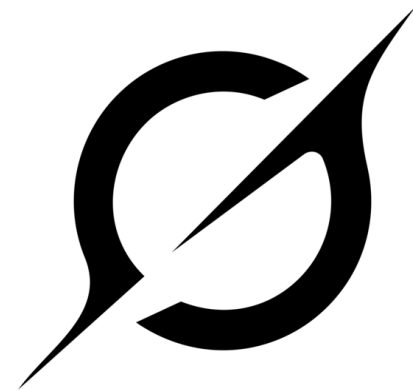
Attention and Transformers

Efficiently modeling sets of objects with attention.

EECS 189/289, Spring 2026 @ UC Berkeley



Gemini



Claude



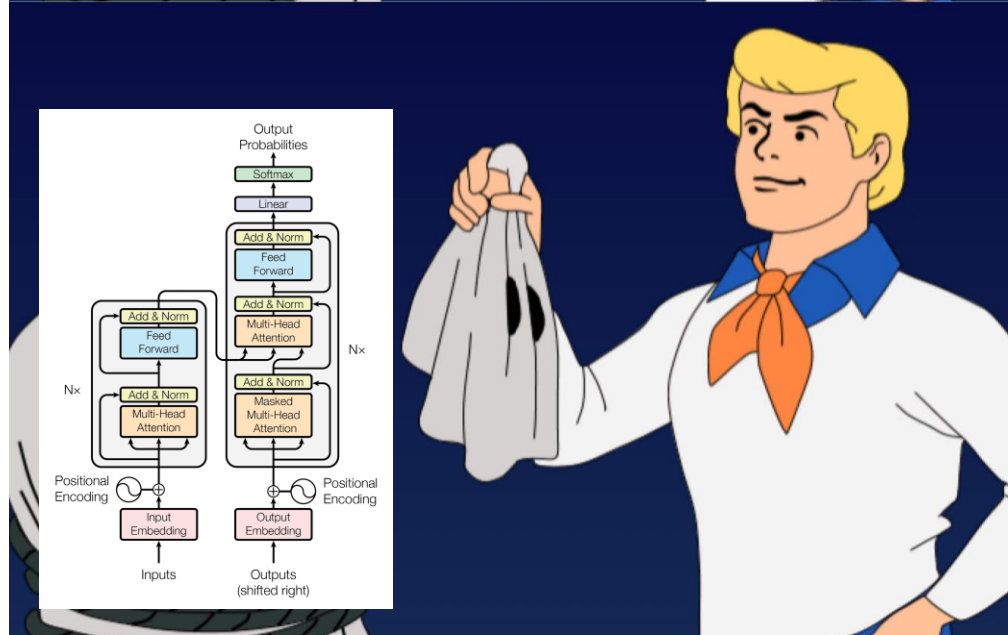
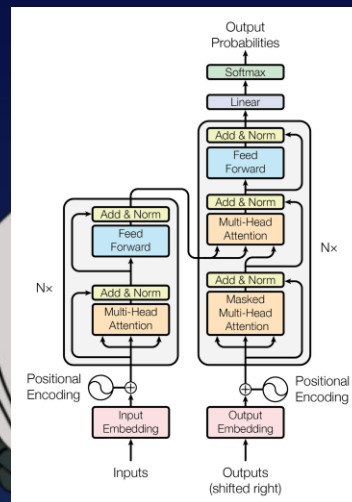
LLaMA
by  Meta



deepseek

 Gemini
 Claude
 LLaMA by Meta


 deepseek



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

NeurIPS 2017

Attention Is All You Need

Ok, so what is it that you don't need?

Who invented attention?

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*

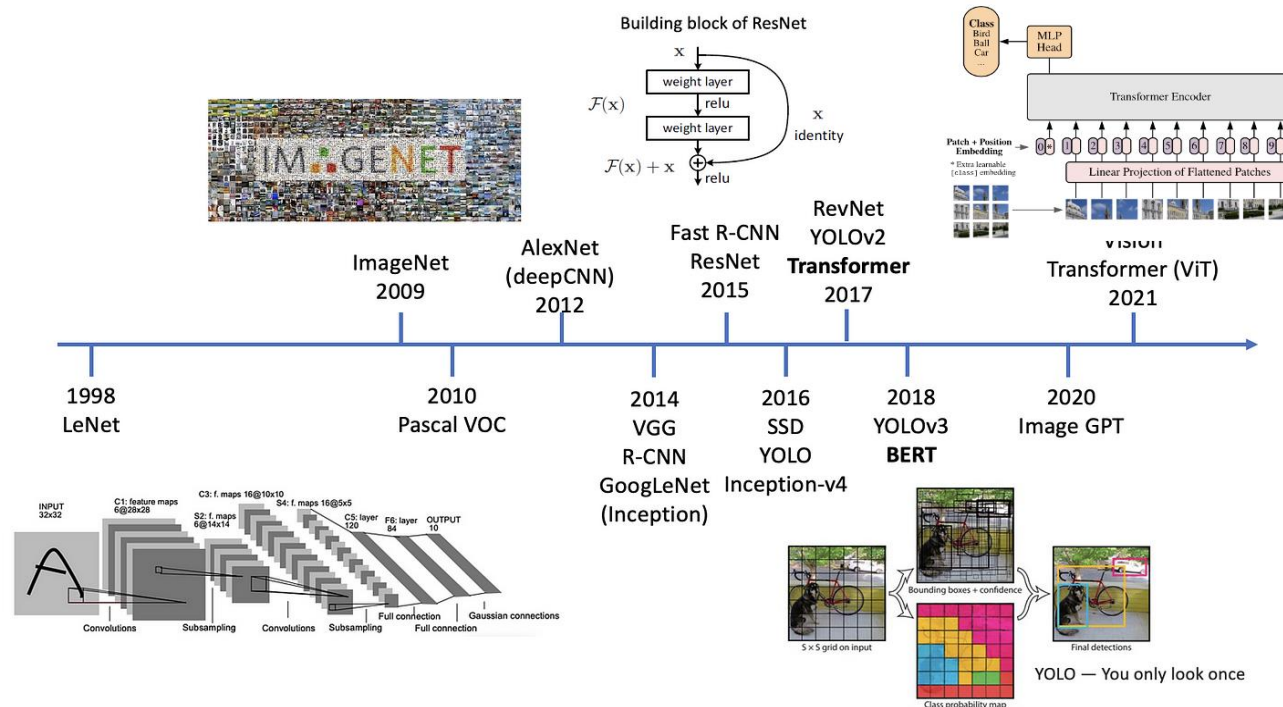
Université de Montréal

ABSTRACT

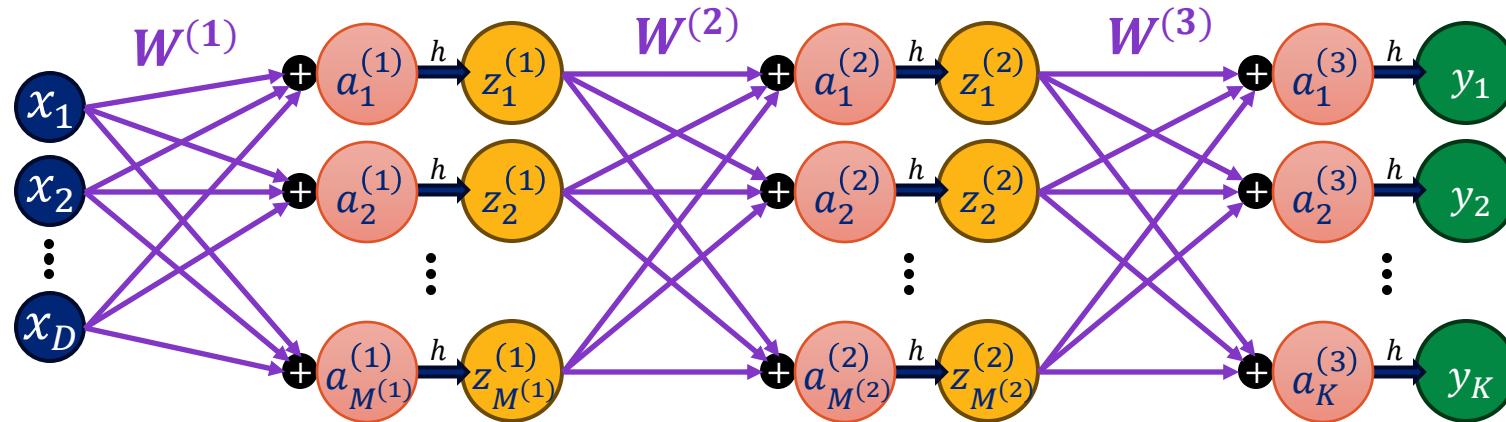
Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Transformative Architecture Ideas

- CNNs
- **Residual connections** – ubiquitous in modern neural networks!
- **Attention**
- **Transformers** – predominant architecture of today's AI/ML era
 - Natural language processing, computer vision, multi-modal models, reinforcement learning, robotics, etc.



Deep Learning is **Feature Transformation**



Deep neural networks **transform raw features**, layer by layer

Each layer is a different **representation** of the data

- **Embeddings**: intermediate representations (layers)

Final layers represent the “**abstract concepts**” associated with the prediction task.

MLP Networks

We started with the “**multi-layer perceptron**”

A **stack** of [Linear Layer + Non-Linearity]

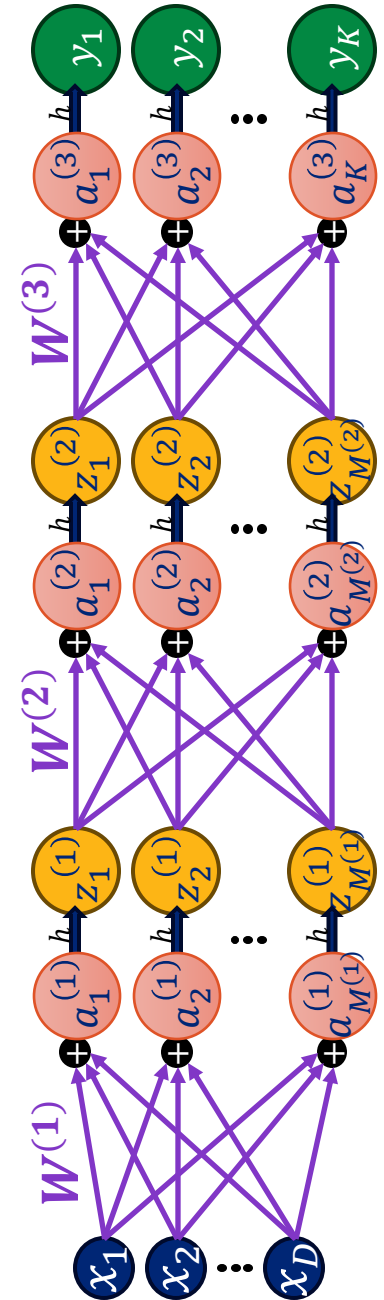
- **Example:** $y = \text{ReLU}(W^{(2)} \text{ReLU}(W^{(1)}x))$

Strengths:

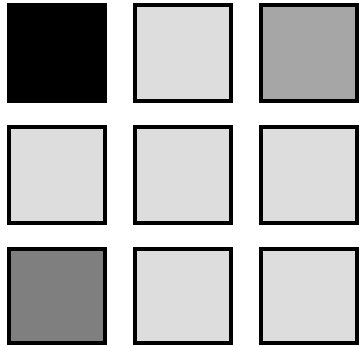
- Highly expressive!

Weaknesses:

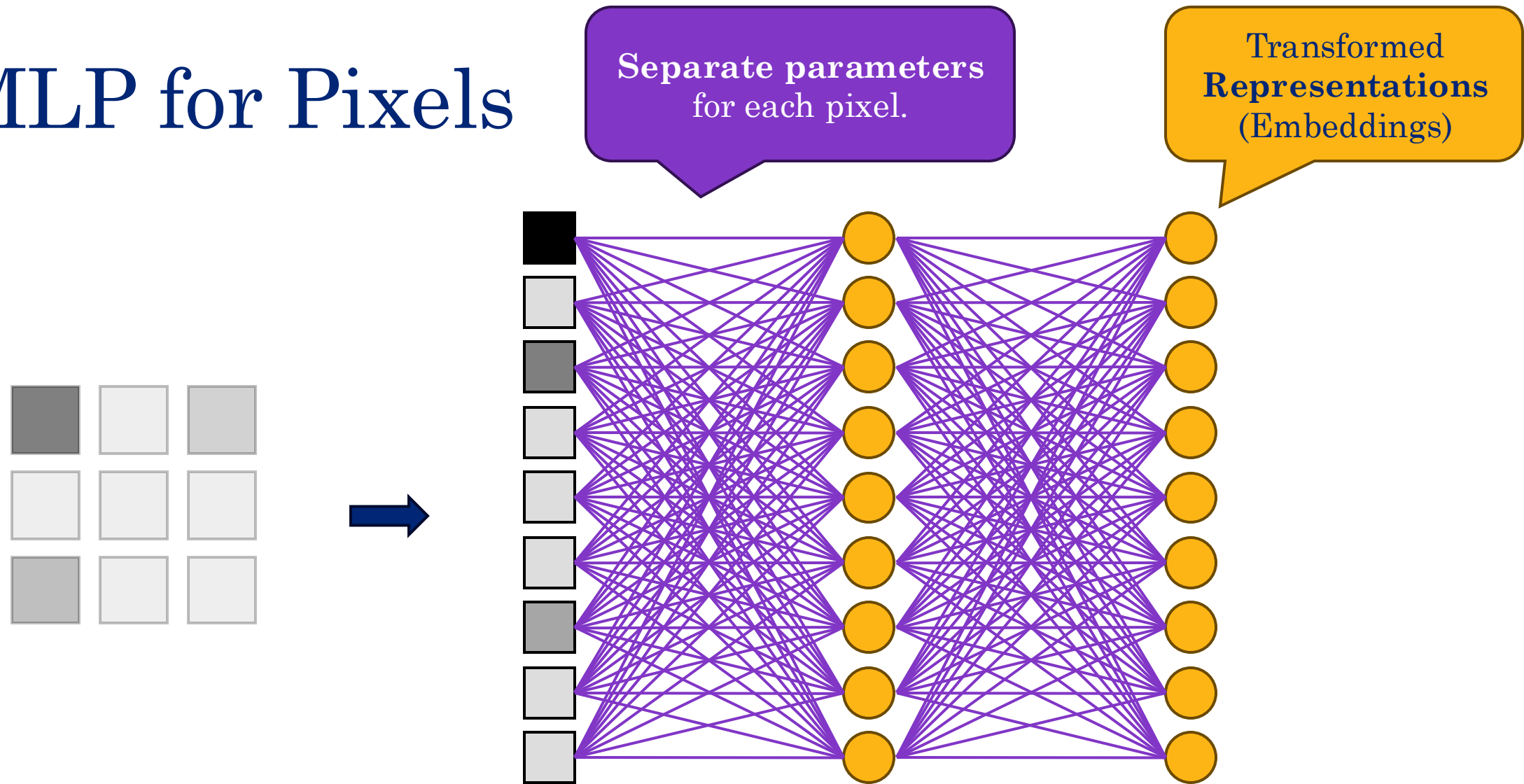
- Significant number of parameters!
- Fixed input size.



MLP for Pixels

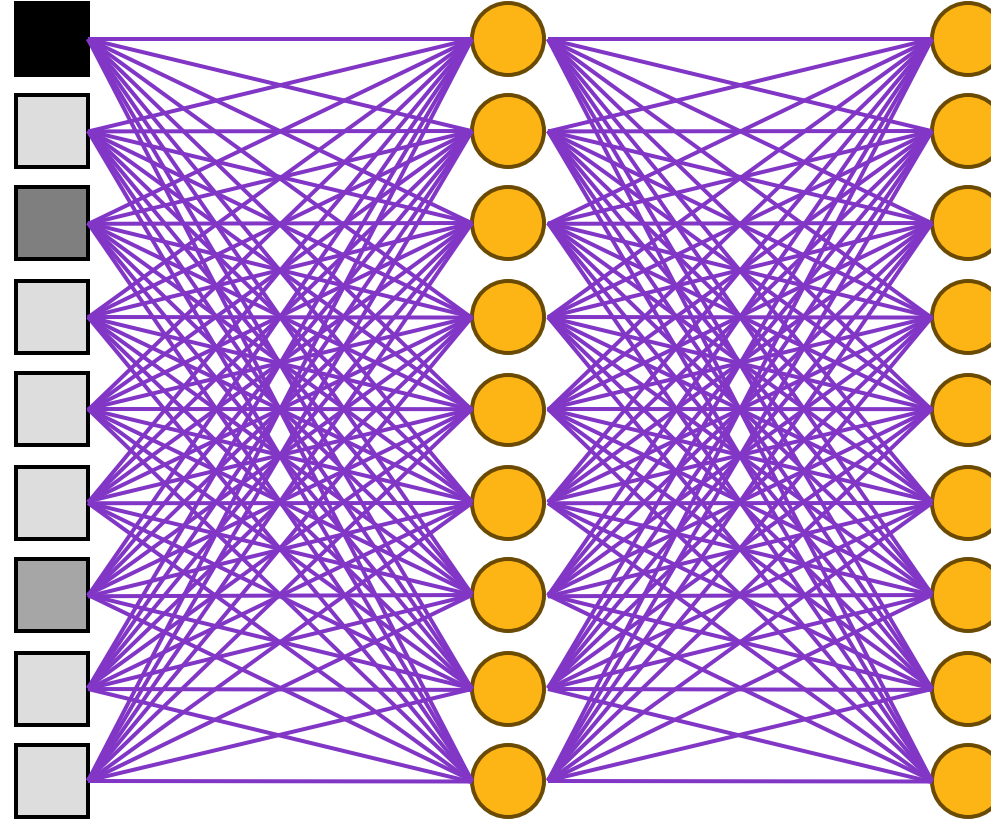


MLP for Pixels



If there are P pixels, how many parameters are in each layer?

MLP for Pixels

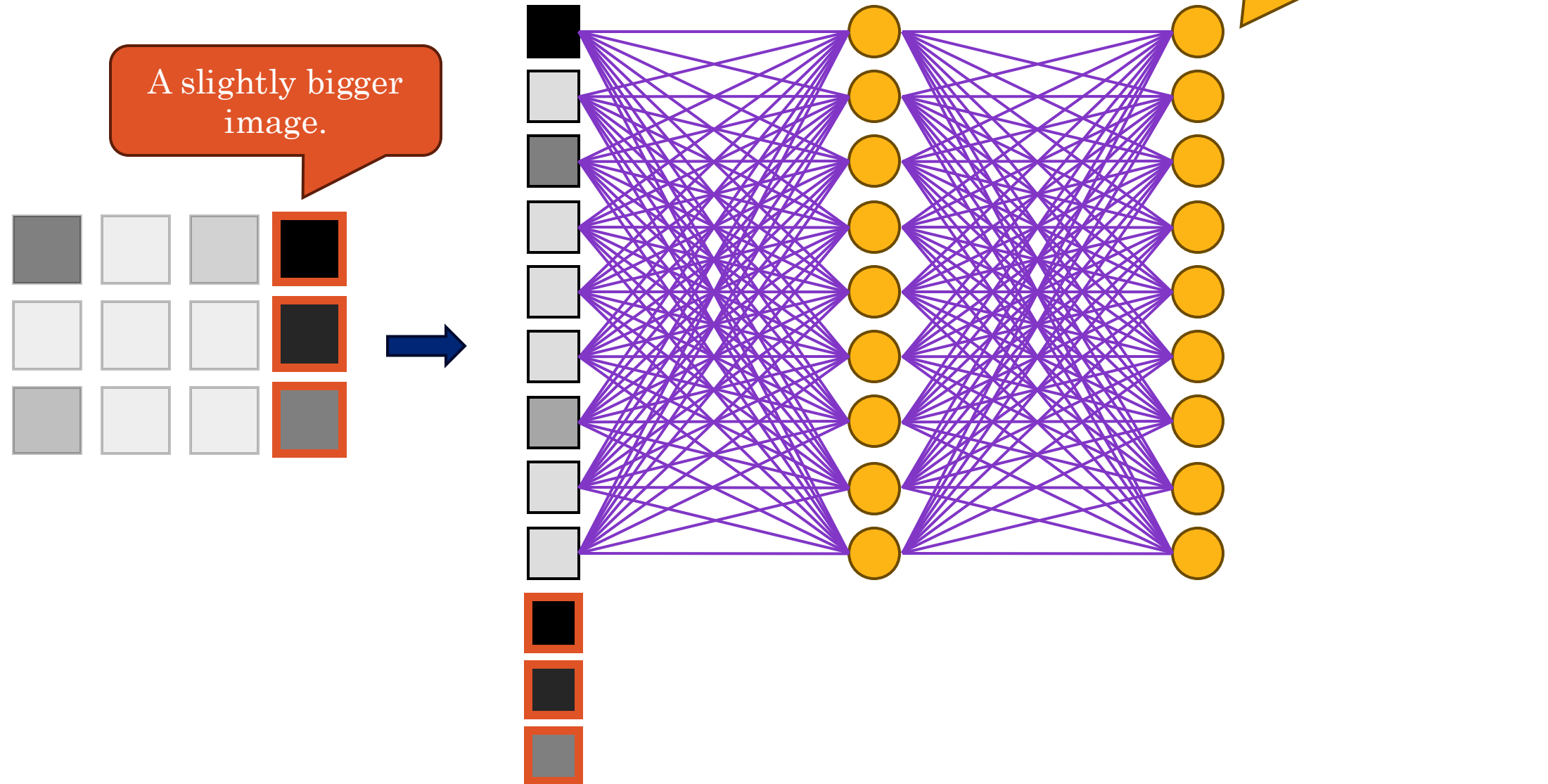


How many Parameters?

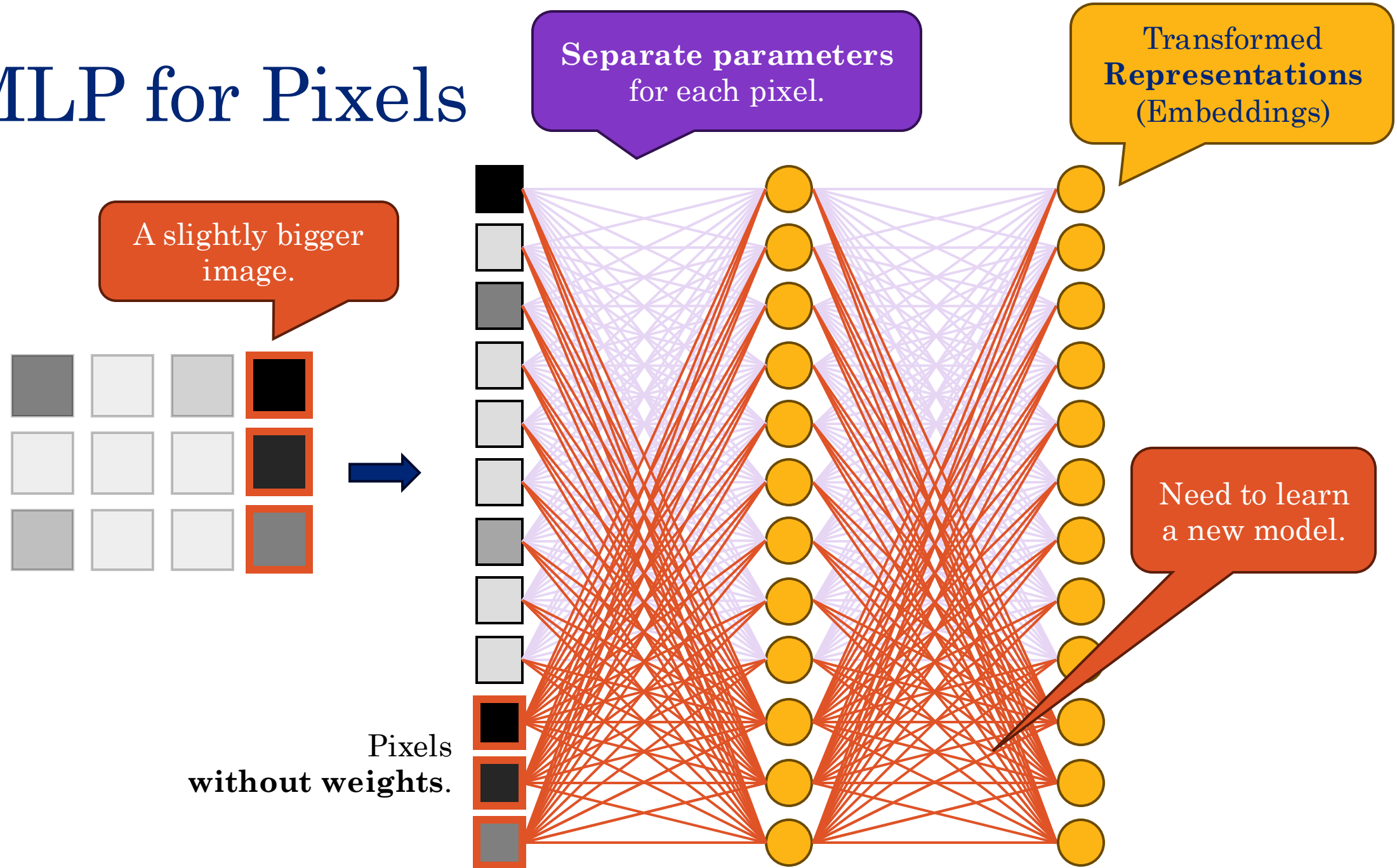
If there are P pixels then there are P^2 **weights** per layer.

Need to learn a **separate representation** for textures at every location.

MLP for Pixels

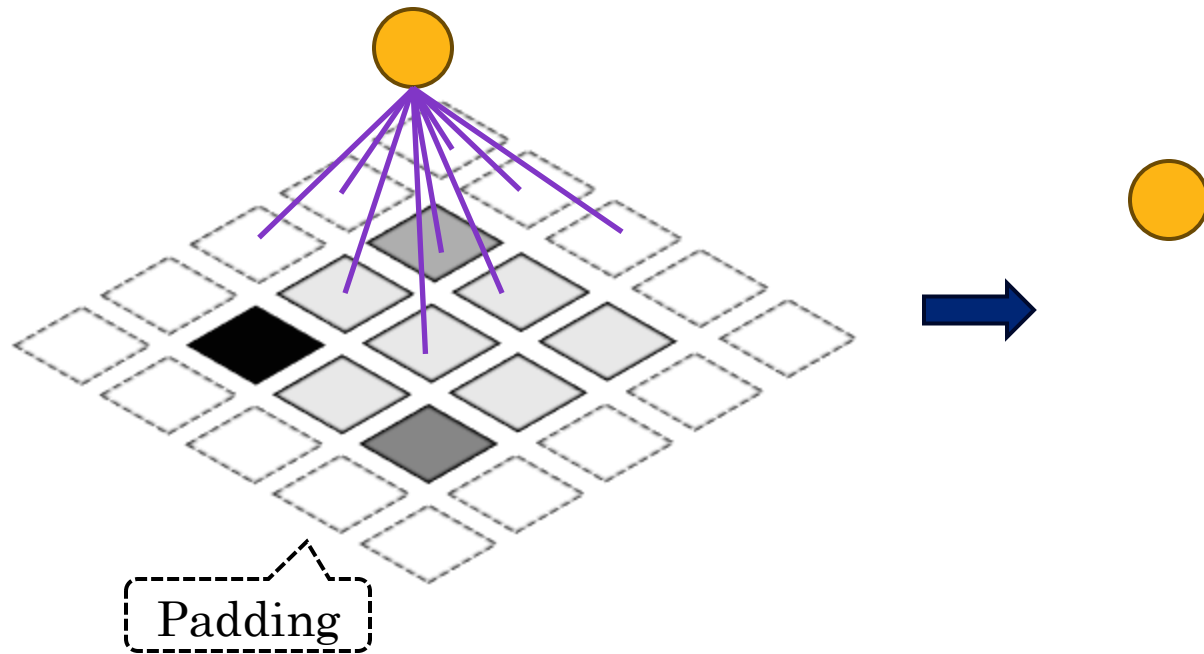


MLP for Pixels



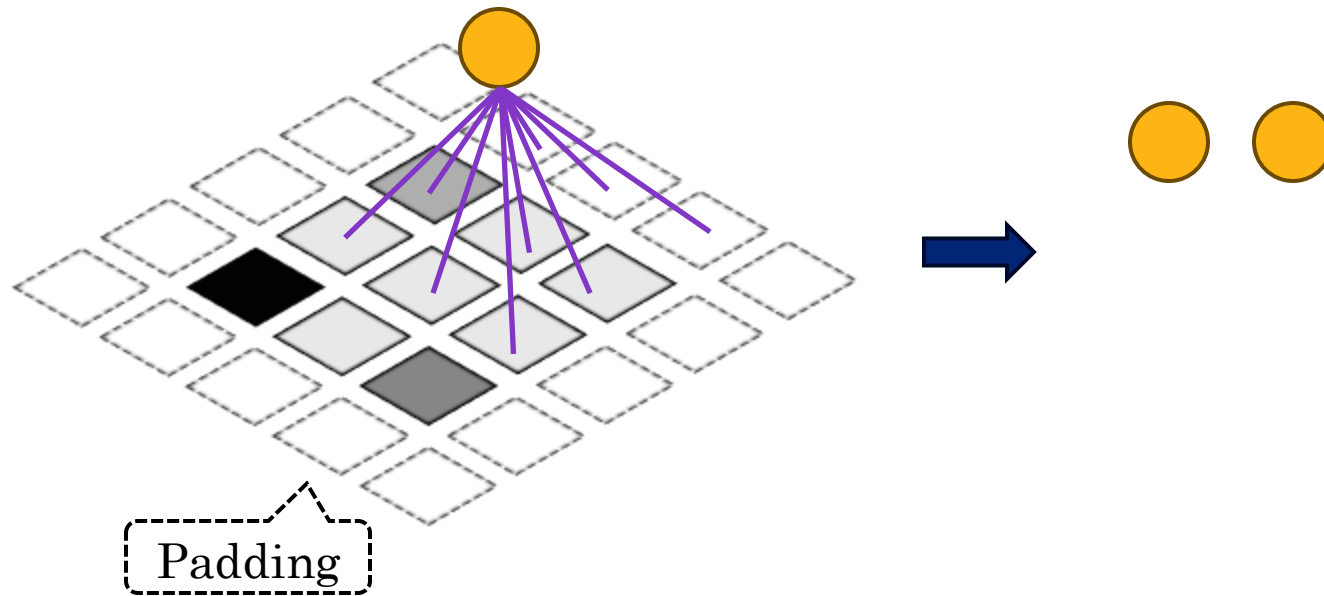
Convolution Networks

“Slide” (convolve) a neuron across the input.



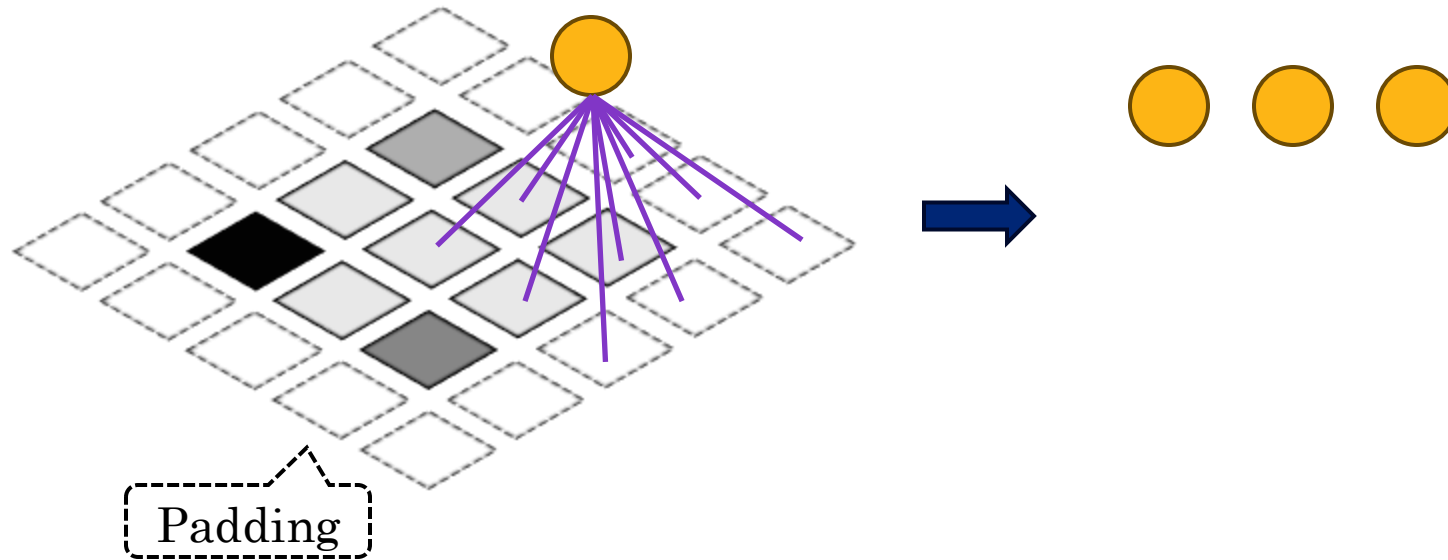
Convolution Networks

“Slide” (convolve) a neuron across the input.



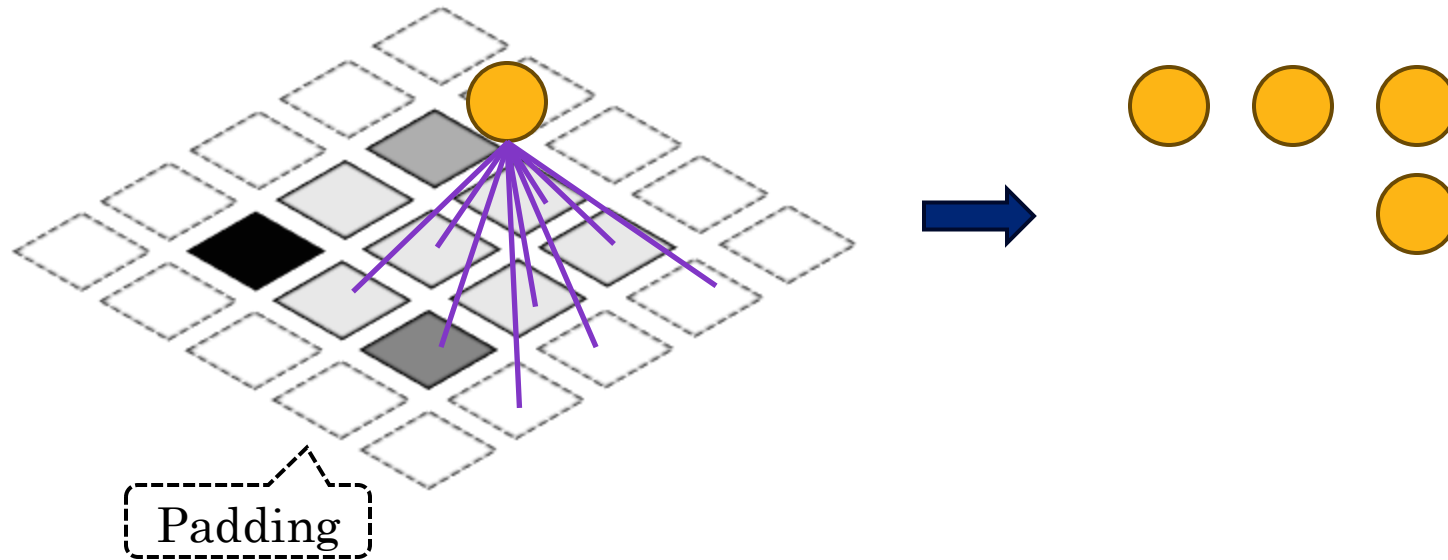
Convolution Networks

“Slide” (convolve) a neuron across the input.



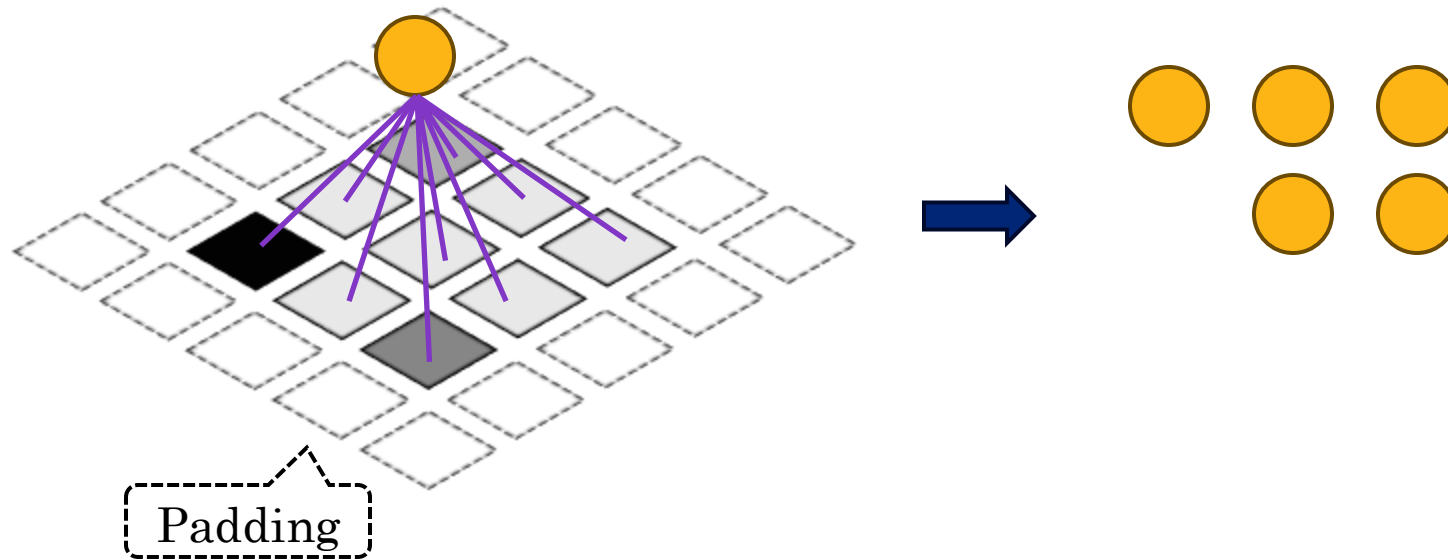
Convolution Networks

“Slide” (convolve) a neuron across the input.



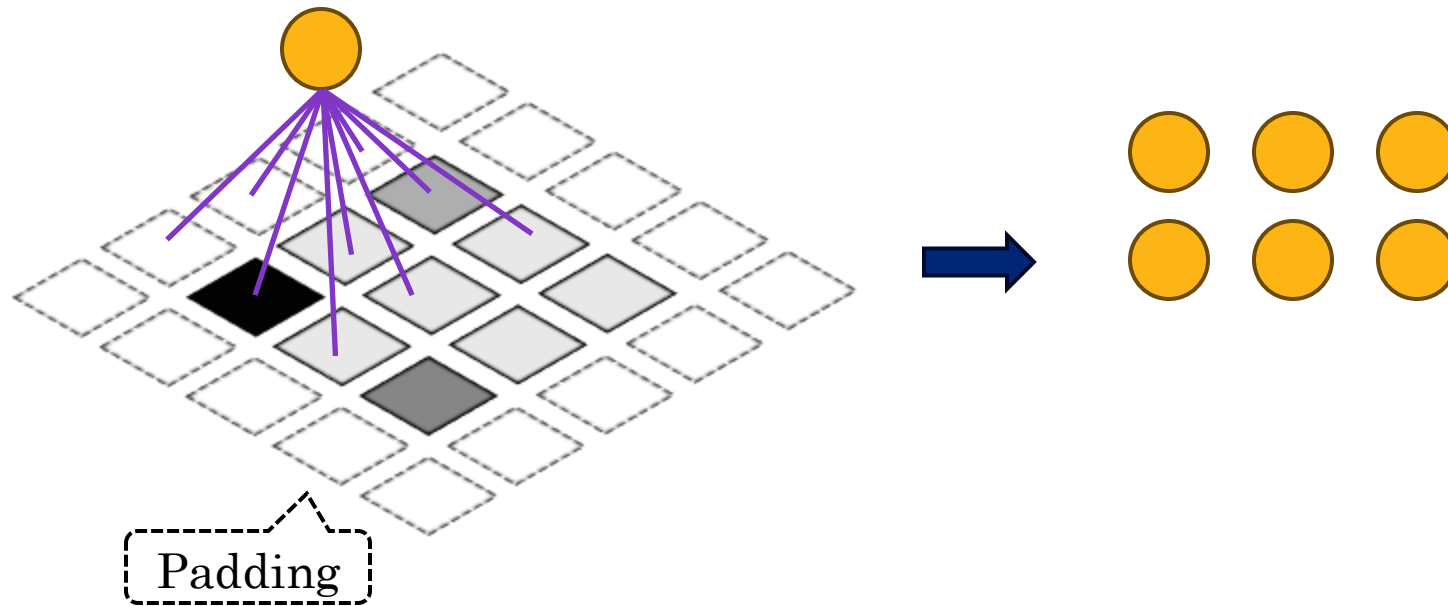
Convolution Networks

“Slide” (convolve) a neuron across the input.



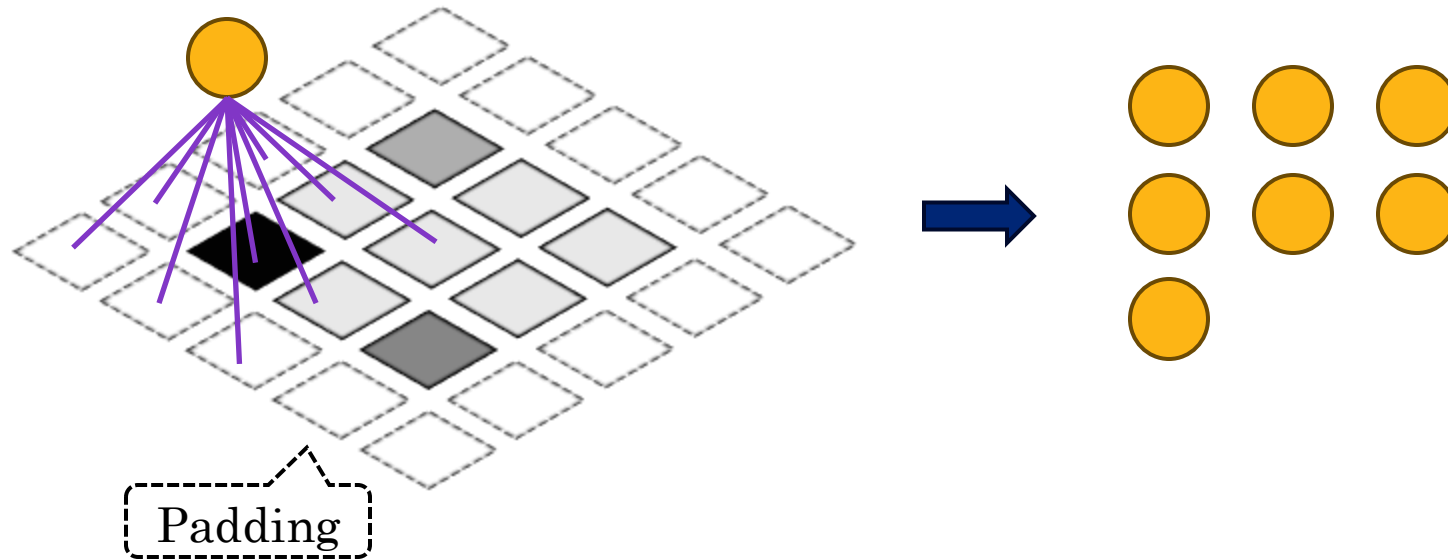
Convolution Networks

“Slide” (convolve) a neuron across the input.



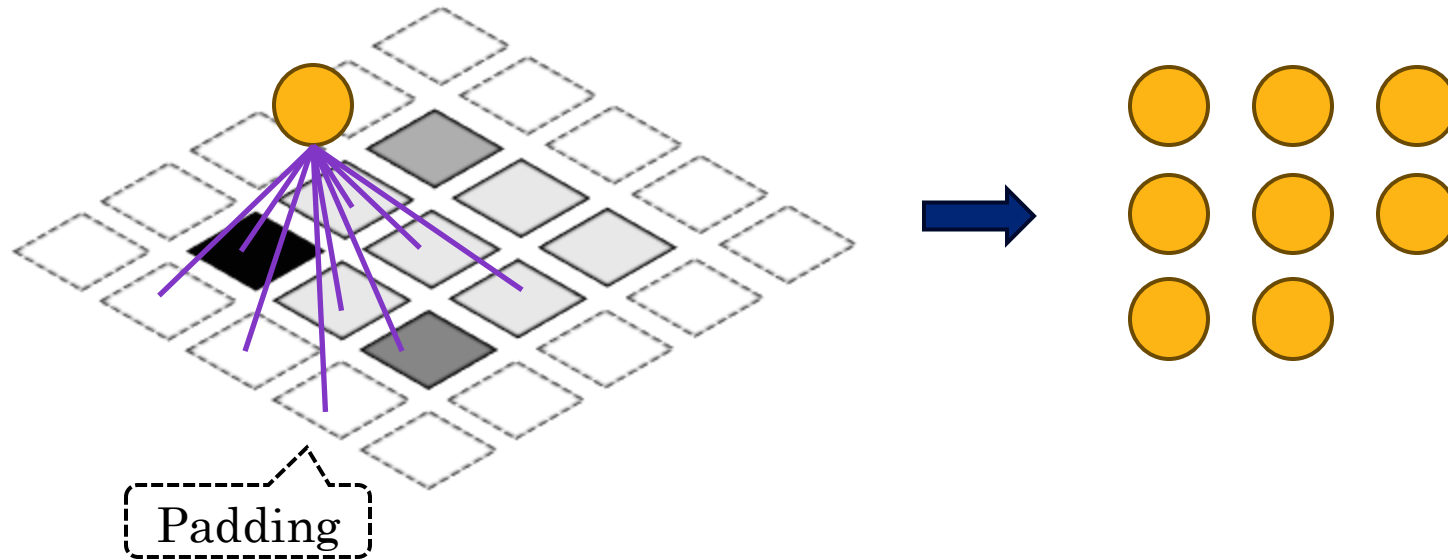
Convolution Networks

“Slide” (convolve) a neuron across the input.



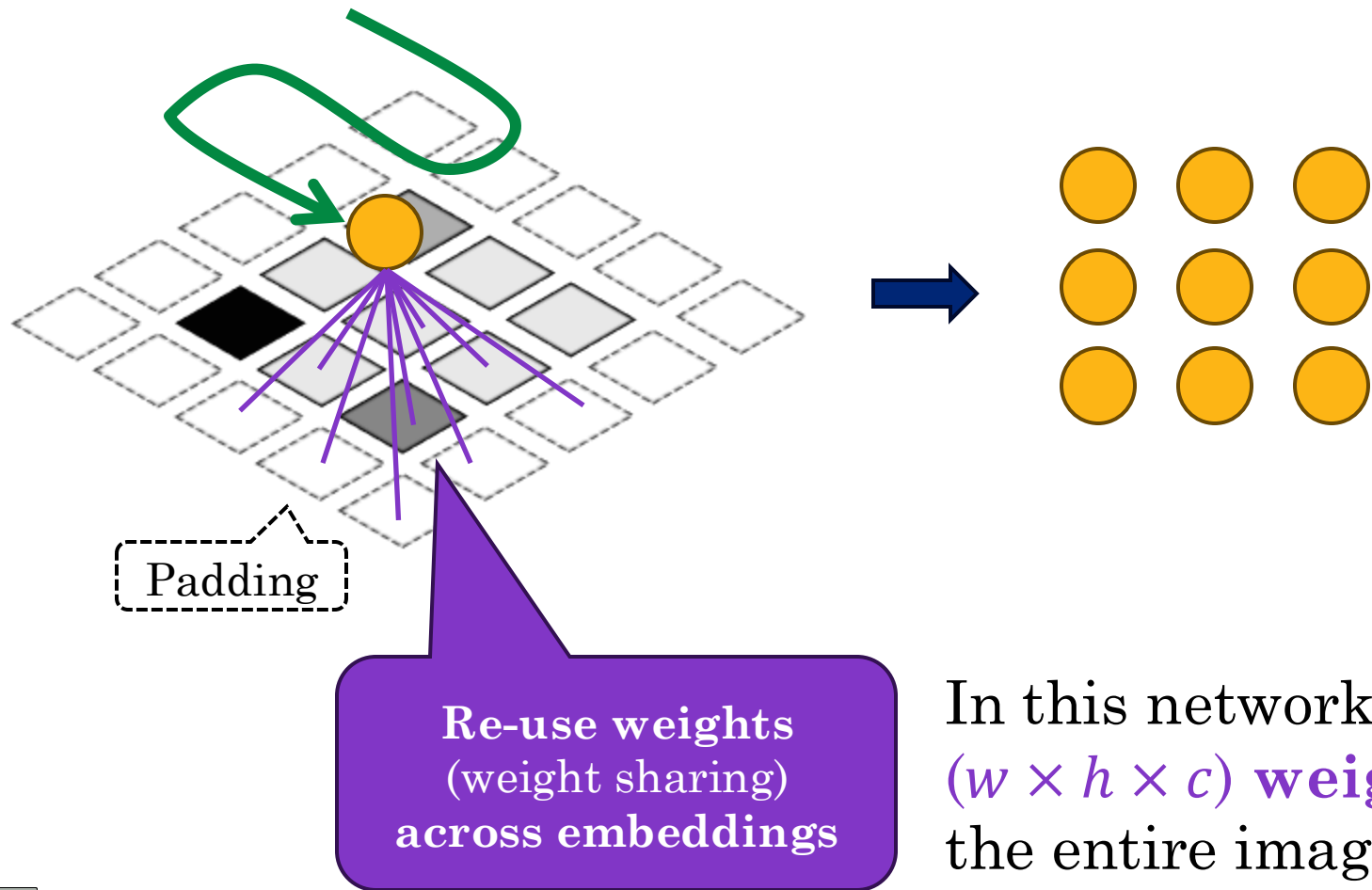
Convolution Networks

“Slide” (convolve) a neuron across the input.



Convolution Networks

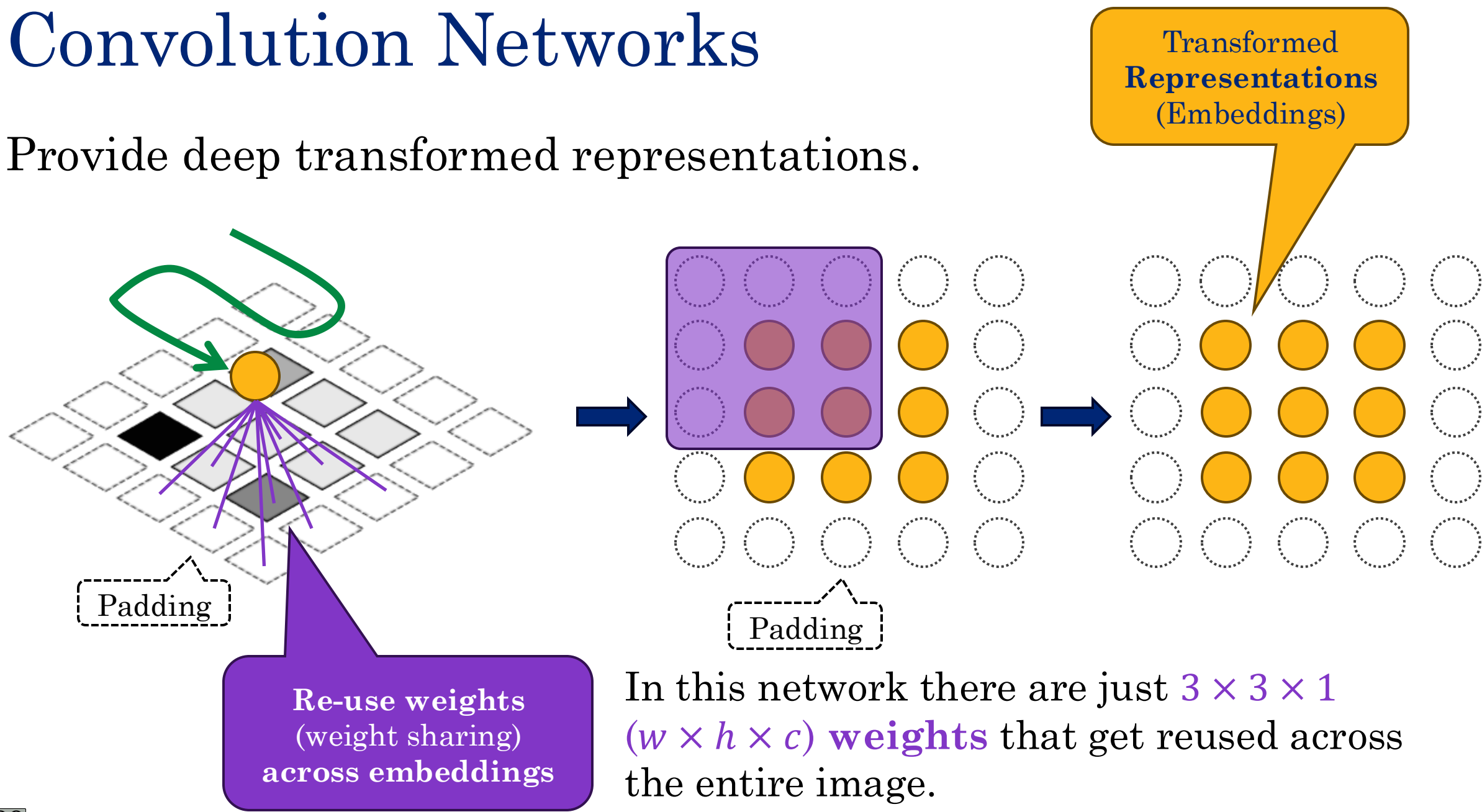
“Slide” (convolve) a neuron across the input.



In this network there are just $3 \times 3 \times 1$ ($w \times h \times c$) **weights** that get reused across the entire image.

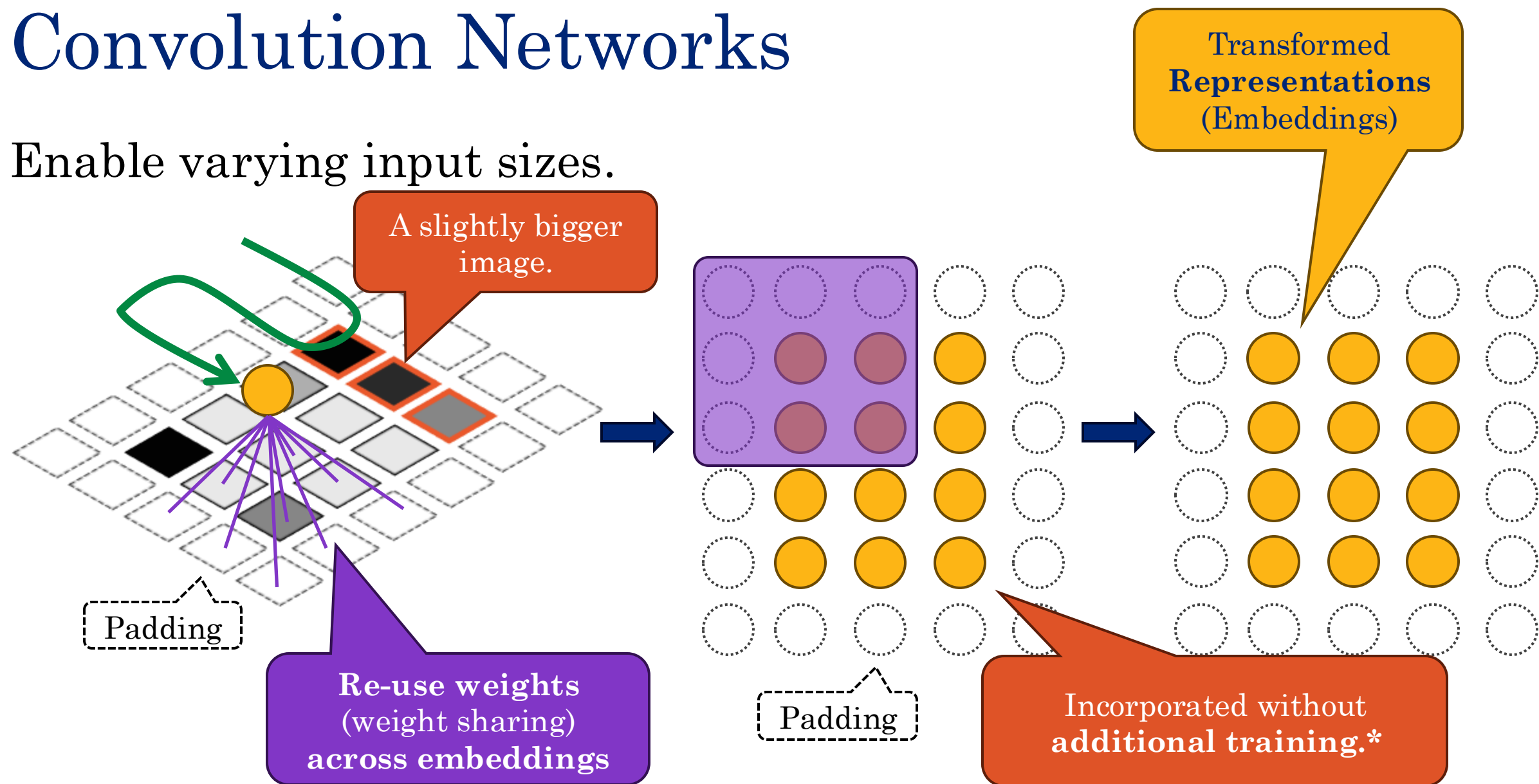
Convolution Networks

Provide deep transformed representations.



Convolution Networks

Enable varying input sizes.



*Significant changes in image resolution may required deeper models and additional training.

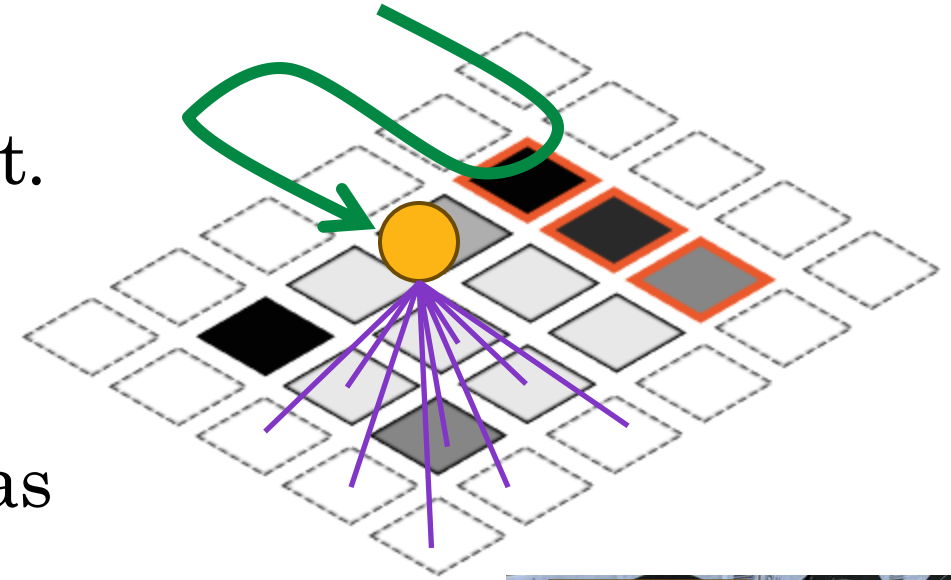
Convolution Networks

“Slide” (convolve) a neuron across the input.

Strengths:

- Parameter efficient
- Encode **spatial invariance** inductive bias
- Enable **varying input dimensions**
 - At least spatially (not channel dimension)

Weaknesses?





What is in this image?

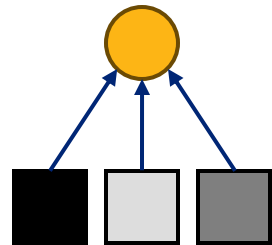
**Broader context
is often required to
understand a part.**



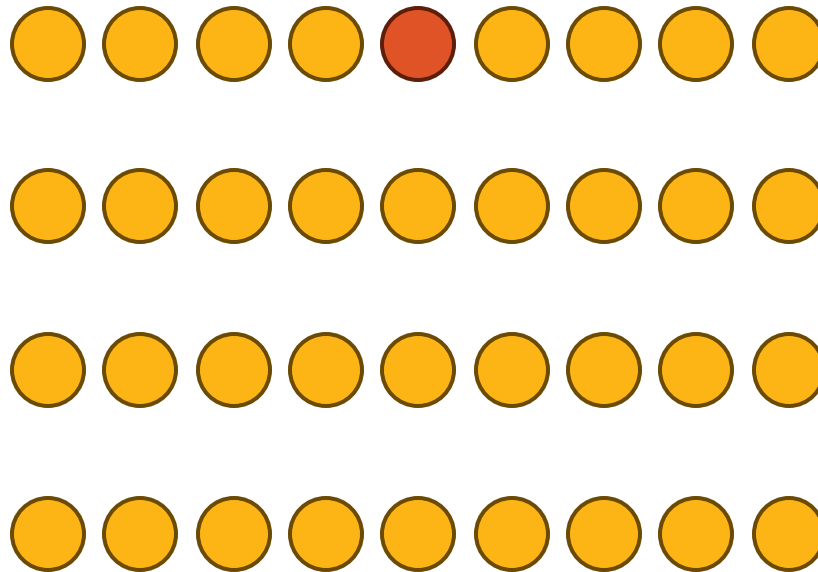
Firefighter
spiderman

Convolution Receptive Field

What is the width of the **receptive field** (in pixels) of the **top center activation** assuming a width 3 convolution **on all layers**

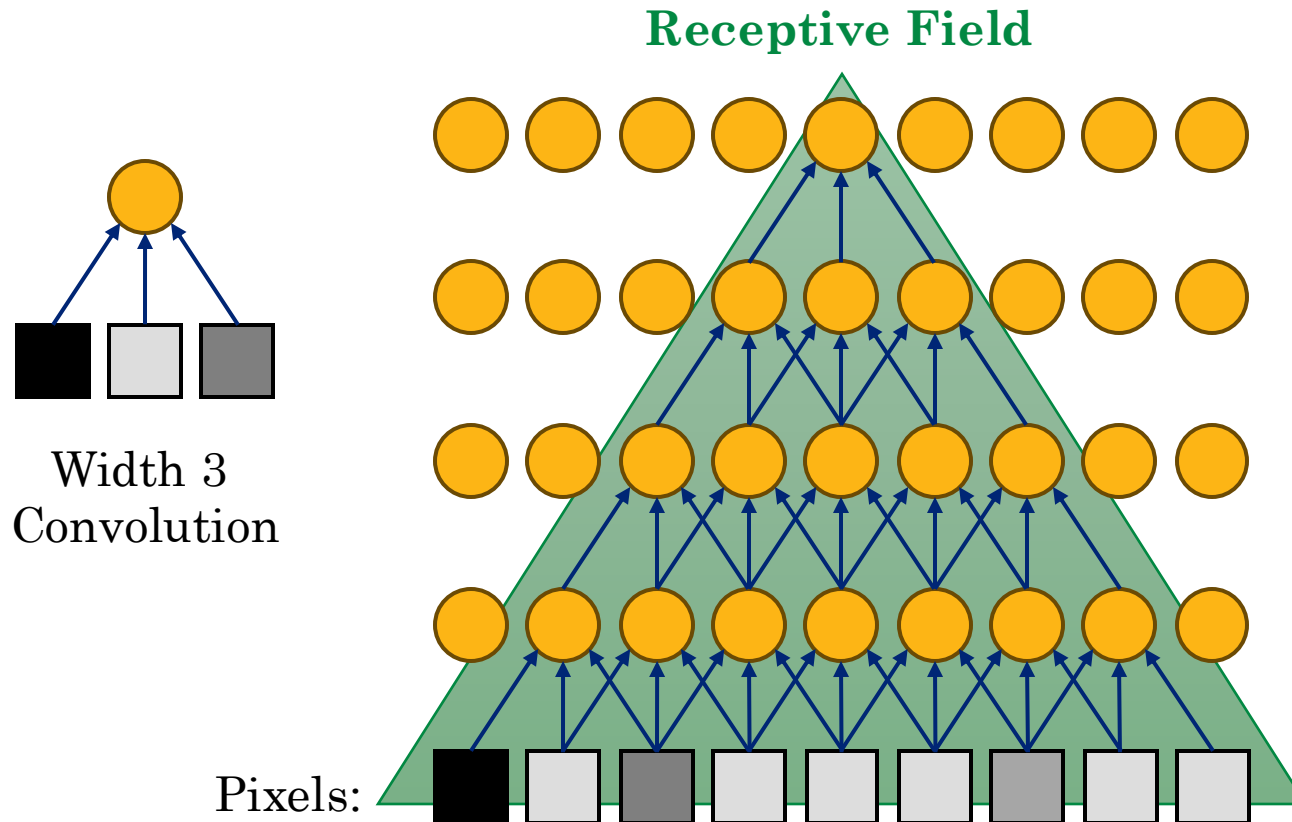


Width 3
Convolution



Convolution Receptive Field

Higher layers in the network see more context.



Convolution Networks

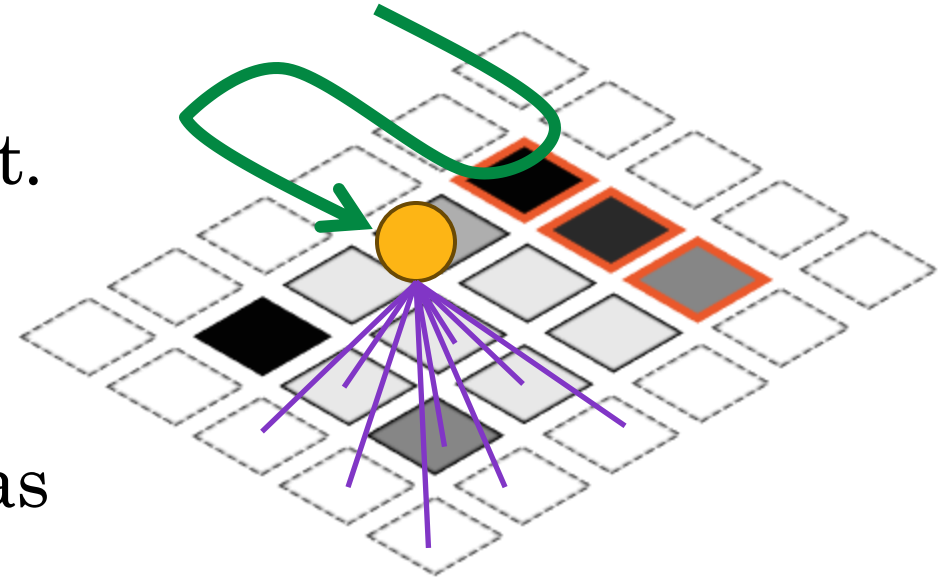
“Slide” (convolve) a neuron across the input.

Strengths:

- Parameter **efficient**
- Encode **spatial invariance** inductive bias
- Enable **varying input dimensions**
 - At least spatially (not channel dimension)

Weaknesses:

- **Builds local representations** and only **top layers** of the network have “**full**” context (can see everything...).



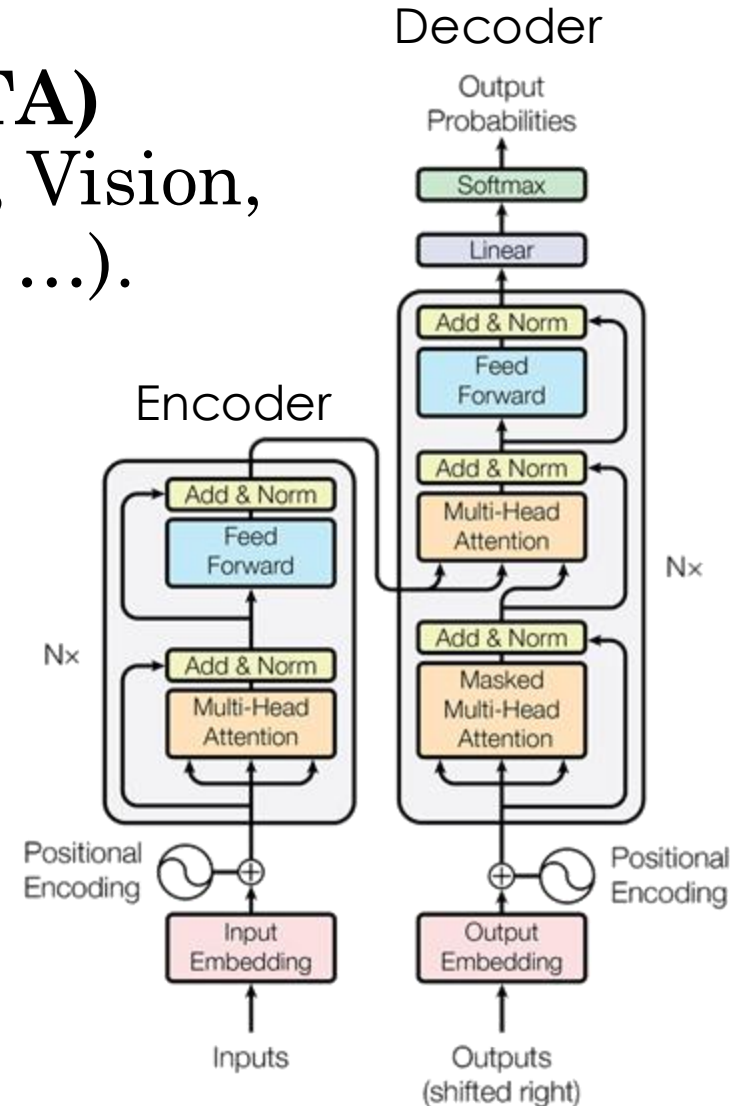
Soln: Transformer Architecture

The basic building block of **state-of-the-art (SoTA) neural networks** for **most domains** (e.g., Chat, Vision, Speech Recognition, Image and Video Generation ...).

Originally introduced for **machine translation**.

- [Attention is All You Need](#) (Read it!)
- “Somewhat” difficult to follow...

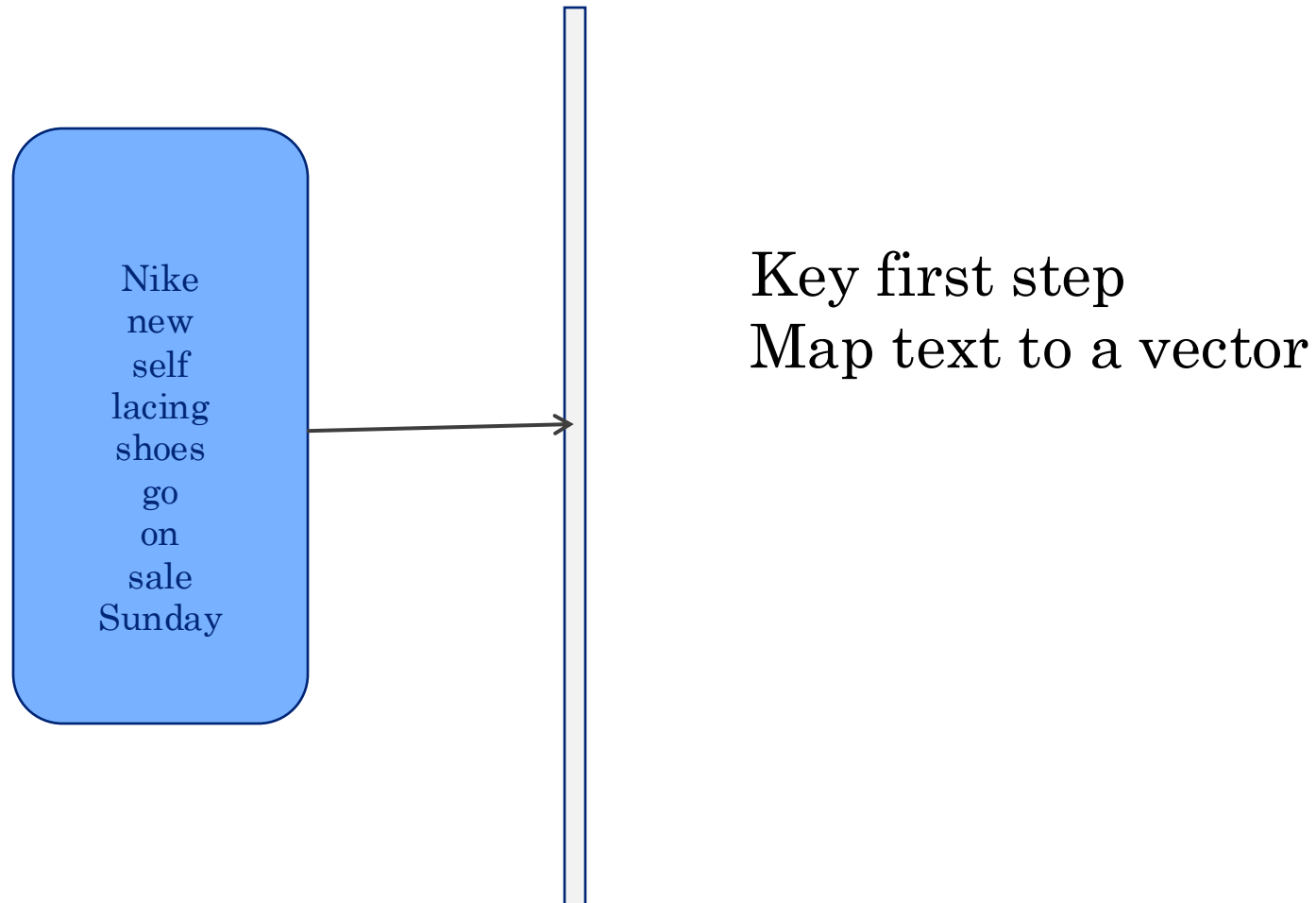
We will try to build the Transformer from **first principles** spanning **language** and **visual modeling**



Roadmap

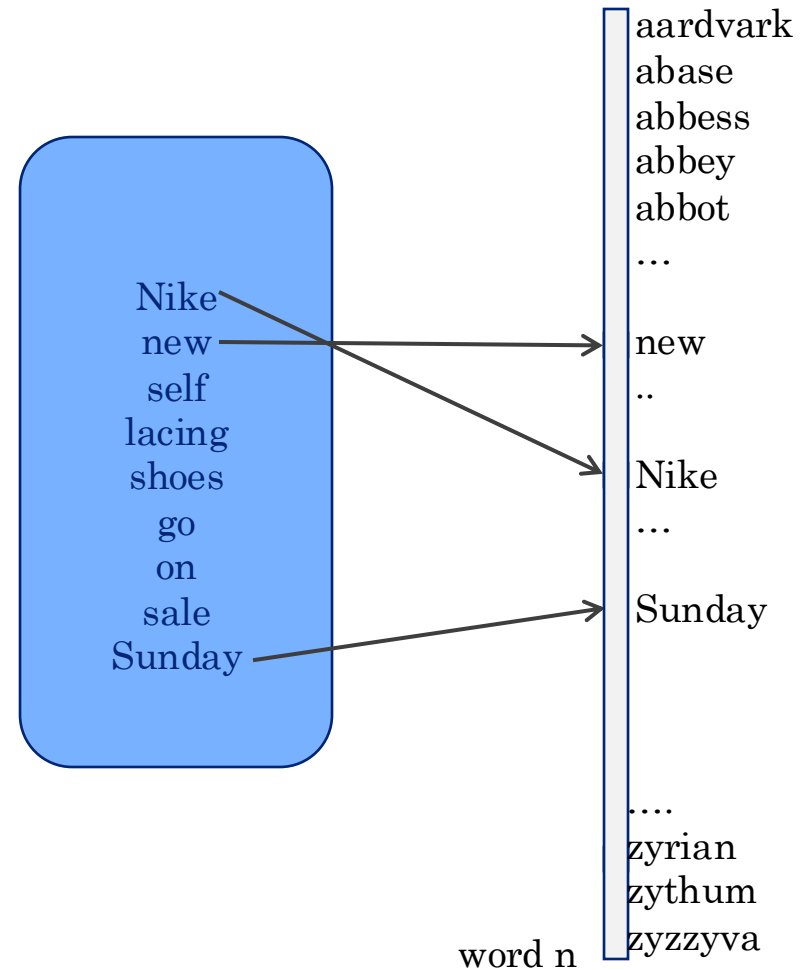
- Representations of word documents:
 1. Bag of words
 2. TF-IDF
- Introduction to RNNs. Attention in RNNs.
- Introduction to Attention, Self-Attention, Transformer

Representing documents as vectors



Documents to vectors

Each tweet as a long (50K), super-sparse vector (5-10 non-zeros)
with 1s in word indices



Roadmap

- Representations of word documents:
 1. ~~Bag of words~~
 2. **TF-IDF**
- Introduction to RNNs
- Introduction to Attention, Self-Attention, Transformer

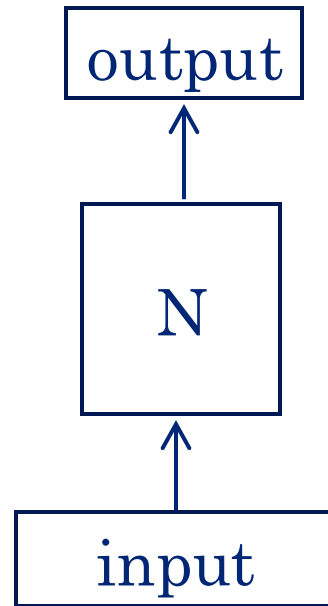
Representing documents with bag of words

- Frequent words will appear with higher likelihoods:
- ‘The’, ‘and’, etc..
- Would be better to have a representation that represents words that have atypically high frequencies.

TF-IDF

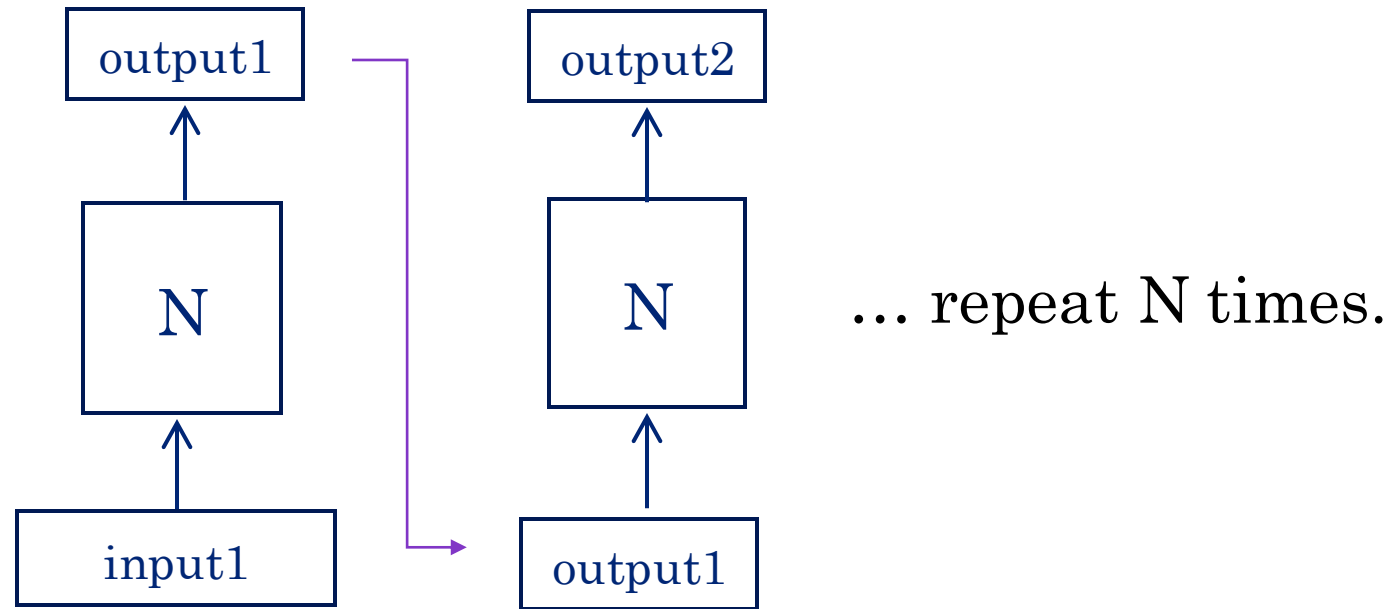
- For each word i , compute $TF\text{-}IDF(i) = TF(i) * IDF(i)$
- $TF(i)$: Term frequency = # times word i appears in a document / # words in doc
- $IDF(i)$ =(Inverse Document Frequency): How uncommon word i is (in a group of docs)
- $IDF(i) = \text{Log}(\text{Total number of documents} / \text{number of documents with word } i)$
- Say a document contains 100 words. The word football appears 2 times. So $TF = 2/100$. Now assume that in 1000 documents, the word football appears in 300 of them.
- $IDF = \log(1000/300)$.
- So $TF\text{-}IDF = TF * IDF = 2/100 * \log(1000/300)$

What is a Recurrent Neural Network (RNN)



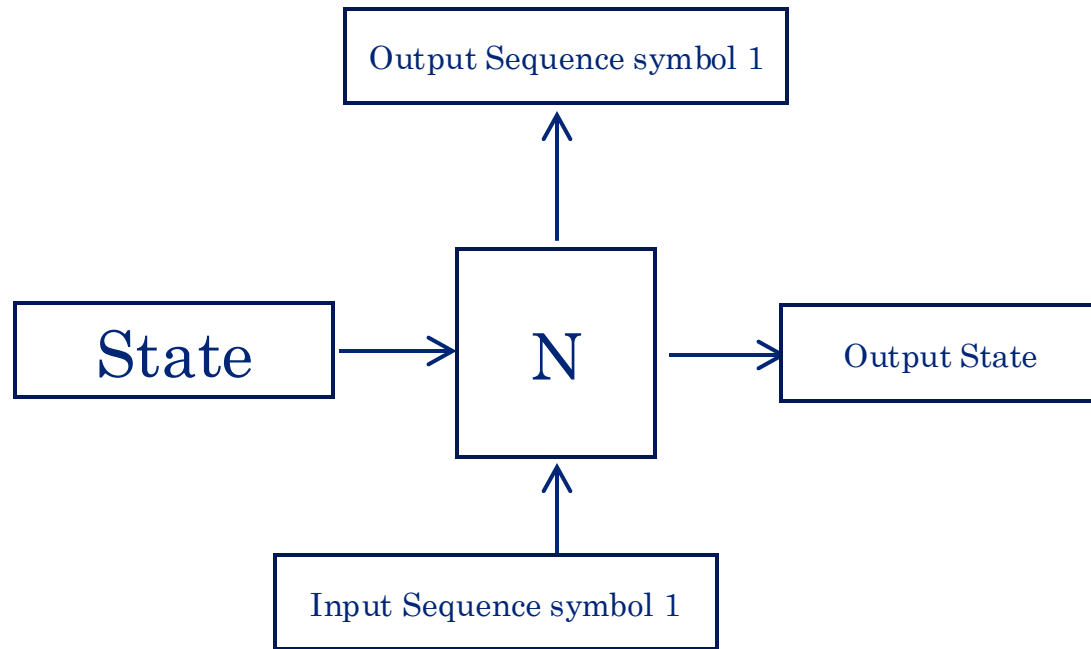
Feed-forward Network
(Can have fully connected layers (MLPs)
or Convolutional and linear layers etc).

What is a Recurrent Neural Network (RNN)



A recurrent neural network is feeding its output as input iteratively

What is a Recurrent Neural Network (RNN)



Typically there are two inputs: Previous state and previous output.
Used to process a sequence and produce another sequence.
(E.g. Time series, Text etc.)

Baseline: An RNN for captioning



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



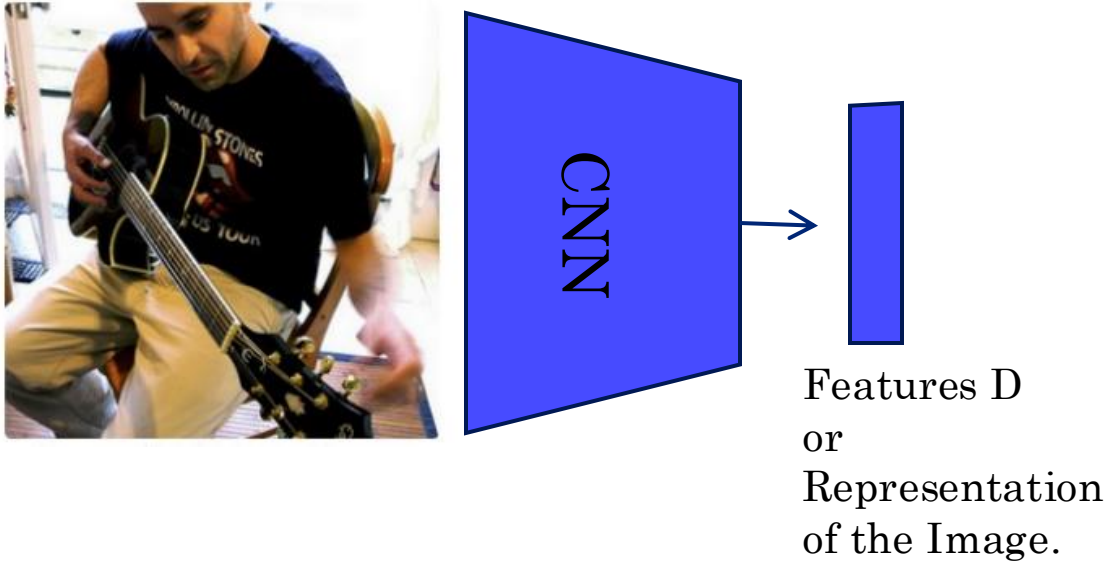
"two young girls are playing with lego toy."



"boy is doing backflip on wakeboard."

Dataset Microsoft COCO (2014) 120K Images and 5 sentence descriptions for each image.

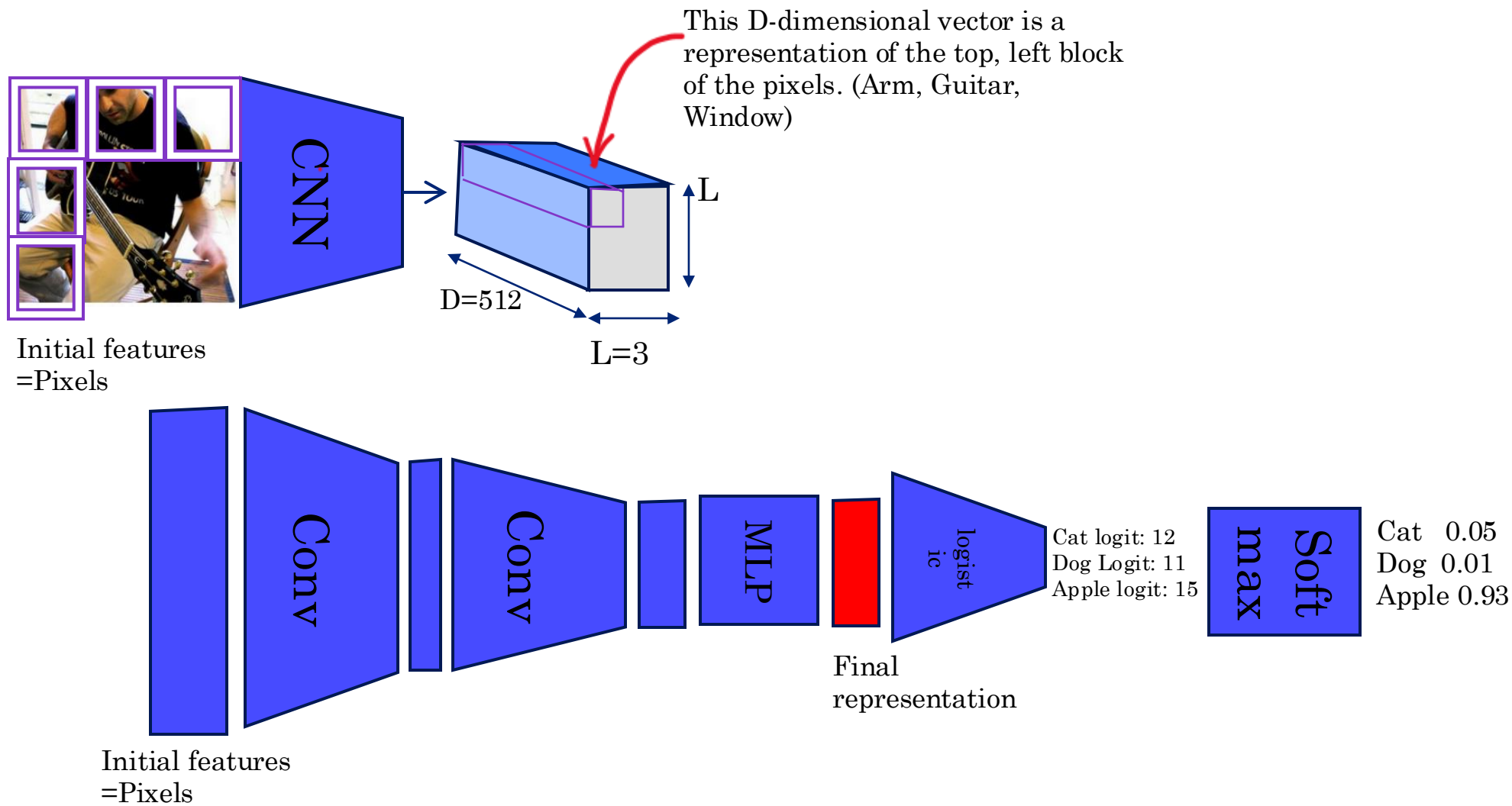
Baseline: An RNN for captioning



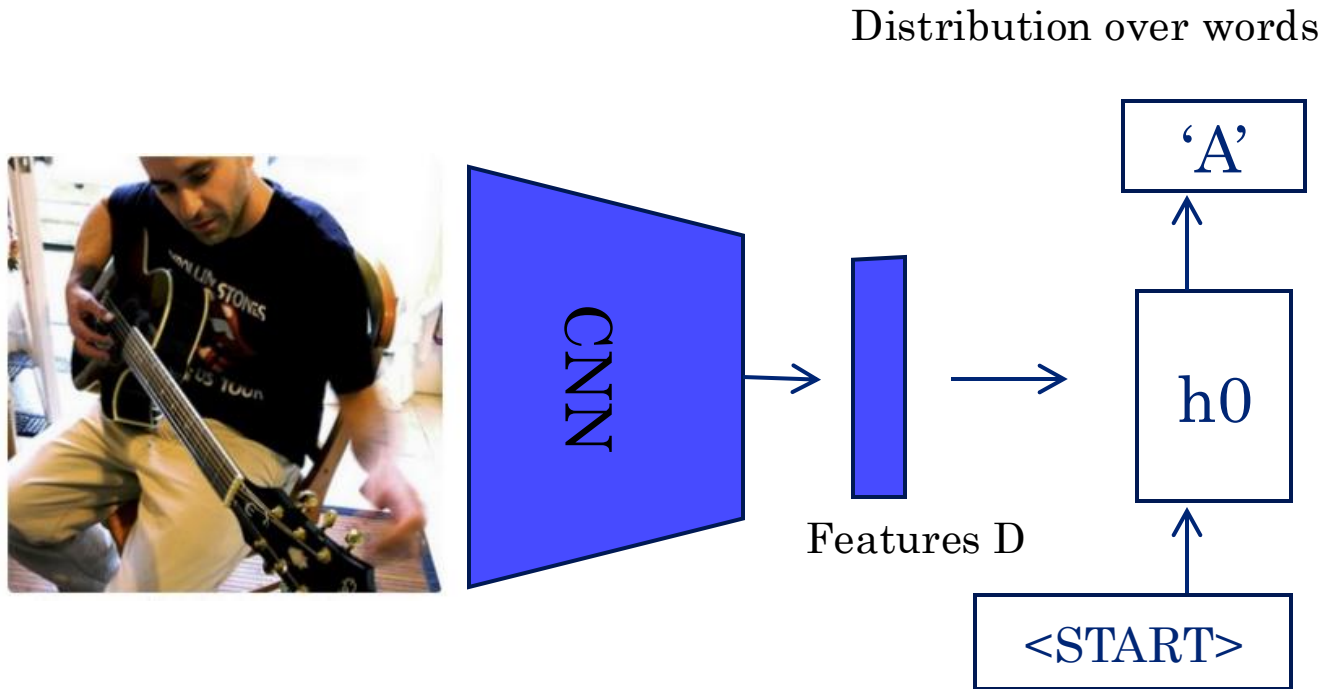
Deep Visual semantic Alignments for generating Image Descriptions, Karpathy and Fei-Fei, CVPR'15

Show and Tell: A neural Image caption Generator, Vinyals et al. , CVPR'15.

Understanding that CNN produces a feature representation of a block of the image

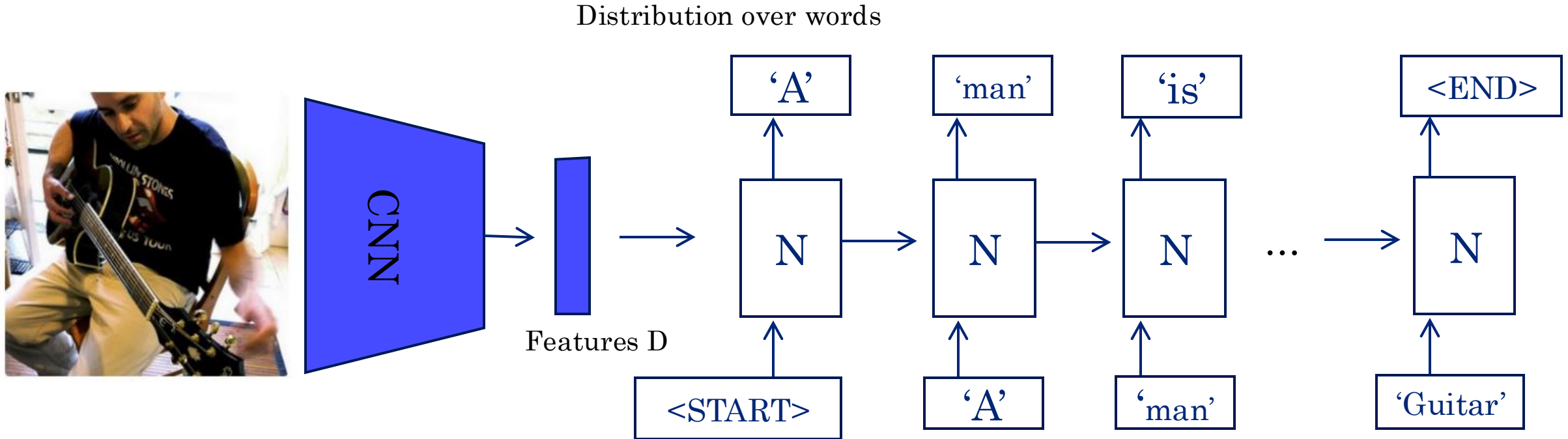


Baseline: An RNN for captioning

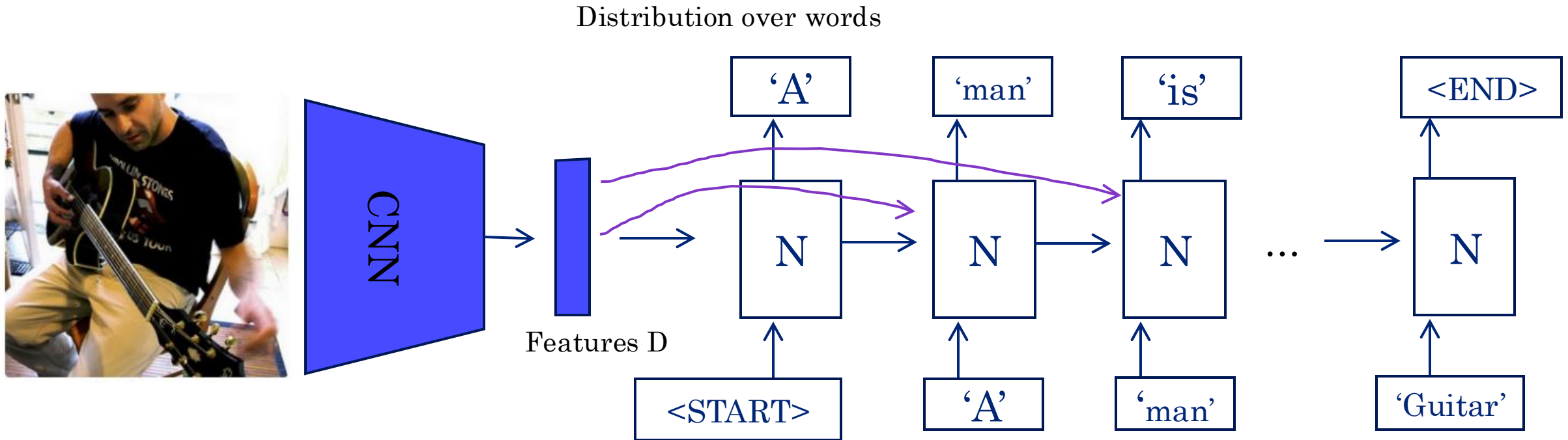


Deep Visual semantic Alignments for generating Image Descriptions, Karpathy and Fei-Fei Li, CVPR'15
Show and Tell: A neural Image caption Generator, Vinyals et al. , CVPR'15.

Baseline: An RNN for captioning

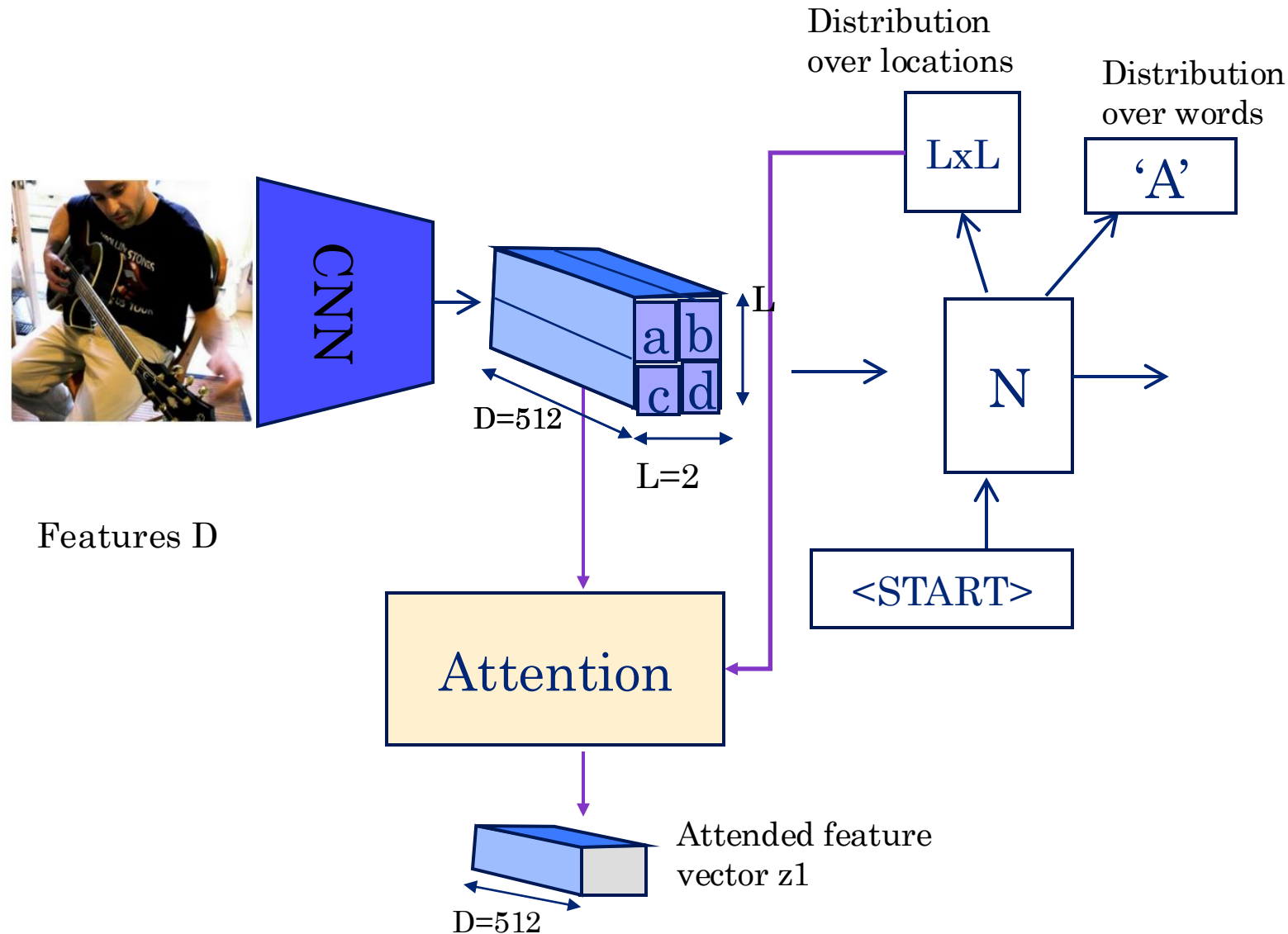


Baseline: An RNN for captioning



You could try to feed the image in every stage of the RNN

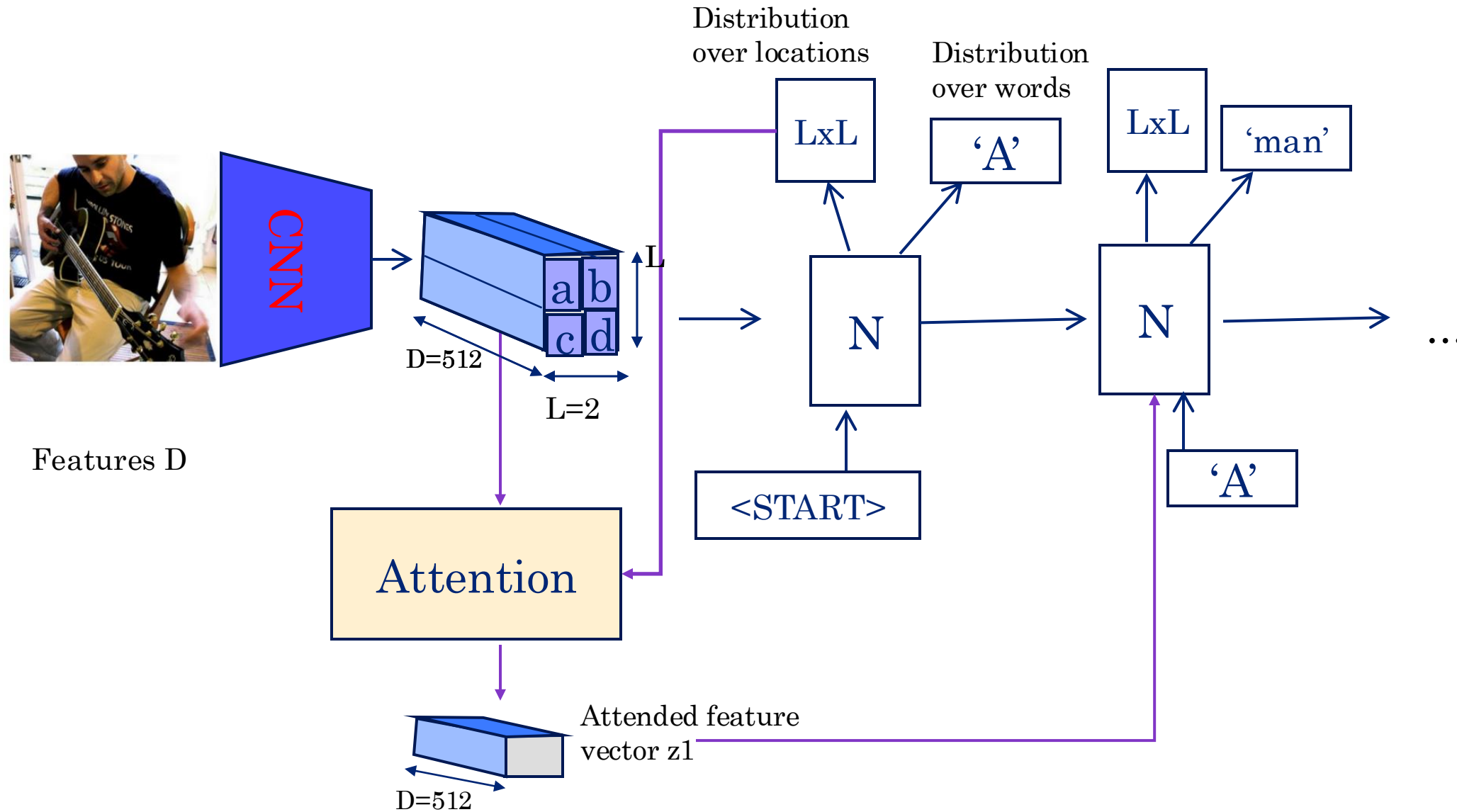
Adding soft attention



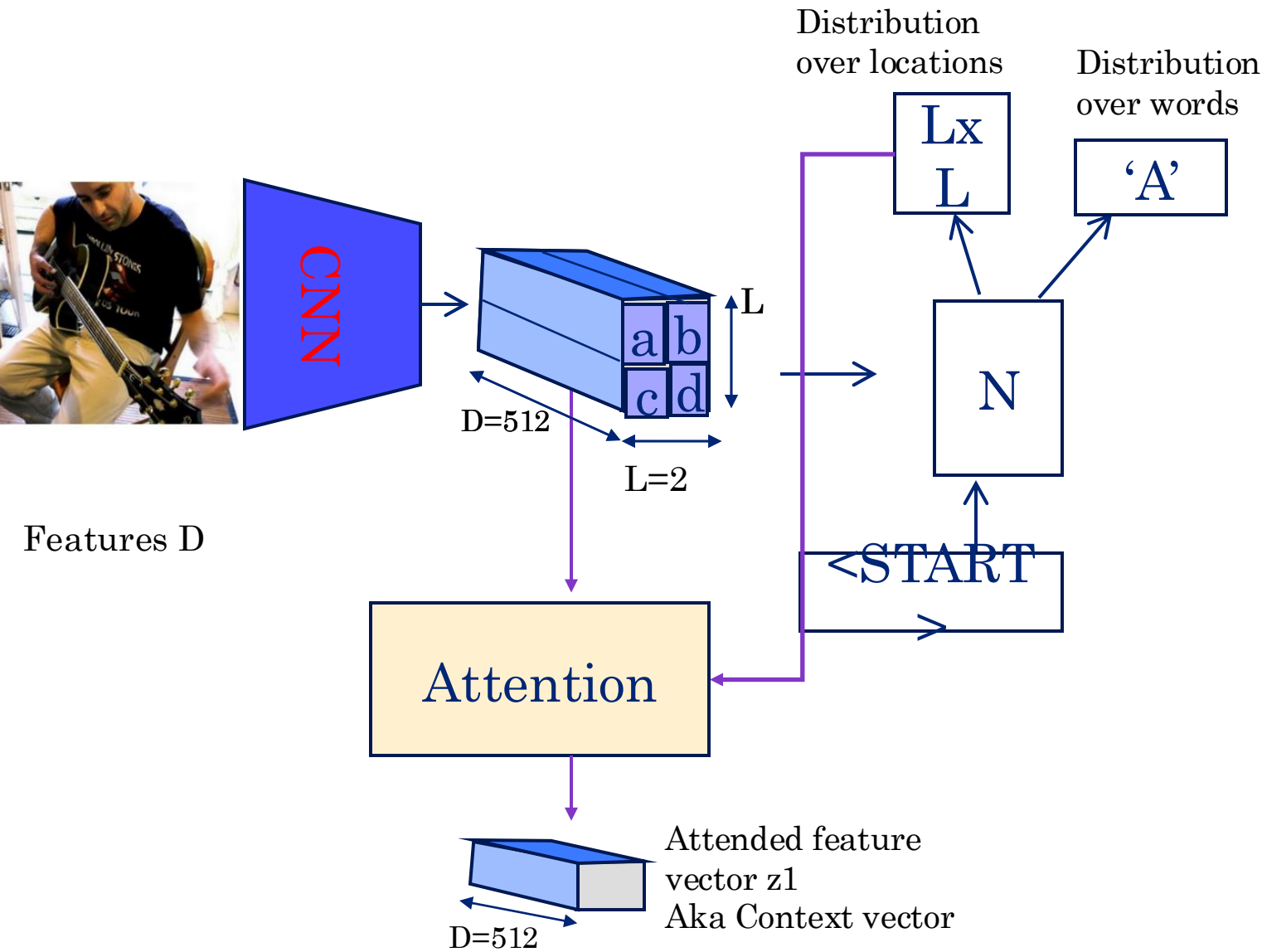
Show, **Attend** and Tell, Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard Zemel, Yoshua Bengio ICML 2015.

Bahdanau, Dzmitry; Cho, Kyunghyun; Bengio, Yoshua
"Neural Machine Translation by Jointly Learning to Align and Translate"
ICLR 2015.

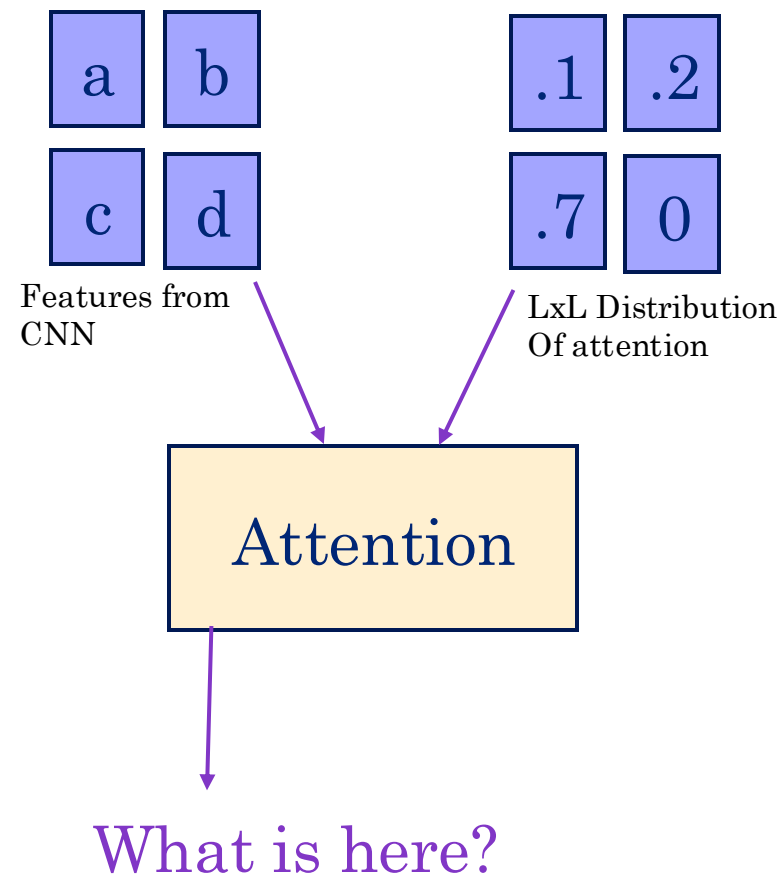
Adding soft attention



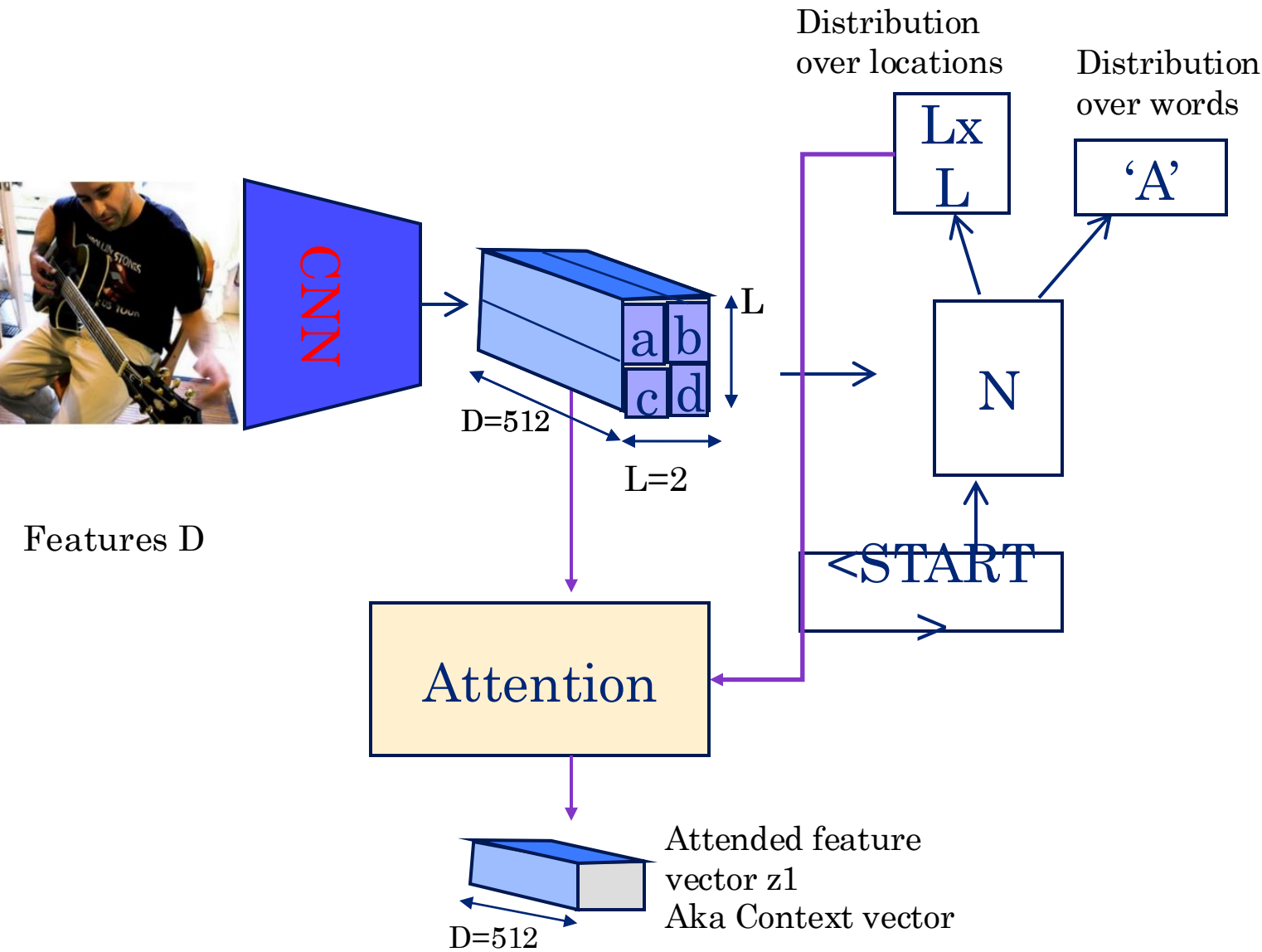
Soft vs Hard attention



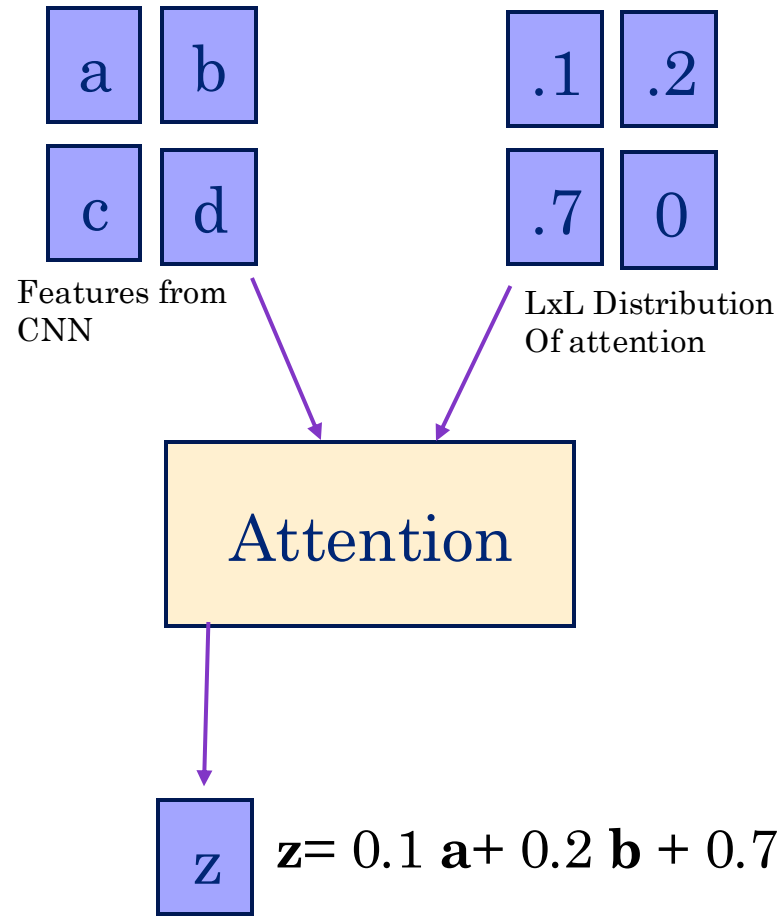
Say $L=2$
Soft attention:
 z_1 is a D -dimensional vector



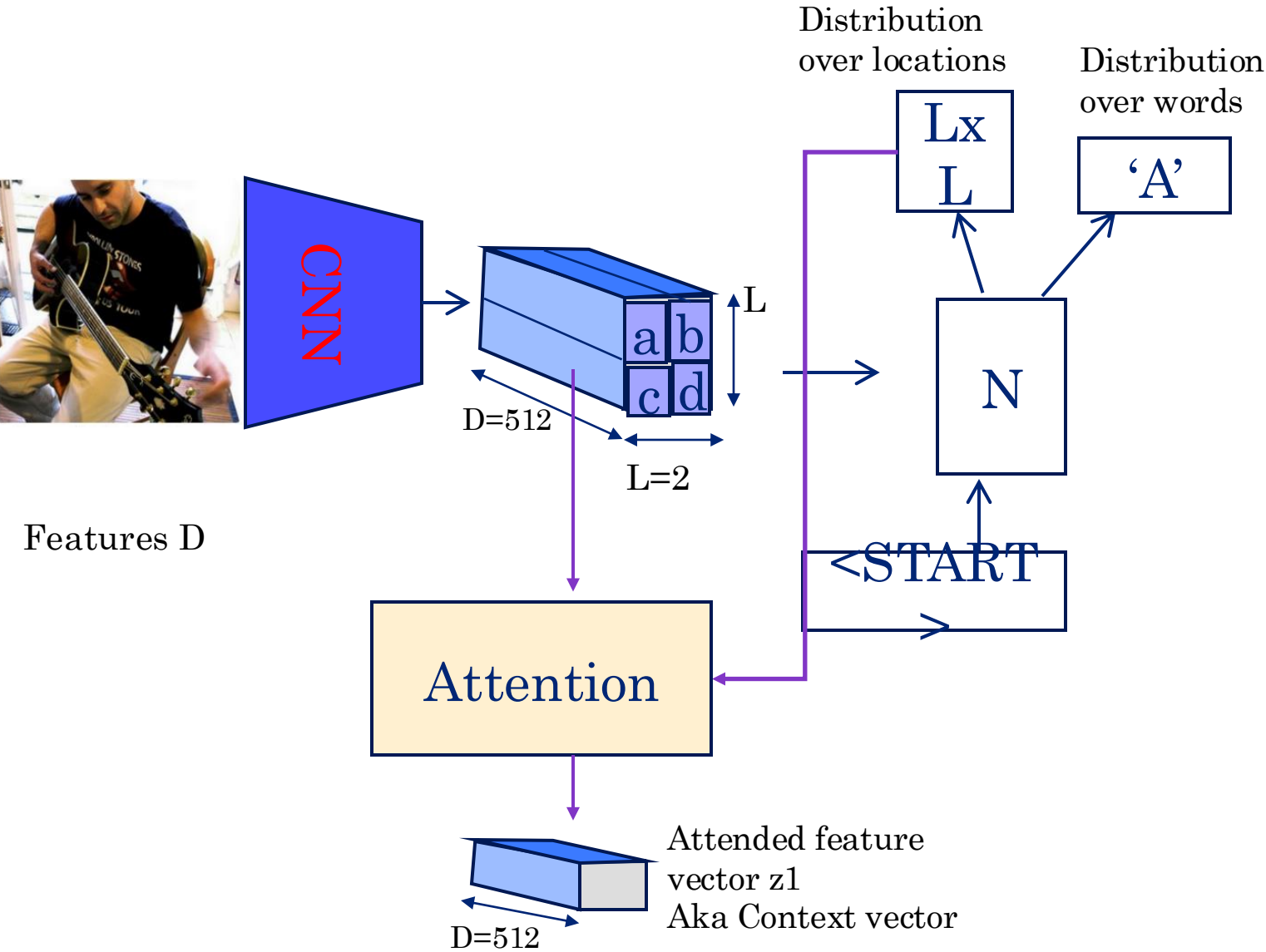
Soft vs Hard attention



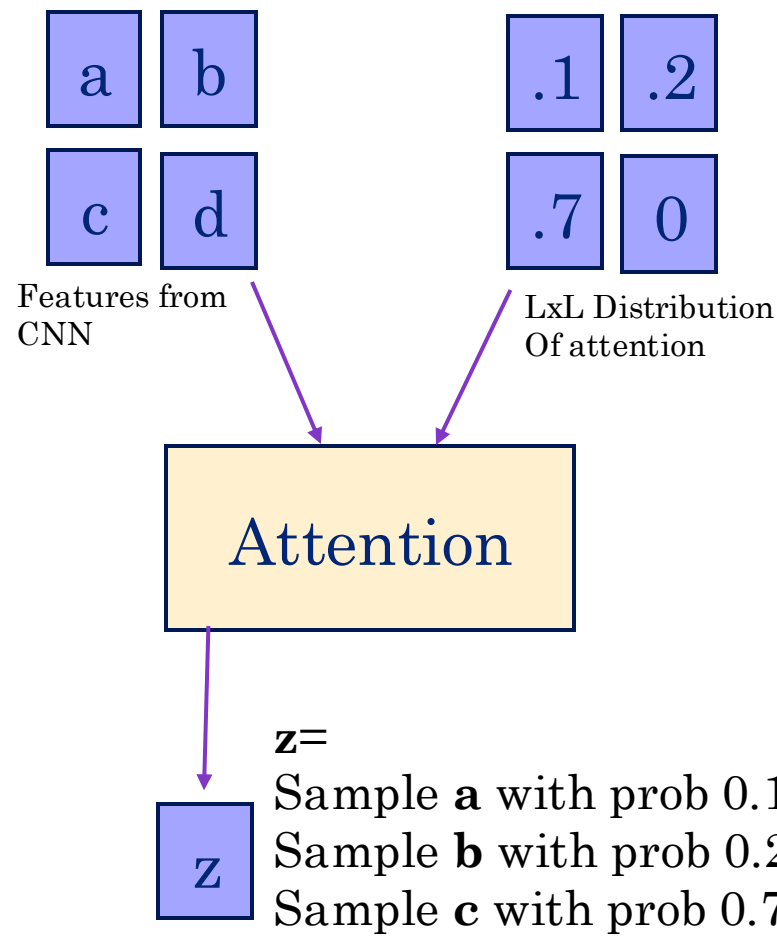
Say $L=2$
Soft attention:
 z_1 is a D -dimensional vector



Soft vs Hard attention



Say $L=2$
Hard attention:
 z_1 is a D -dimensional vector



Soft vs Hard attention



Soft attention is end-to-end differentiable so can be trained.

Hard attention is not.

So must use REINFORCE algorithm to train.

Distribution over locations

$L \times L$

Distribution over words

'A'

Attended feature vector z_1

Aka Context vector

$D=512$

Say $L=2$

Hard attention:

z_1 is a D -dimensional vector

a	b	.1	.2
c	d	.8	0

Features from CNN

$L \times L$ Distribution Of attention

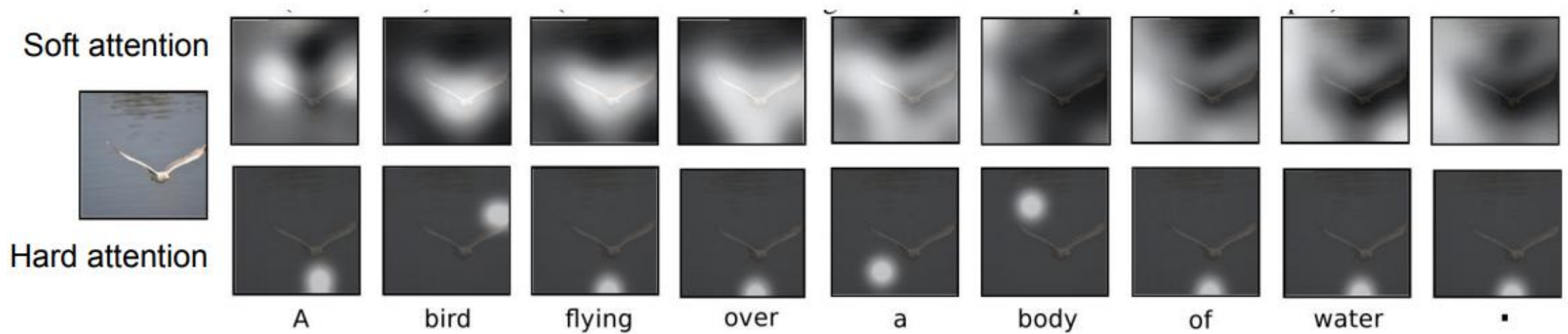
Attention

$z =$

Sample **a** with prob 0.1

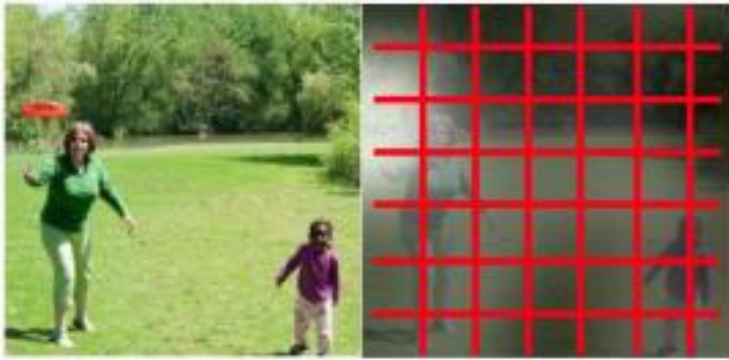
Sample **b** with prob 0.2

Interpretable attention



Show, **Attend** and Tell, Xu et al. ICML 2015.

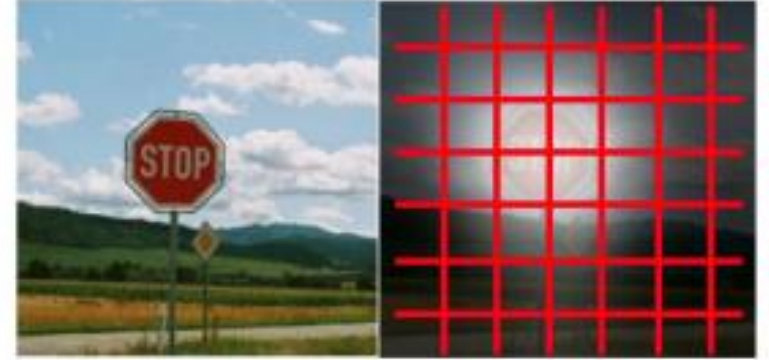
Interpretable attention



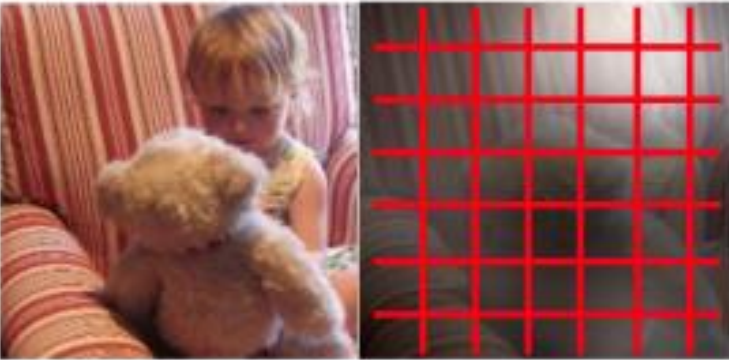
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



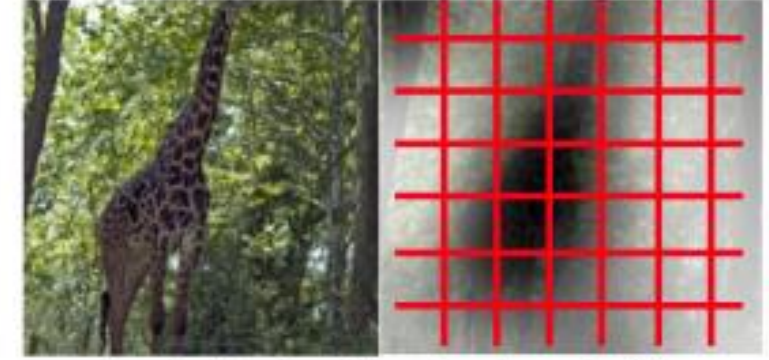
A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.

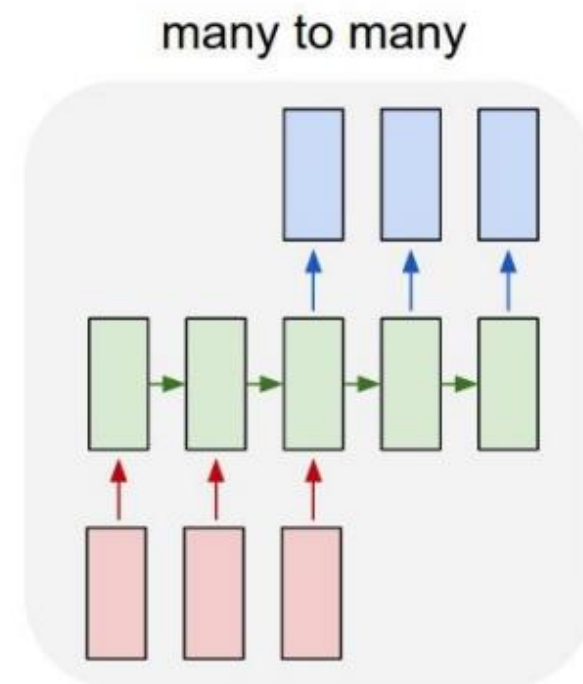
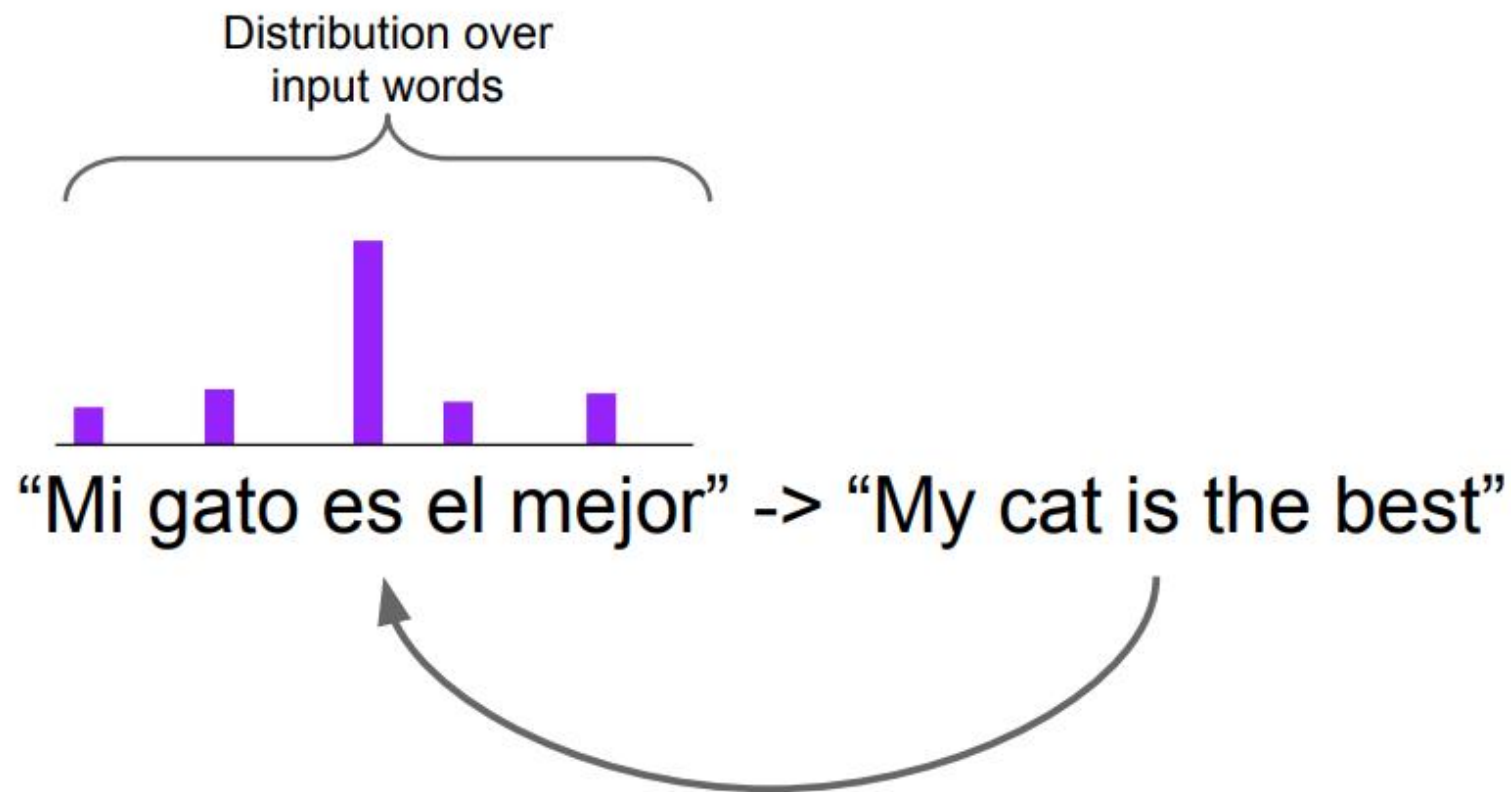


A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Soft attention for Translation



Bahdanau et al. Neural Machine Translation by Jointly Learning to Align and Translate, ICLR 2015.

Soft attention for Everything

Machine Translation, attention over input:

- Luong et al, "Effective Approaches to Attention-based Neural Machine Translation," EMNLP 2015

Speech recognition, attention over input sounds:

- Chan et al, "Listen, Attend, and Spell", arXiv 2015
- Chorowski et al, "Attention-based models for Speech Recognition", NIPS 2015



+Local+Global: **Someone** is **frying** a **fish** in a **pot**

Video captioning, attention over input frames:

- Yao et al, "Describing Videos by Exploiting Temporal Structure", ICCV 2015

What season does this appear to be?
GT: fall Our Model: fall



What is soaring in the sky?
GT: kite Our Model: kite

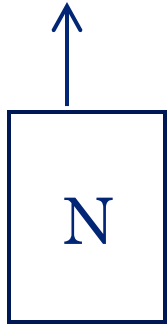


Image, question to answer, attention over image:

- Xu and Saenko, "Ask, Attend and Answer: Exploring Question-Guided Spatial Attention for Visual Question Answering", arXiv 2015
- Zhu et al, "Visual7W: Grounded Question Answering in Images", arXiv 2015

Location-based addressing Attention

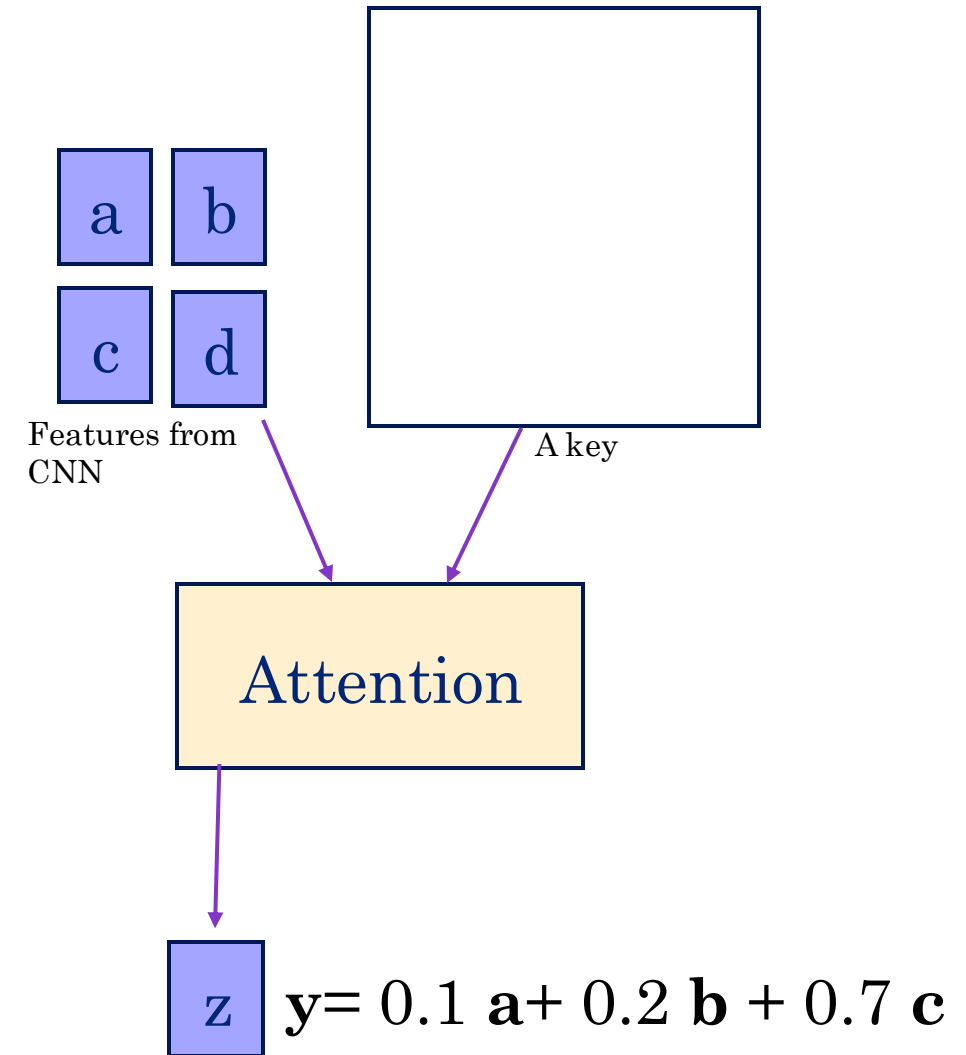
Distribution
over locations



The RNN outputs
a distribution
over a fixed
number of
locations.

What if the
number of
locations is not
fixed a-priori?

Content-based
addressing



Stopped Here

We cover content-based attention in Lecture 22.

Content based attention.

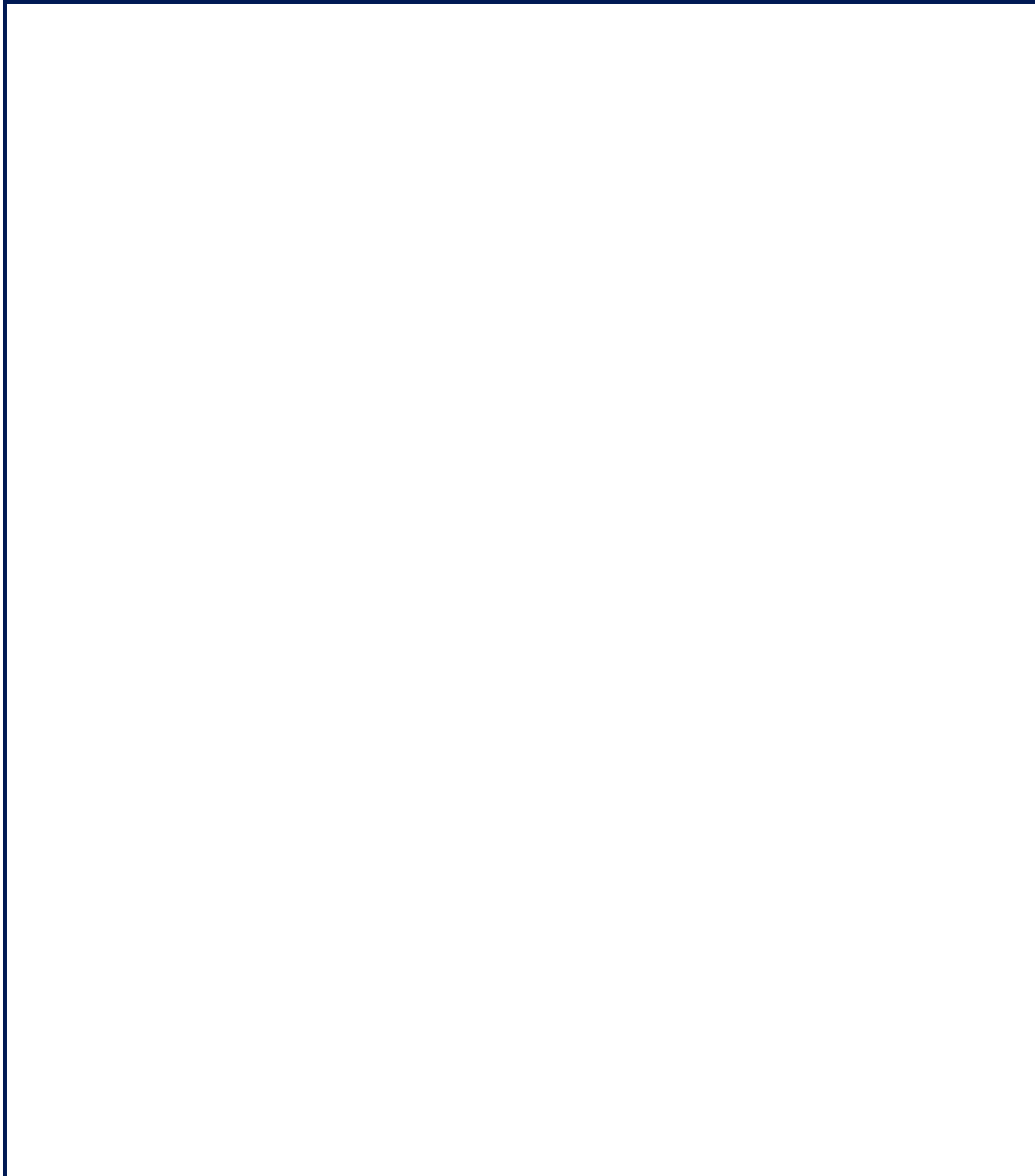
Lecture 22: Today we cover
what is Self-Attention,

How to implement it in
Pytorch

Interpretation as a soft
lookup

What is the transformer

Query is:



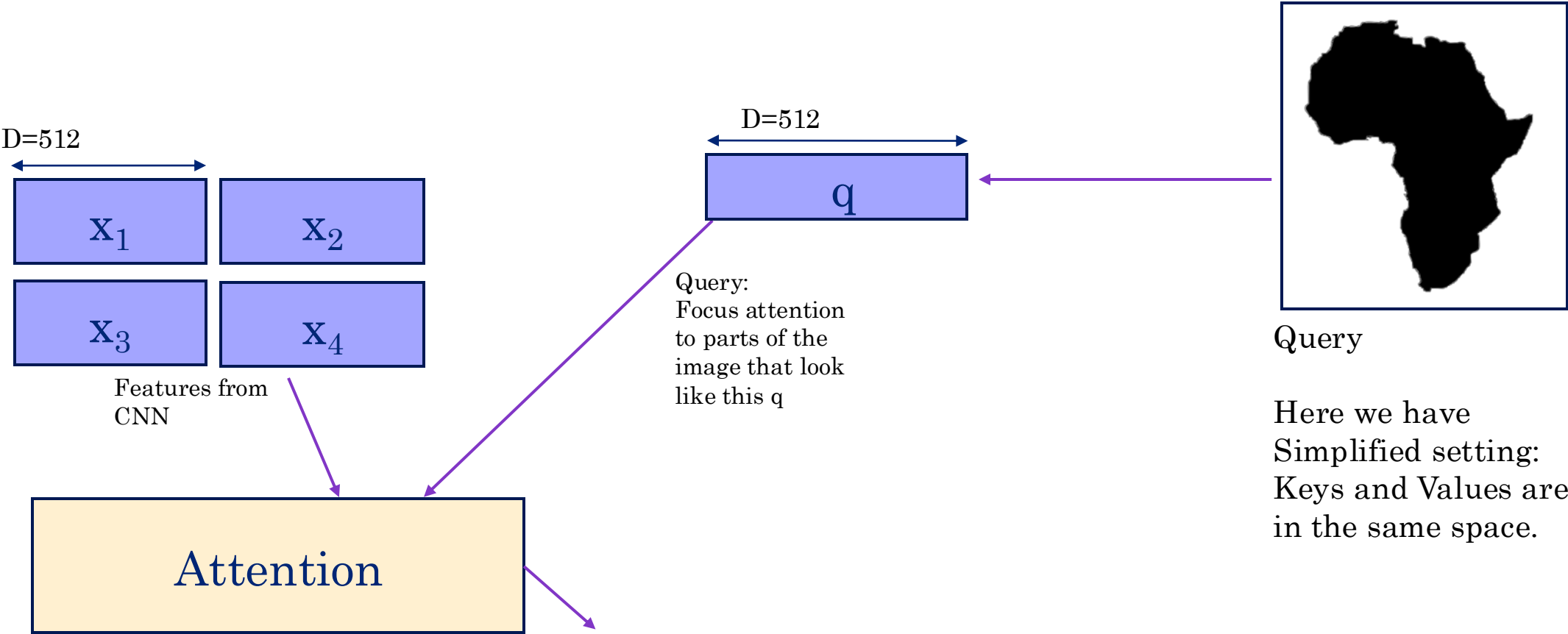
Content based attention.

Query
is:

Today we cover
what is Self-Attention,
How to implement it in
Pytorch
Interpretation as a soft
lookup
What is the transformer



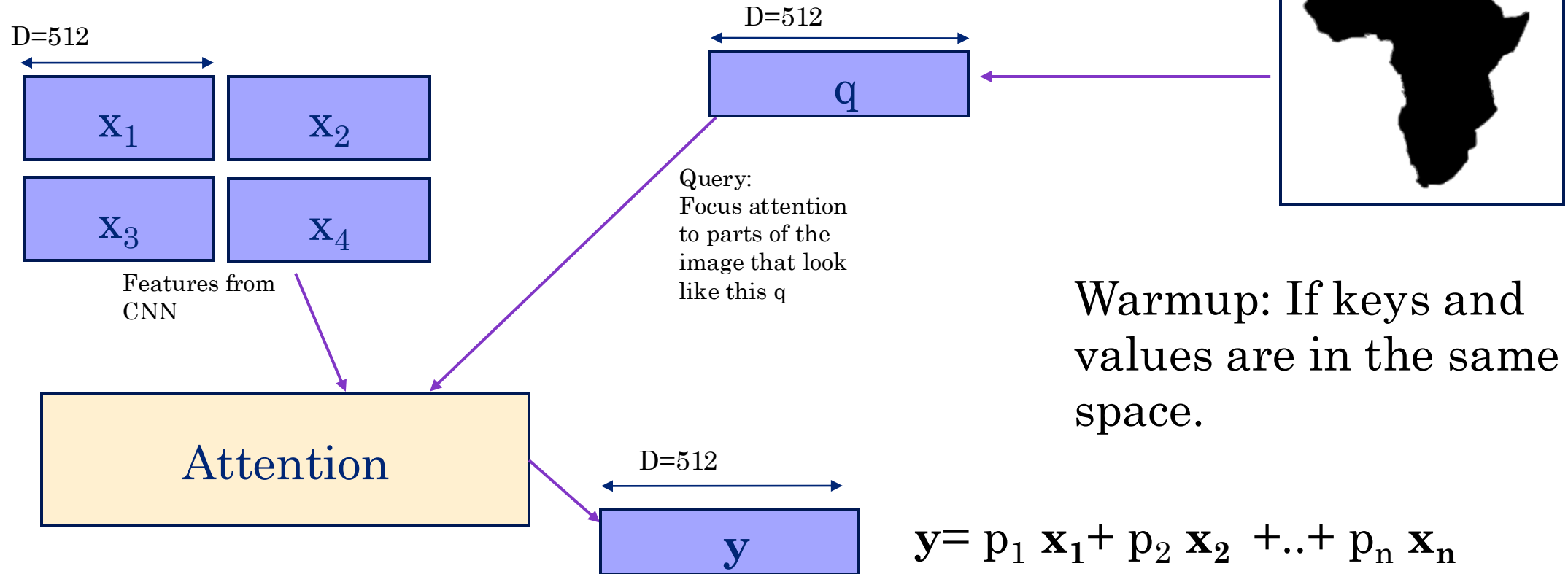
Content-based addressing Attention



Hard max and SoftMax

- $X=[1,2,3]$
- $\text{Hardmax}(X)=[0,0,1]$
- $\text{Softmax}(X)=[e^1, e^2, e^3] / C$. $C=e^1+e^2+e^3$
- $\text{Softmax}(X)=[0.09, 0.24, 0.66]$
- Softmax on a matrix= Softmax the Rows.
- $X= \begin{bmatrix} \boxed{x_1} \\ \boxed{x_2} \\ \dots \\ \boxed{x_N} \end{bmatrix}$. $\text{Softmax}[X]= \begin{bmatrix} \boxed{\text{Softmax}[x_1]} \\ \boxed{\text{Softmax}[x_2]} \\ \dots \\ \boxed{\text{Softmax}[x_N]} \end{bmatrix}$.

Content-based addressing Attention



The job of attention is to create a probability distribution $p_1, p_2, \dots, p_n$
 p_i should be large if i th position looks like the query.

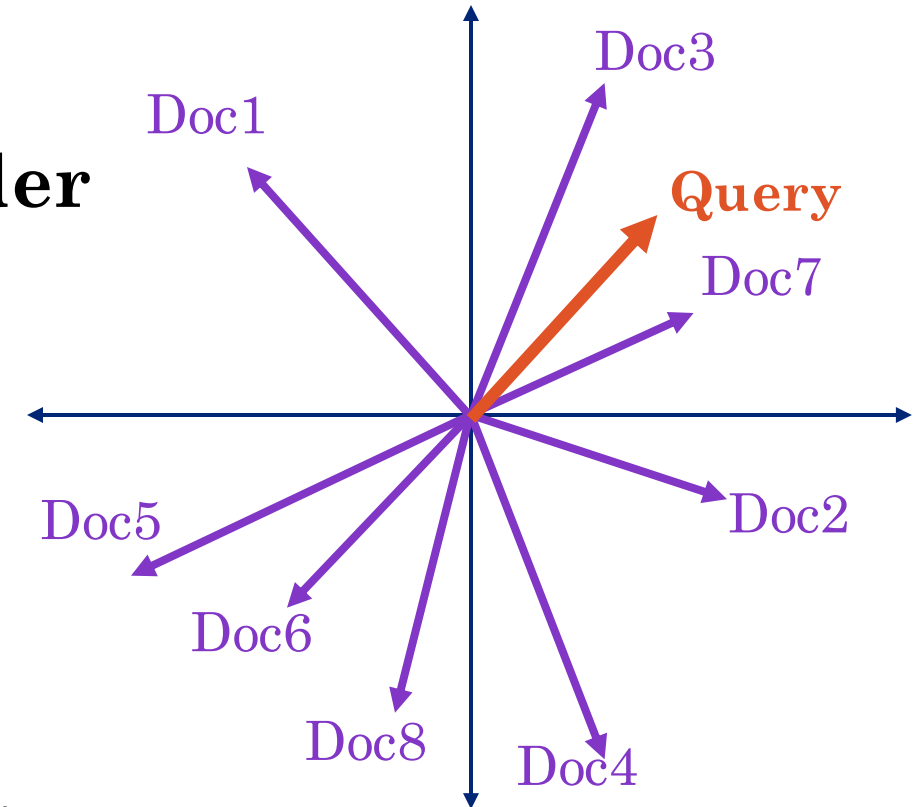
Classic Inner-Product Search for IR

Inner-Product Search is a basic **information retrieval (IR)** method to rank document relevance by:

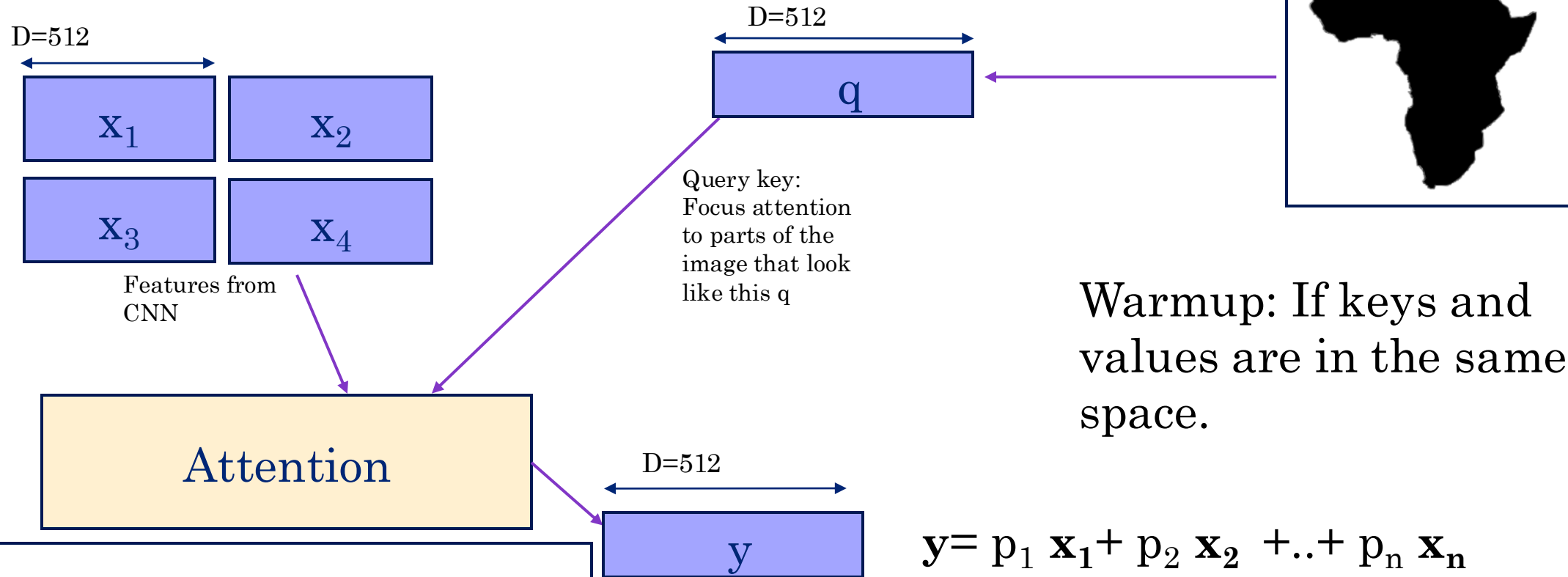
1. Assigning **vector “key”** to **each document**.
2. Constructing a **query vector**.
3. Returns documents in **descending order** of the inner-product.

Example: Doc3, Doc7, Doc2, Doc1, ...

The IR community has developed numerous techniques to construct these vectors but here we will use **linear layers**.



Content-based addressing Attention



Use softmax

$$p_i = 1/C * \exp(\mathbf{q}^T \mathbf{x}_i)$$

$$C = \sum_j \exp(\mathbf{q}^T \mathbf{x}_j)$$

Attention Output is

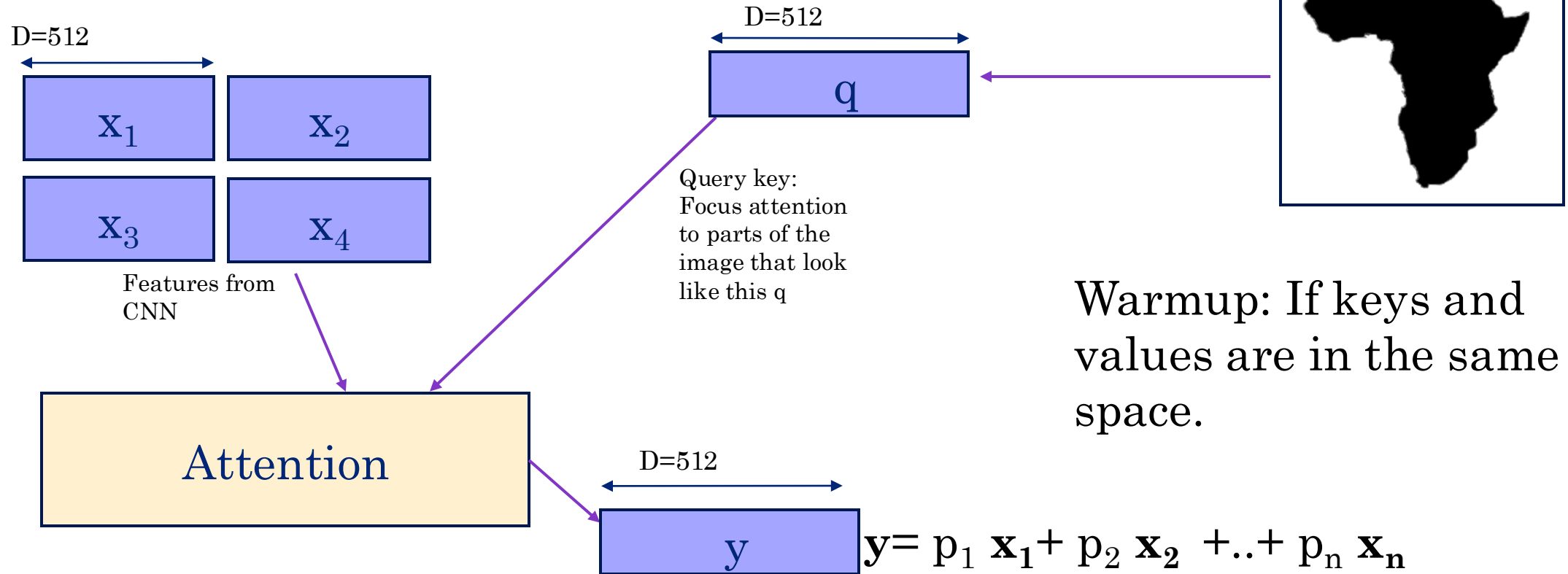
$$\mathbf{y} = \sum p_i \mathbf{x}_i$$

The job of attention is to create a probability distribution

$p_1, p_2, \dots, p_n$

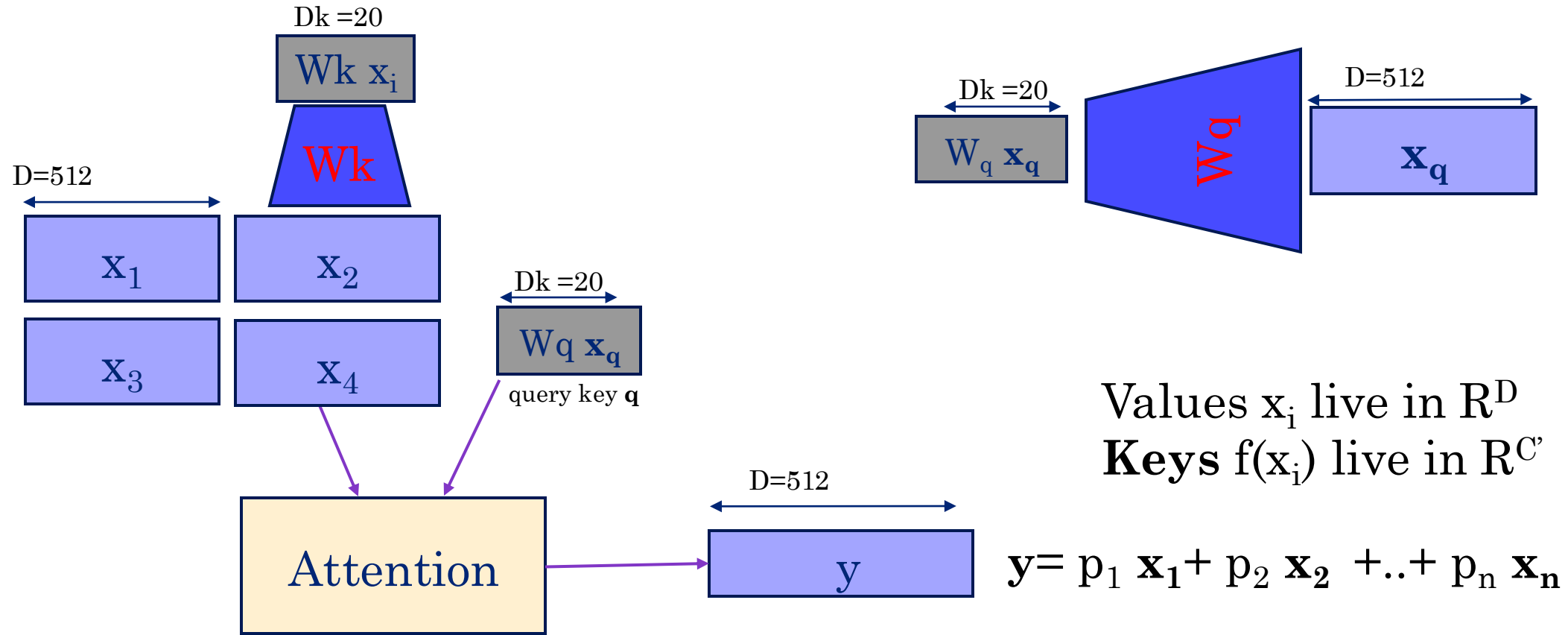
p_i should be large if i th position looks like the query.

Content-based addressing Attention



The job of attention is to create a probability distribution $p_1, p_2, \dots, p_n$ **over a variable number of n keys**
 p_i should be large if i th position looks like the query.

Content-based addressing Attention: Keys vs Values



Use softmax on **keys**.

$$q = W_q x_q$$

$$f(x_i) = W_k x_i$$

$$p_i = 1/C * \exp(q^T f(x_i))$$

$$C = \sum_j \exp(q^T f(x_j))$$

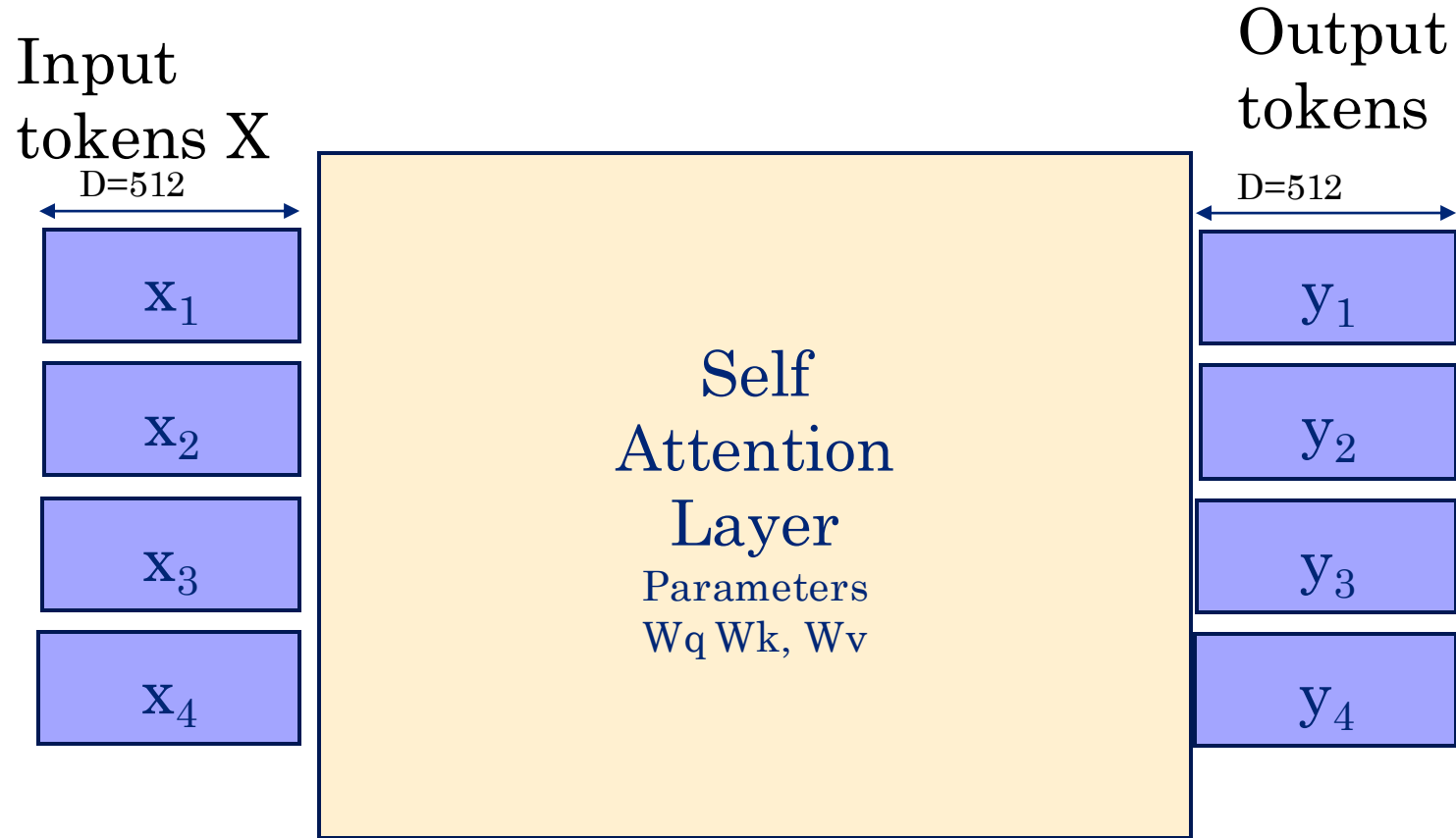
Attention Output is $y = \sum p_i x_i$

The job of attention is to create a probability distribution

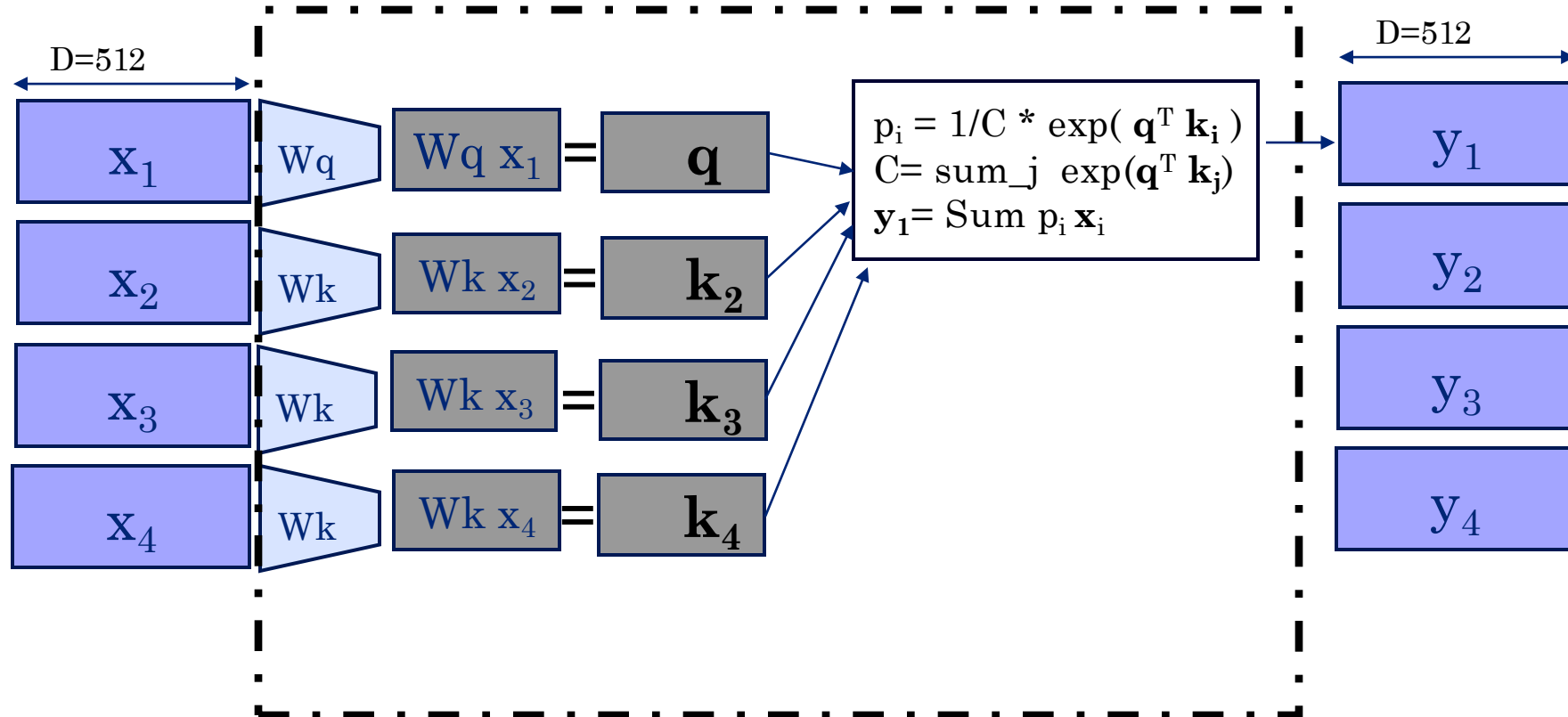
$p_1, p_2, \dots, p_n$

p_i should be large if i th position looks like the query.

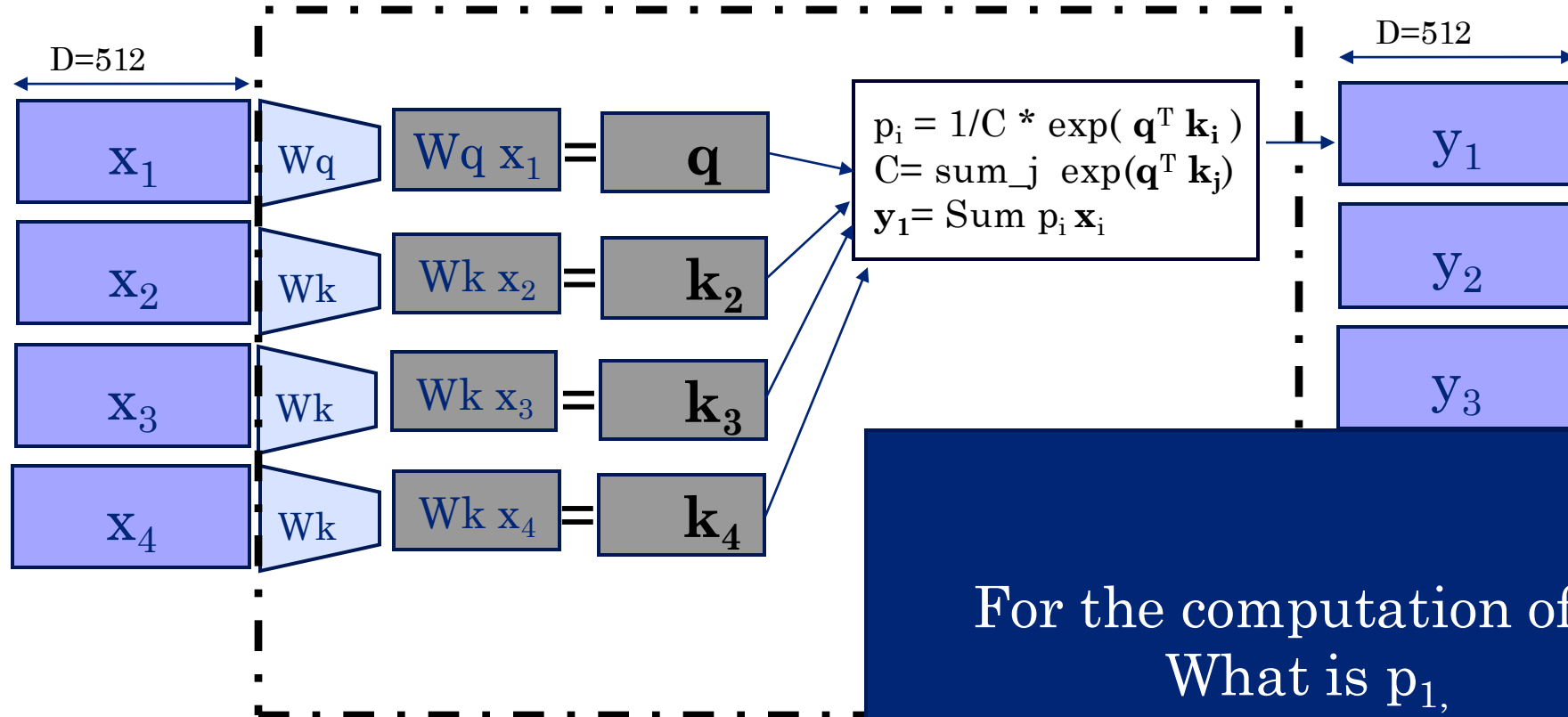
Attention is a layer. It has W_f , W_g as parameters



Computing y_1



Computing z_1



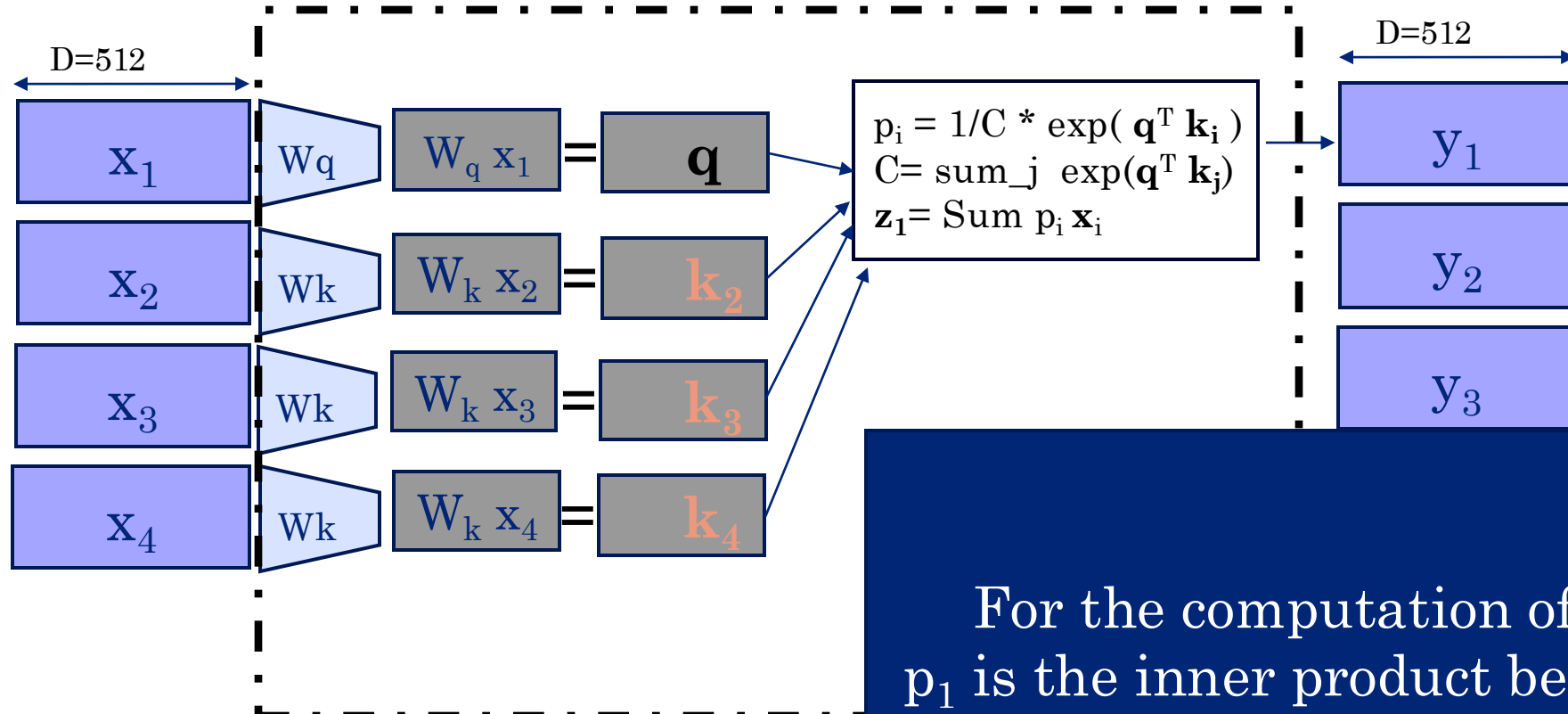
For the computation of y_1
What is p_1 ,
what similarity is used?

Computing z_1

$$p_1 = 1/C \exp(\mathbf{q}^T \mathbf{k}_1)$$

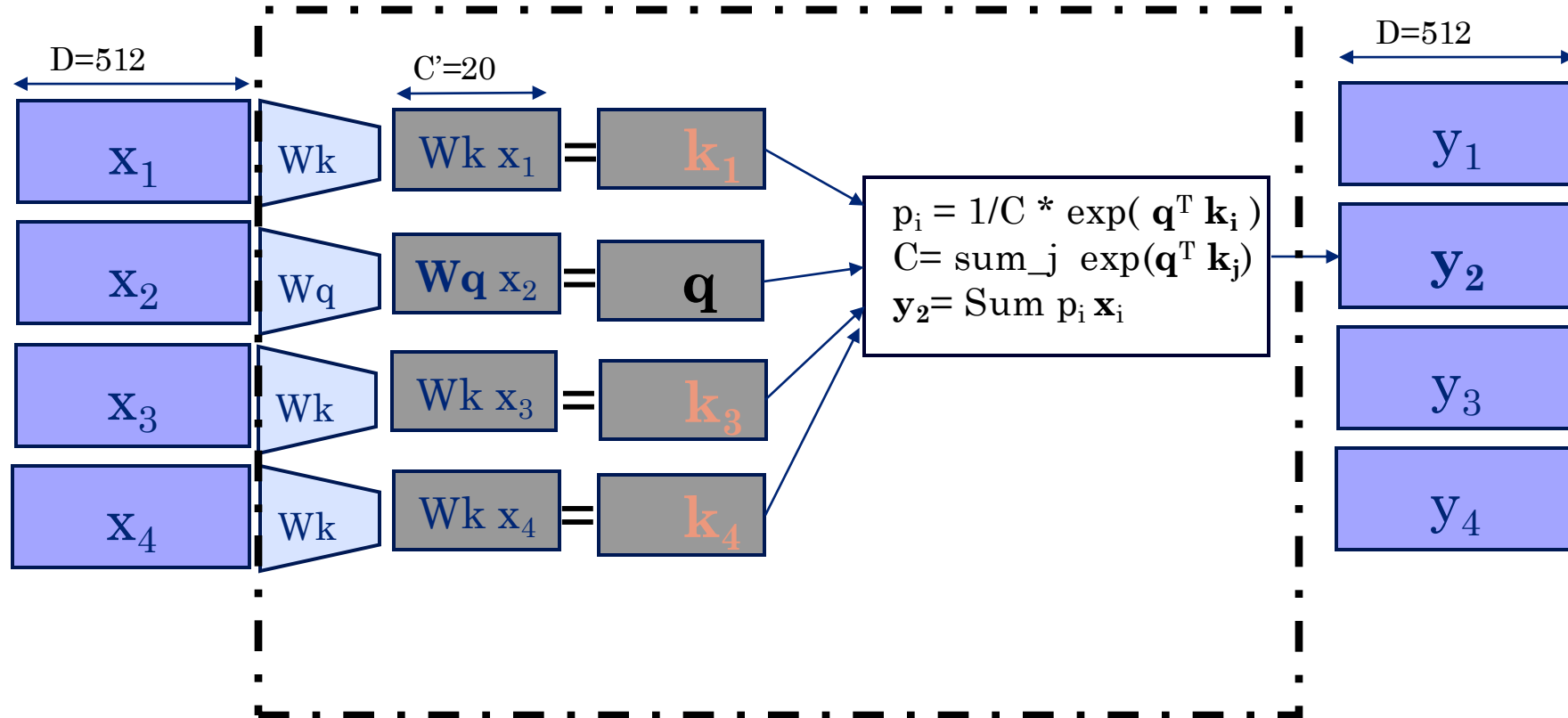
$$\mathbf{q} = \mathbf{W}_q \mathbf{x}_1$$

$$\mathbf{k}_1 = \mathbf{W}_k \mathbf{x}_1$$



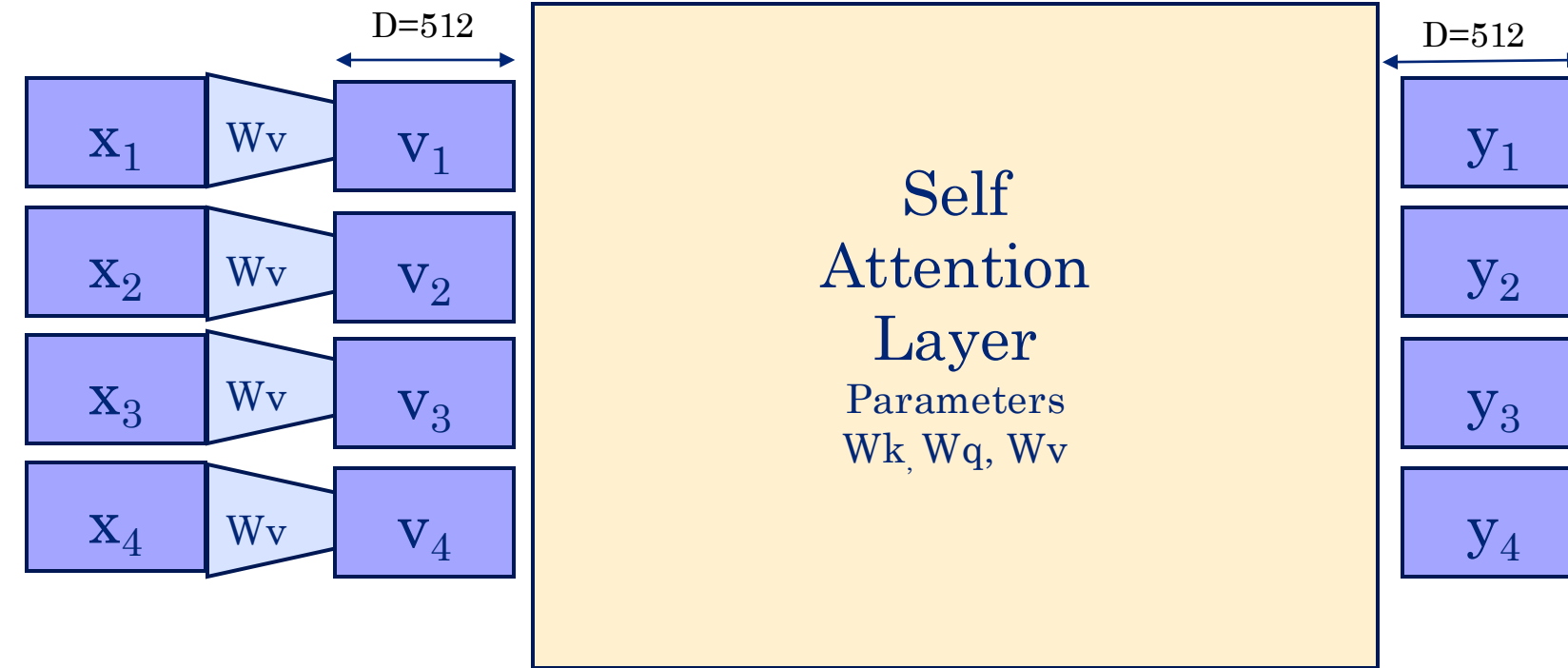
For the computation of y_1
 p_1 is the inner product between
 $\mathbf{W}_q \mathbf{x}_1$ and $\mathbf{W}_k \mathbf{x}_1$

Computing z_2



Adding also Values. A matrix multiplies $x_i \rightarrow v_i$.

Attention is a layer. It has W_k , W_q , W_v , as parameters. It is differentiable.



The whole thing can be trained end-to-end with backpropagation since it is differentiable.

Finally Done: Self-Attention in Matrix Form

Input $X \in \mathbb{R}^{N \times D}$

1. Form 3 matrices: Q, K, V:

- $V = X W^{(v)}$
- $K = X W^{(k)}$
- $Q = X W^{(q)}$

2. Output $Y = \text{SelfAttention}(Q, K, V) =$

$$Y = \text{SoftMax}_{\text{row}} \left[\frac{Q K^T}{\sqrt{D}} \right] V = A V$$

$$V = X W^{(v)}$$

$$K = X W^{(k)}$$

$$Q = X W^{(q)}$$

$$A = \text{SoftMax}_{\text{row}} \left[\frac{Q K^T}{\sqrt{D_k}} \right]$$

$$W^{(v)}, W^{(k)}, W^{(q)} \in \mathbb{R}^{D \times D}$$

$$A \in \mathbb{R}^{N \times N}$$

Self-Attention summariz

```
In [18]: def self_attention(Q,K,V):
...:     d=Q.shape[-1]
...:     return torch.softmax(Q @ K.T /math.sqrt(d), dim=-1) @ V
...:

In [19]: X=torch.randn(100,10)

In [20]: WQ= torch.randn(10,10)

In [21]: WK=torch.randn(10,10)

In [22]: WV=torch.randn(10,10)

In [23]: Q,K,V = X@WQ , X@WK , X@WV

In [24]: Y= self_attention(Q,K,V)
```

Input $X \in \mathbb{R}^{N \times D}$

1. Form 3 matrices: Q, K, V:

- $Q = X W^{(q)}$
- $K = X W^{(k)}$
- $V = X W^{(v)}$

2. Output $Y = \text{SelfAttention}(Q, K, V) =$

$$Y = \text{SoftMax}_{\text{row}} \left[\frac{Q K^T}{\sqrt{D}} \right] V = A V$$

Challenges in translation: need forward and backward connections.

The animal didn't cross the street because it was too tired
L'animal n'a pas traversé la rue parce qu'il était trop fatigué

The animal didn't cross the street because it was too wide
L'animal n'a pas traversé la rue parce qu'elle était trop large

Challenges in translation: need forward and backward connections.

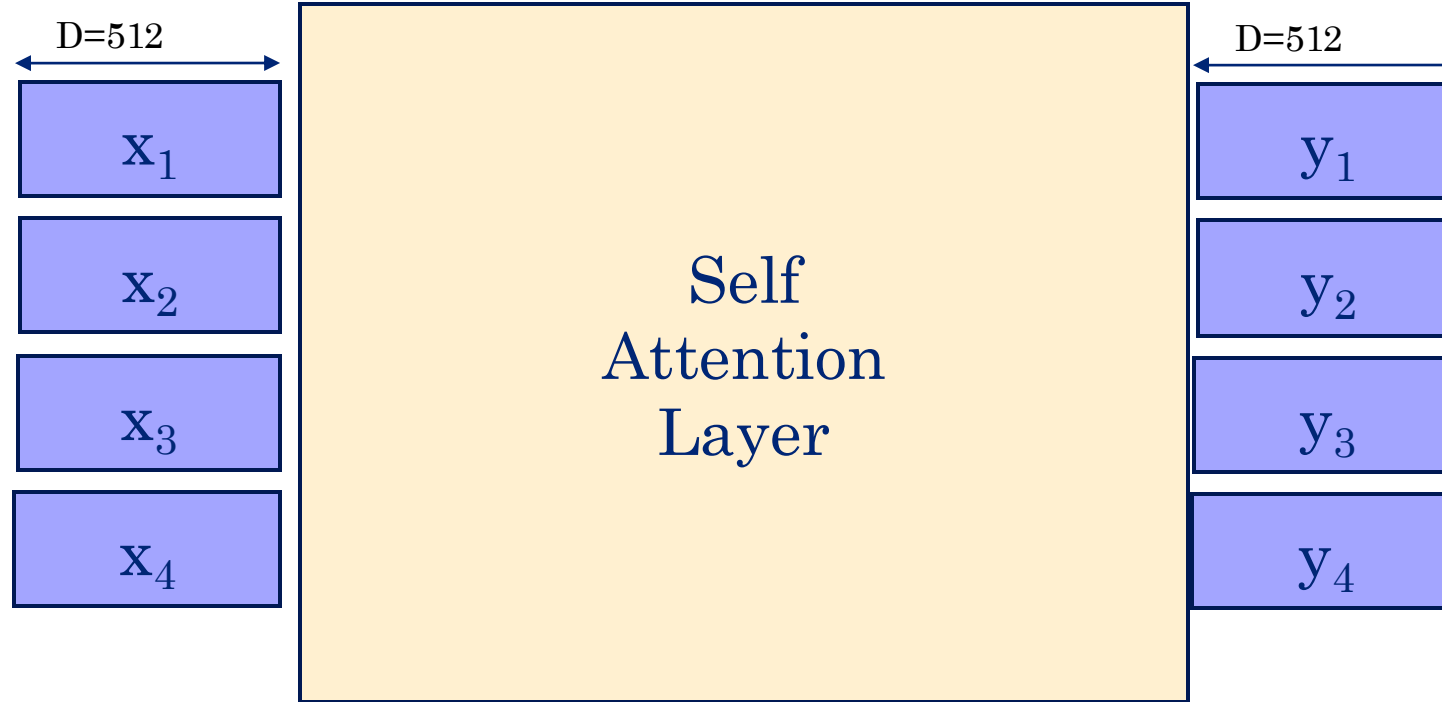
The animal didn't cross the street because it was too tired
L'animal n'a pas traverse la rue parce qu'il était trop fatigue

The animal didn't cross the street because it was too wide
L'animal n'a pas traverse la rue parce qu'elle était trop large

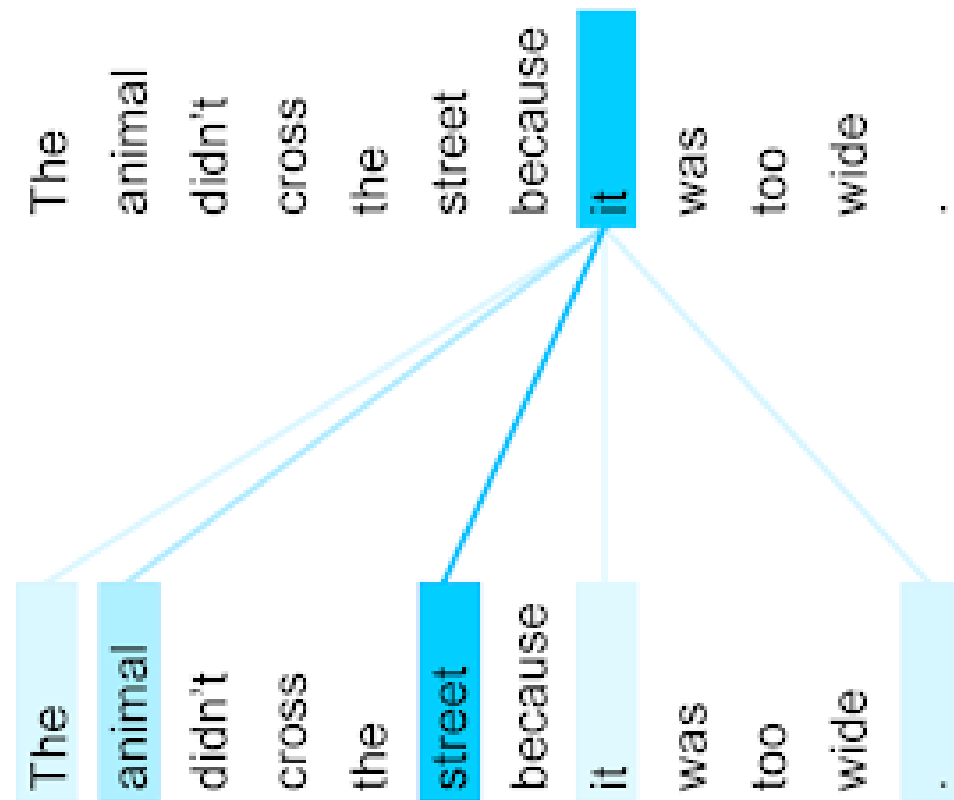
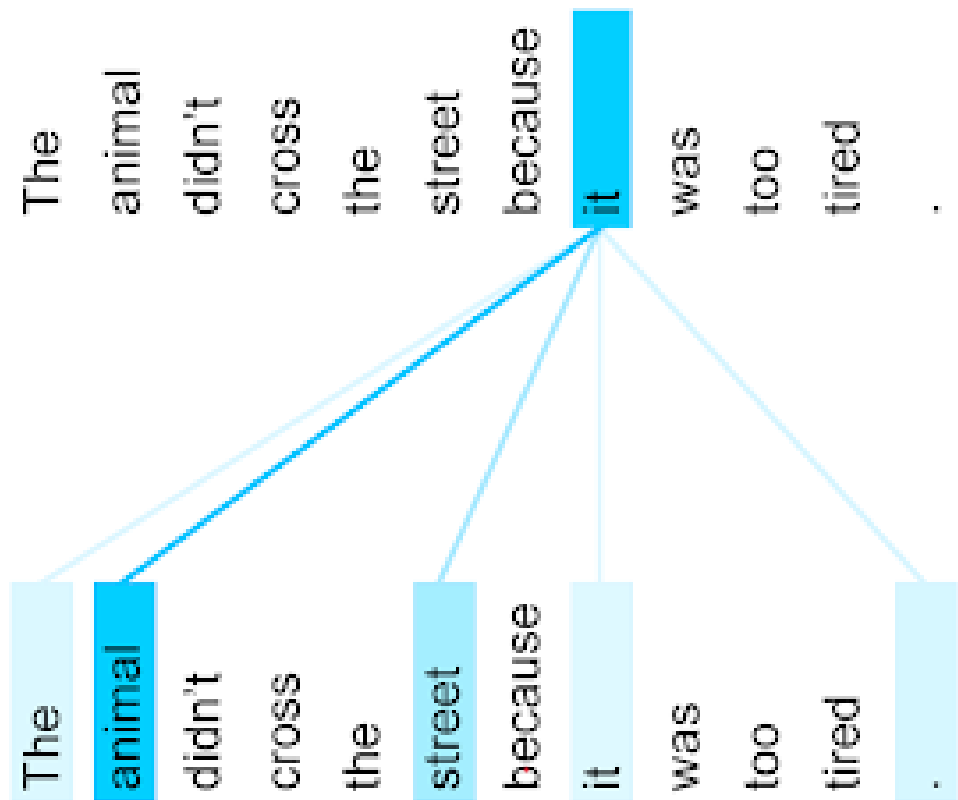
Addressing the challenge through self-attention

The
animal
didn't
cross
the
street
because
it
was
too
tired

Tokens

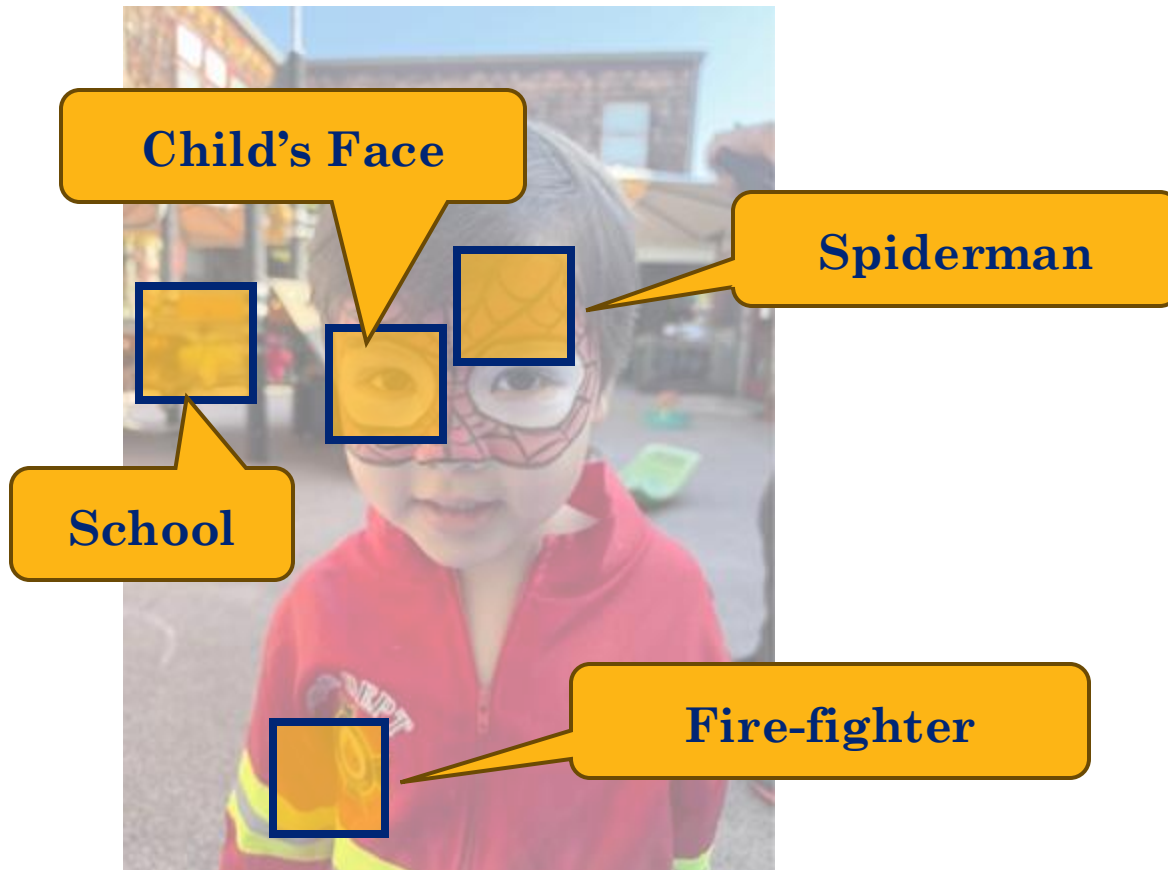


Self-attention allows connections in the future



Need to Include Context in Representations

Computer Vision



Language Modeling

*I swam across the river to get to the other **bank**.*

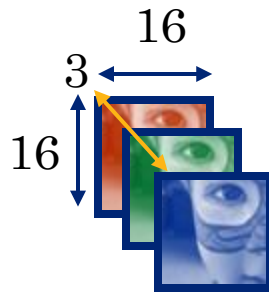
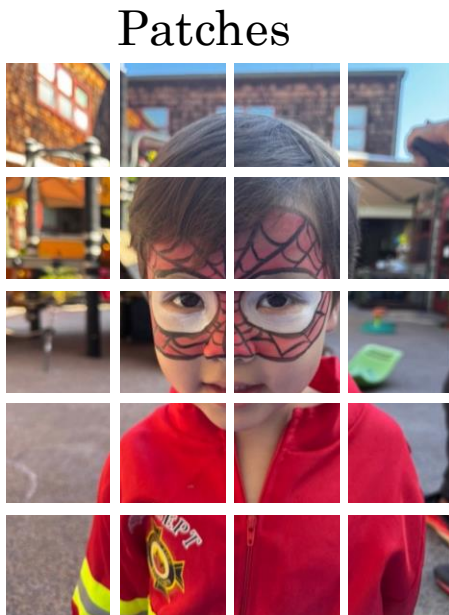
*I walked across the road to get cash from the other **bank**.*

The **meaning of bank** depends on the words (context) around it.

Starting with Token Embeddings

Token Embeddings: The Transformer model takes as input a set of N token embeddings vectors in $\mathbb{R}^D$ representing the basic building blocks of an image or text.

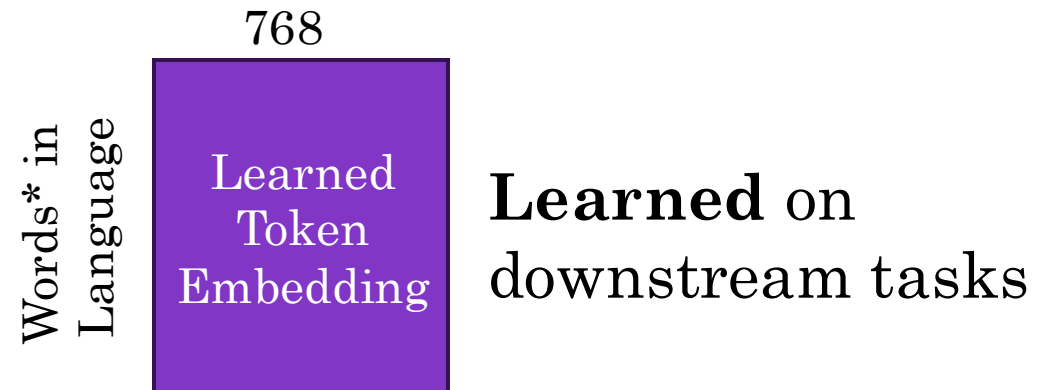
- **Example:** Patches of images or words*



$$16 * 16 * 3 = 768$$

Each token embedding is a 768-dimensional vector.

*I walked across the road to get cash
from the other bank.*



*Typically, model word parts not words (more later).

Basic Setup of a Transformer Layer

Input: A matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ of N token embeddings vectors in $\mathbb{R}^D$

Output: A matrix $\mathbf{Y} \in \mathbb{R}^{N \times D}$ of N token embeddings vectors in $\mathbb{R}^D$

Step 1: Start by **extracting the “context”** from each of the patches in isolation using a linear layer:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

where $\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$ is a learned matrix.

- We will determine D_v later but you can think of this as D for now.
- **Note** we will “imagine” that $\mathbf{V}$ encodes the “context” but SGD will determine the “true meaning” of $\mathbf{V}$.

Incorporating the Context of other Tokens

Focusing on the n^{th} token $\mathbf{Y}_n$ how could we incorporate the context of all the other tokens?

Proposal – Average Pooling:

$$\mathbf{Y}_n = \mathbf{X}_n + \frac{1}{N} \sum_{i=1}^N \mathbf{v}_i$$

- Combines all the “extracted context”
- No additional parameters
- Invariant to number of tokens?
 - Yes! Averaging more tokens should behave similarly.
- But...

Symbols:

$\mathbf{X} \in \mathbb{R}^{N \times D}$

$\mathbf{Y} \in \mathbb{R}^{N \times D}$

$\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$

$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$

Not all Context is Relevant

Averaging context *incorrectly* treats all context as **equally important** to the context of the n^{th} token Y_n :

*I **swam** across the **river** to get to the other **bank**.*



The diagram shows two curved arrows pointing from the word 'bank' to the words 'swam' and 'river'. The arrow pointing to 'river' is labeled 'Most important' and the arrow pointing to 'swam' is labeled 'Important'.

Some context could be harmful:

*The **food** was **good**, not **bad** at all.*



The diagram shows two curved arrows pointing from the word 'good' to the words 'food' and 'bad'. The arrow pointing to 'bad' is labeled 'Harmful?*' and the arrow pointing to 'food' is labeled 'Important'.

*If “bad” becomes **transformed** by “not” (context) then it might be helpful.

Constructing Keys and Queries

We construct the “key” vectors and “query” vectors for each token using a linear layer:

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

where $\mathbf{W}^{(k)}, \mathbf{W}^{(q)} \in \mathbb{R}^{D \times D_k}$ are parameters and the size of the key vector D_k is typically chosen to be smaller than or equal to D .

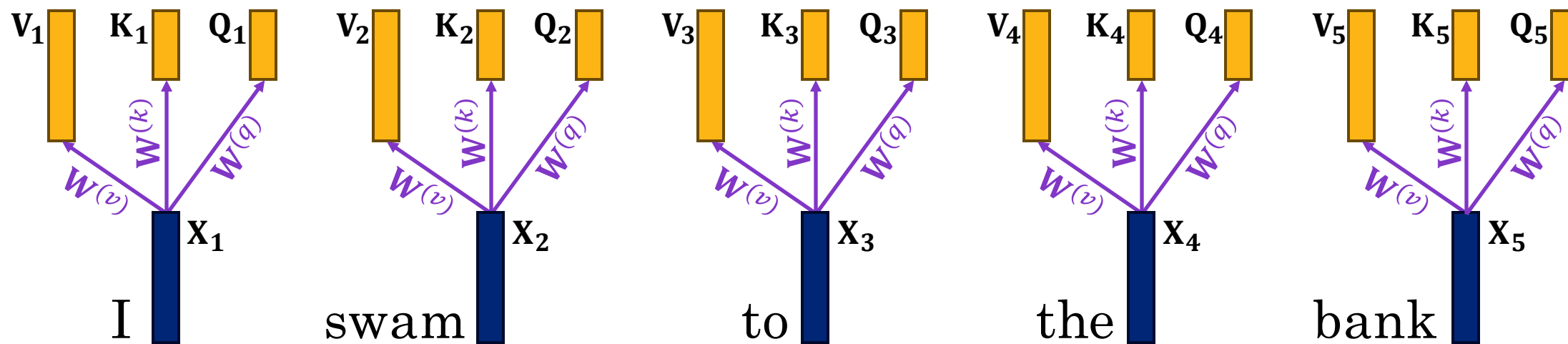
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

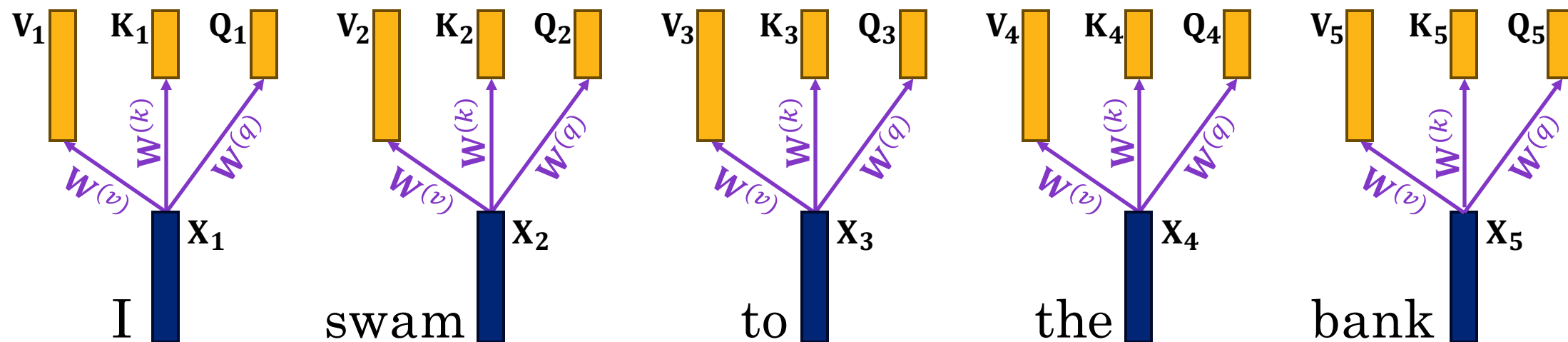
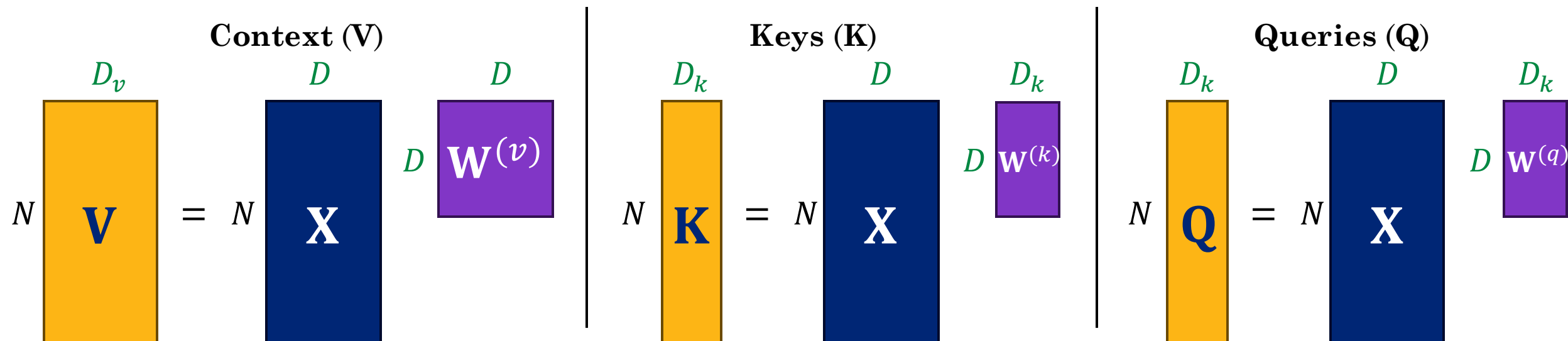
$$\mathbf{Y} \in \mathbb{R}^{N \times D}$$

$$\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$



Matrix Formulation of K, Q, V



Why A Separate Key and Query?

Separate **key** and **query** linear layers (i.e., $\mathbf{W}^{(k)}$, $\mathbf{W}^{(q)}$) allow for **asymmetric queries**

Example:

*I swam across the river to get to the other **bank**.*



Bank can query river but river doesn't have to query bank.

- **Not all relevance relationships are symmetric**

Inner-Product Search for Tokens

The **inner-product** between $\mathbf{Q}_n$ and $\mathbf{K}_i$ encodes the **relevance** of token i to computing the **representation** of token n .

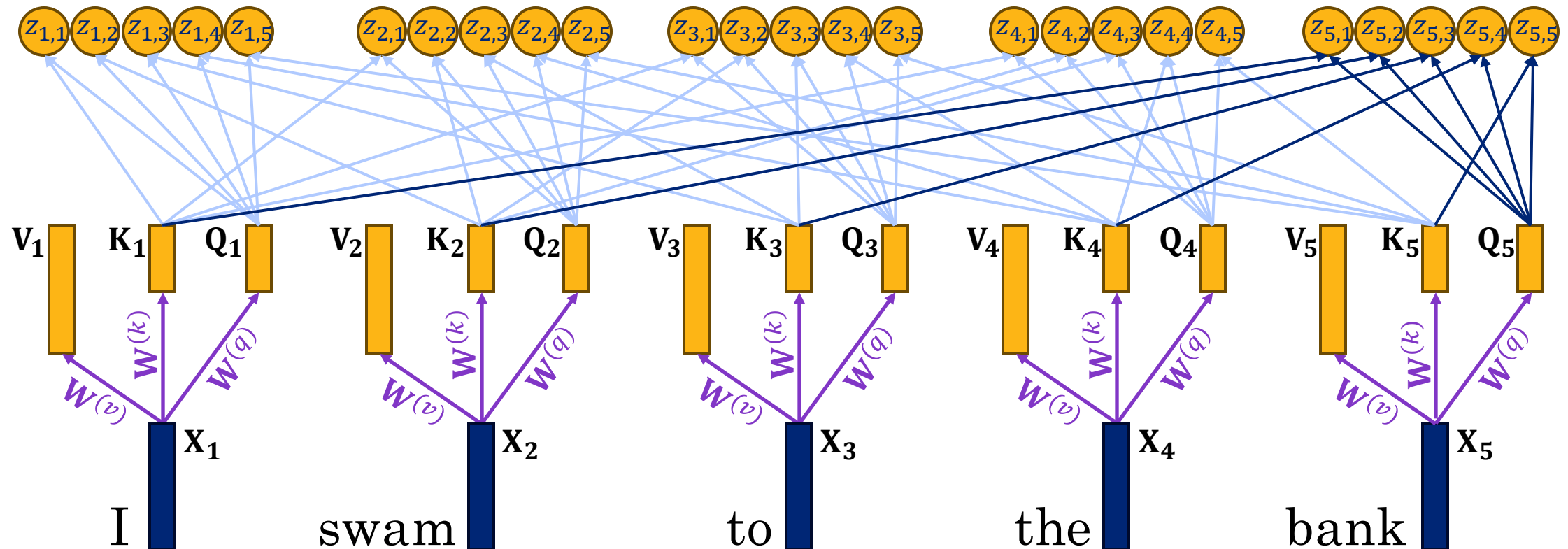
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



The Inner-Product Matrix

The $\mathbf{Z}_{n,i}$ entry of the $\mathbf{Z} \in \mathbb{R}^{N \times N}$ **inner-product** matrix contains the inner product between query $\mathbf{Q}_n$ and key $\mathbf{K}_i$

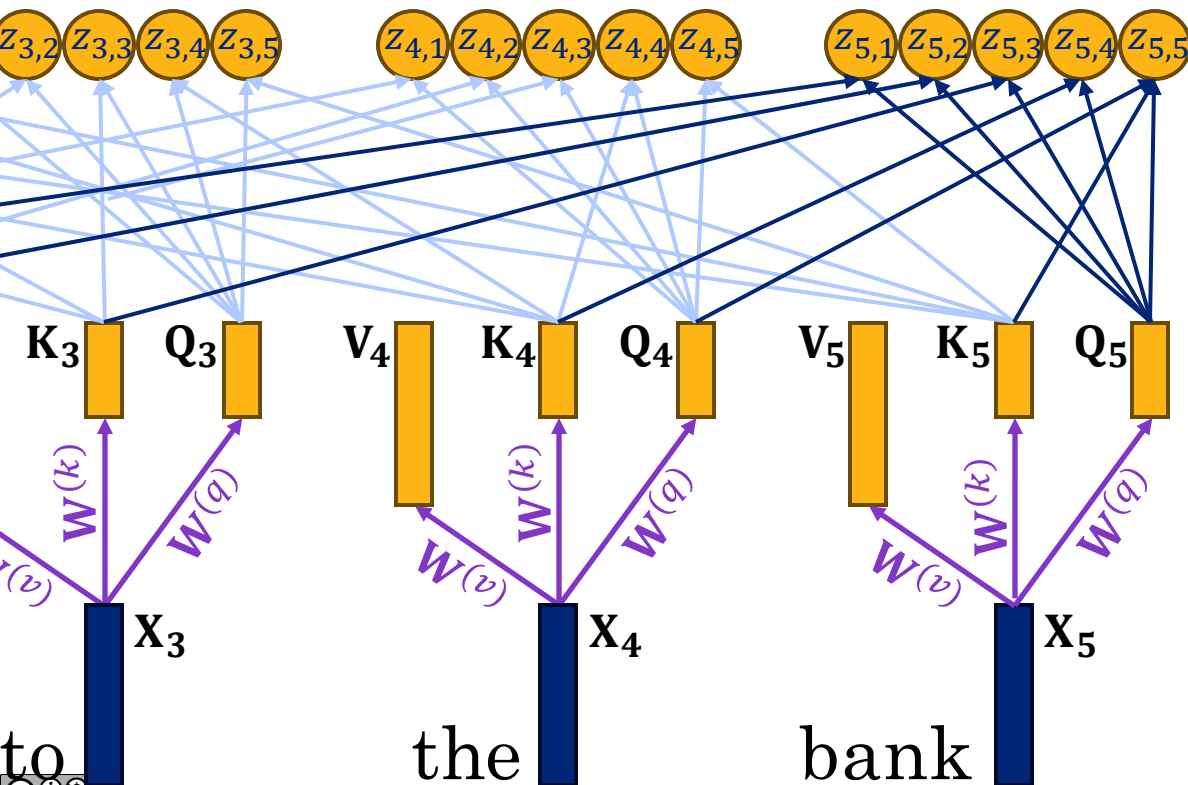
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

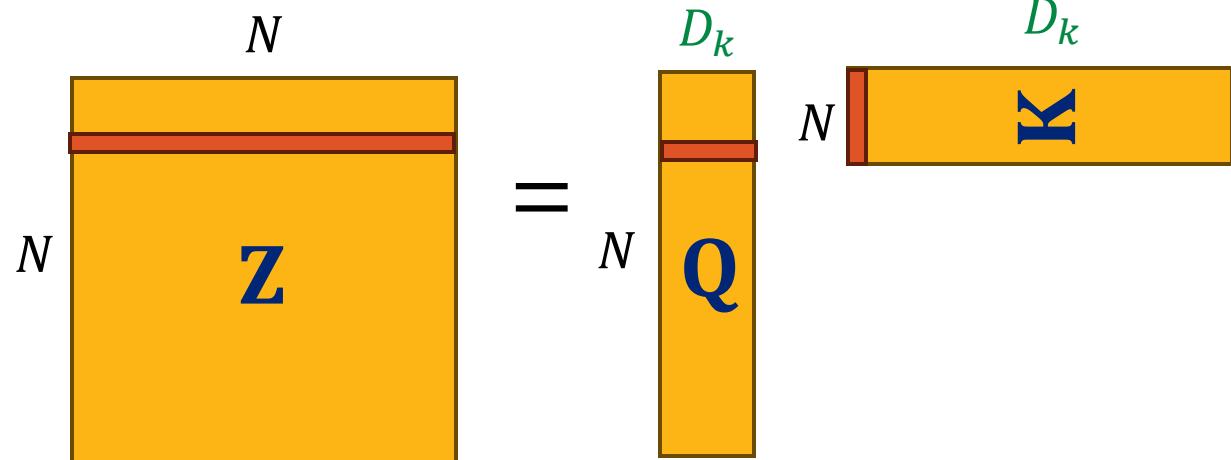
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^\top$$



Issue:

$z_{n,i} \in \mathbb{R}$ could be negative and not guaranteed $\sum_{i=1}^N z_{n,i} = 1$

Solution:

Softmax

Softmax Inner-Product Self-Attention

By computing the **softmax** along the rows of **Z** obtain the **attention matrix** $A \in \mathbb{R}^{N \times N}$

Symbols:

$$X \in \mathbb{R}^{N \times D}$$

$$V = X W^{(v)}$$

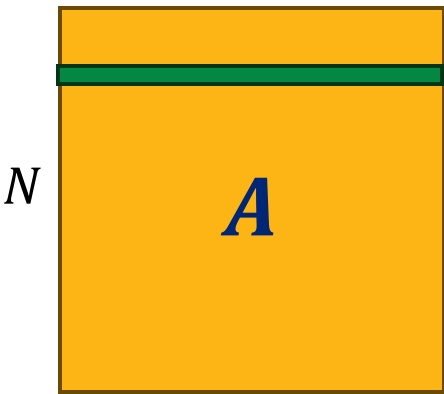
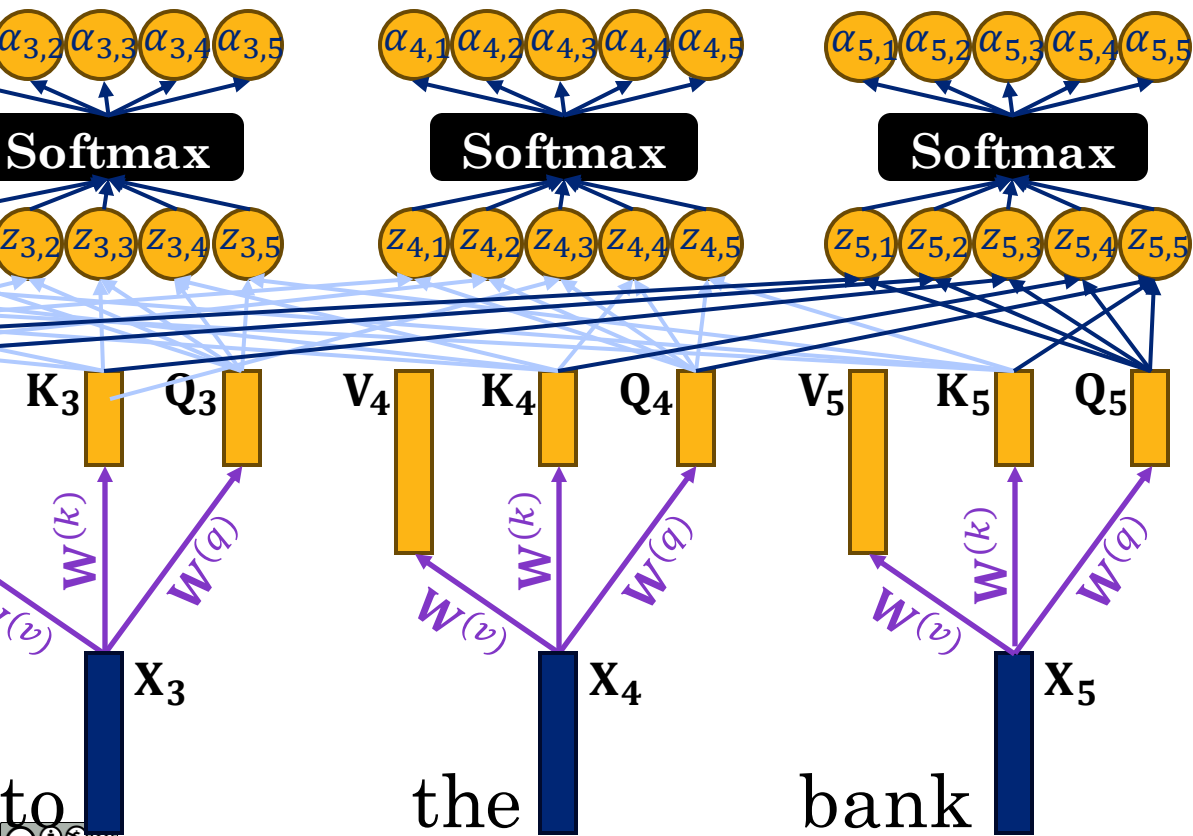
$$K = X W^{(k)}$$

$$Q = X W^{(q)}$$

$$Z = Q K^T$$

$$A = \text{SoftMaxRow}[Z]$$

$$A_{ni} = \frac{\exp(Z_{ni})}{\sum_{j=1}^N \exp(Z_{nj})}$$



Each row has
no negative entries
and sums to 1

Scaled Softmax Inner-Product Self-Attention

By computing the **softmax** along the rows of **Z** obtain the **attention matrix** $\mathbf{A} \in \mathbb{R}^{N \times N}$

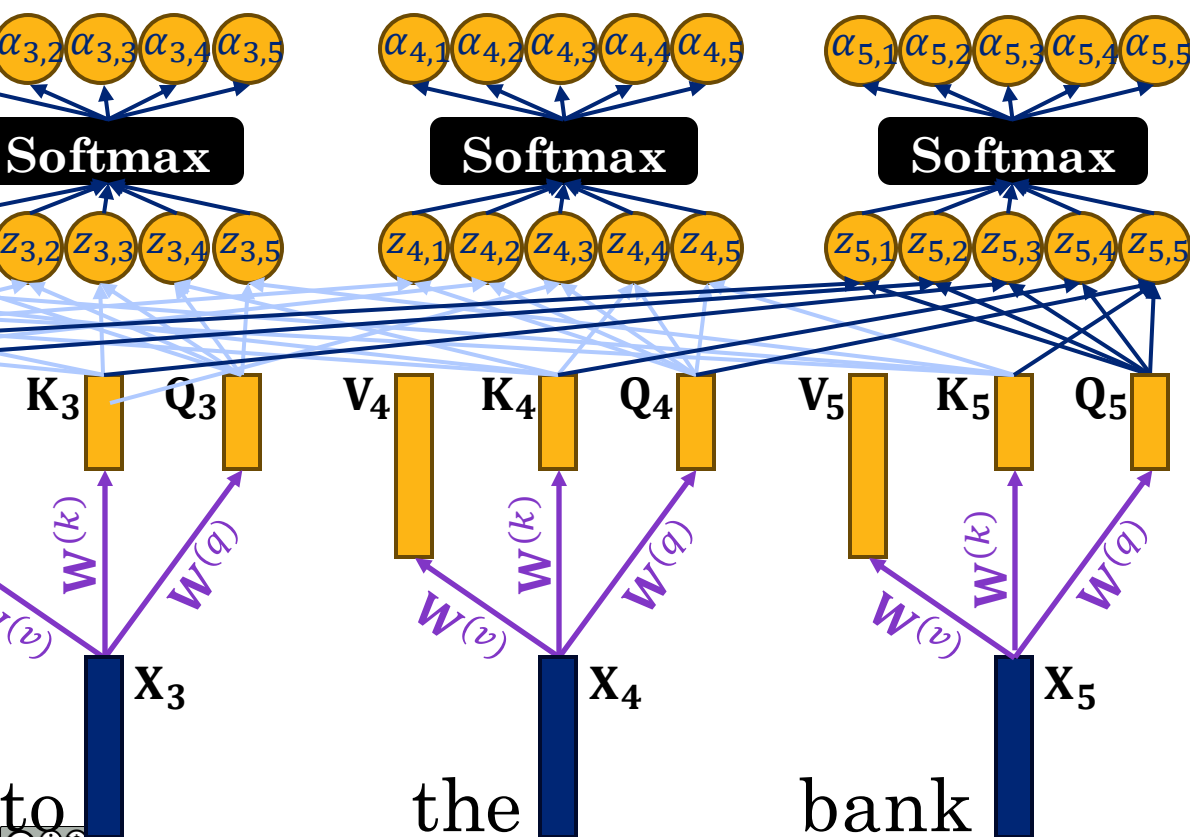
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^\top$$

$$\mathbf{A} = \text{SoftMaxRow} \left[\frac{\mathbf{Z}}{\sqrt{D_k}} \right]$$

$$A_{ni} = \exp \left(\frac{\mathbf{Z}_{ni}}{\sqrt{D_k}} \right) \left(\sum_{j=1}^N \exp \left(\frac{\mathbf{Z}_{nj}}{\sqrt{D_k}} \right) \right)^{-1}$$

Scaled by $\sqrt{D_k}$ to ensure that variance of the inner product of D_k terms stays close to 1.

Improves training...

The Self Attention Layer

The attention layer $\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]$ depends only on keys $\mathbf{K}$, queries $\mathbf{Q}$, and values $\mathbf{V}$ for all tokens and **has no parameters**.

Symbols:

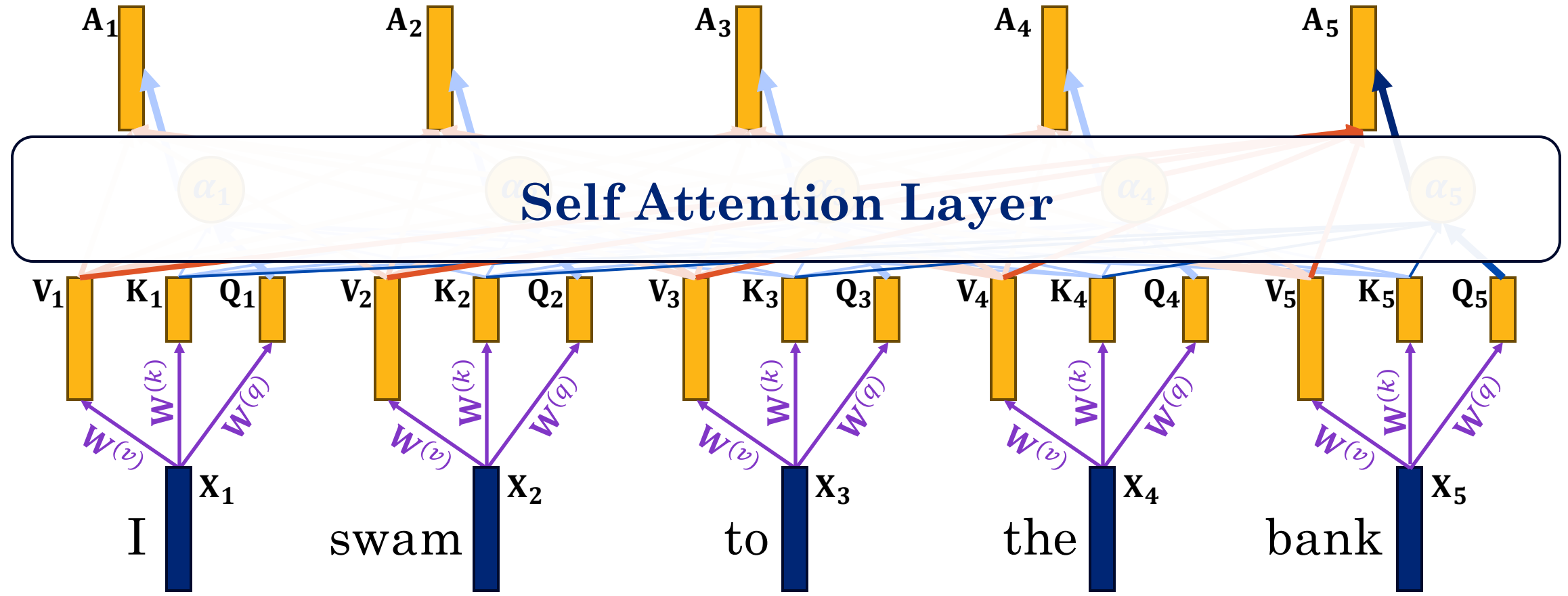
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\alpha = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\alpha \in \mathbb{R}^{N \times N}$$



Parameter and Computational Complexity

How many parameters are needed to construct $\mathbf{K}$, $\mathbf{Q}$, and $\mathbf{V}$?

- Add the parameters $D_v^2 + 2D_k^2$ which **doesn't depend** on N

What is the **computational complexity** of $\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]$?

- $\text{Softmax}\left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}}\right]$ is $O(N^2 D_k)$
- $\mathbf{A} \mathbf{V}$ is $O(N^2 D_v) \leftarrow$ Dominant term

Compared to a **Fully Connected Layer**

- The **flattened feature** vector would be in $\mathbb{R}^{N D_v}$
- Linear layer parameters: $(N D_v)^2 = N^2 D_v^2$
- **Computation:** $O((N D_v)^2)$

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}, \mathbf{W}^{(v)} \in \mathbb{R}^{D_v \times D_v}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}, \mathbf{W}^{(k)} \in \mathbb{R}^{D_k \times D_k}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}, \mathbf{W}^{(q)} \in \mathbb{R}^{D_k \times D_k}$$

$$\mathbf{A} = \text{Softmax}\left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}}\right], \mathbf{A} \in \mathbb{R}^{N \times N}$$

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}] = \mathbf{A} \mathbf{V}$$

Attention as a Soft Lookup table aka a Soft HashMap or Dictionary.

- Lets remember how a Python Dictionary works:
- $D = \{ \text{"A"}:51, \text{"B"}:42, \text{"C"}:31 \}$
- $D[\text{"A"}] = ?$

Attention as a Soft Lookup table aka a Soft HashMap or Dictionary.

- Lets remember how a Python Dictionary works:
- $D = \{ \text{"A":51, "B":42, "C":31} \}$
- $D[\text{"A"}] = 51$
- If query= "A", output is 51.
- So what would a Soft Dictionary be?

Attention as a Soft Lookup table aka a Soft HashMap or Dictionary.

- Lets remember how a Python Dictionary works:
- $D = \{ \text{"A"}:51, \text{"B"}:42, \text{"C"}:31 \}$
- $D[\text{"A"}] = 51$
- If query= "A", output is 51.
- So what would a Soft Dictionary be?
- Make keys and query to be vectors.
- $D = \{ \text{"k1=[1,0,0]"}:51, \text{"k2=[0,1,0]"}:42, \text{"k3=[0,0,1]"}:31 \}$
- If $q=[0,1,0]$, $D[q]=?$
- If $q=[1,1,0]$, $D[q]=?$

Attention as a Soft Lookup table aka a Soft HashMap or Dictionary.

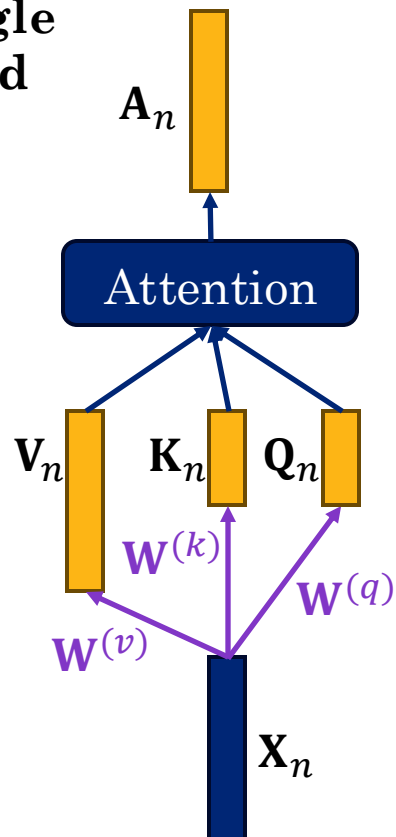
- Lets remember how a Python Dictionary works:
- $D = \{ \text{"A"}:51, \text{"B"}:42, \text{"C"}:31 \}$
- $D[\text{"A"}] = 51$
- If query= "A", output is 51.
- So what would a Soft Dictionary be?
- Make keys and query to be vectors.
- $D = \{ \text{"k1"}=[1,0,0]:51, \text{"k2"}=[0,1,0]:42, \text{"k3"}=[0,0,1]:31 \}$
- If $q=[0,1,0]$, $D[q]=?$
- If $q=[1,1,0]$, $D[q]= p1 \ V1 + p2 \ V2$
- $A = \text{Softmax} [q^T k1, q^T k2]$

Multi-Headed Attention

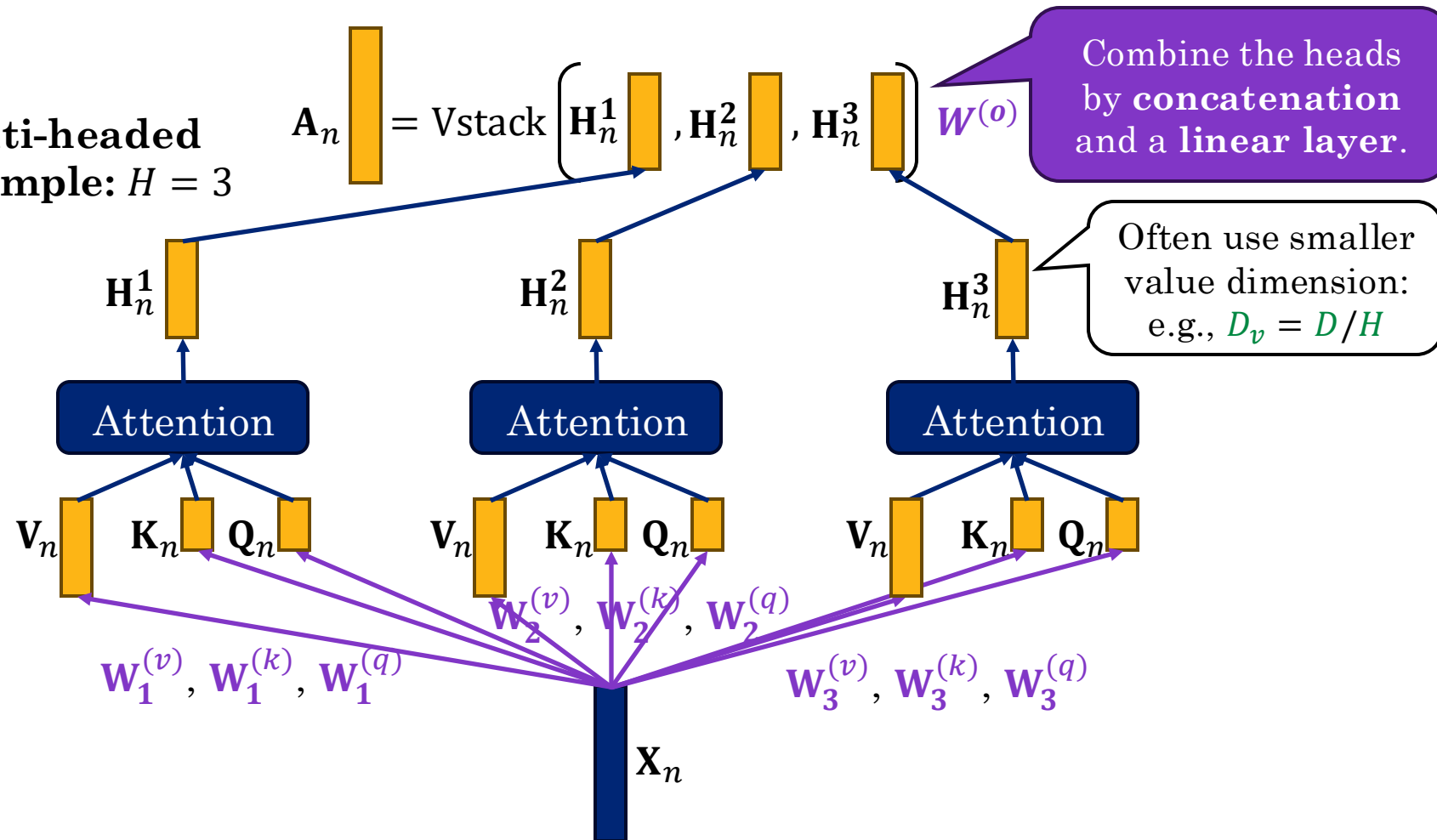
Idea: There may be more than one way to **extract** and **query** the context.

Multi-headed attention makes H parallel copies of the **K**, **Q**, and **V** matrices with **separate weights**.

Single Head



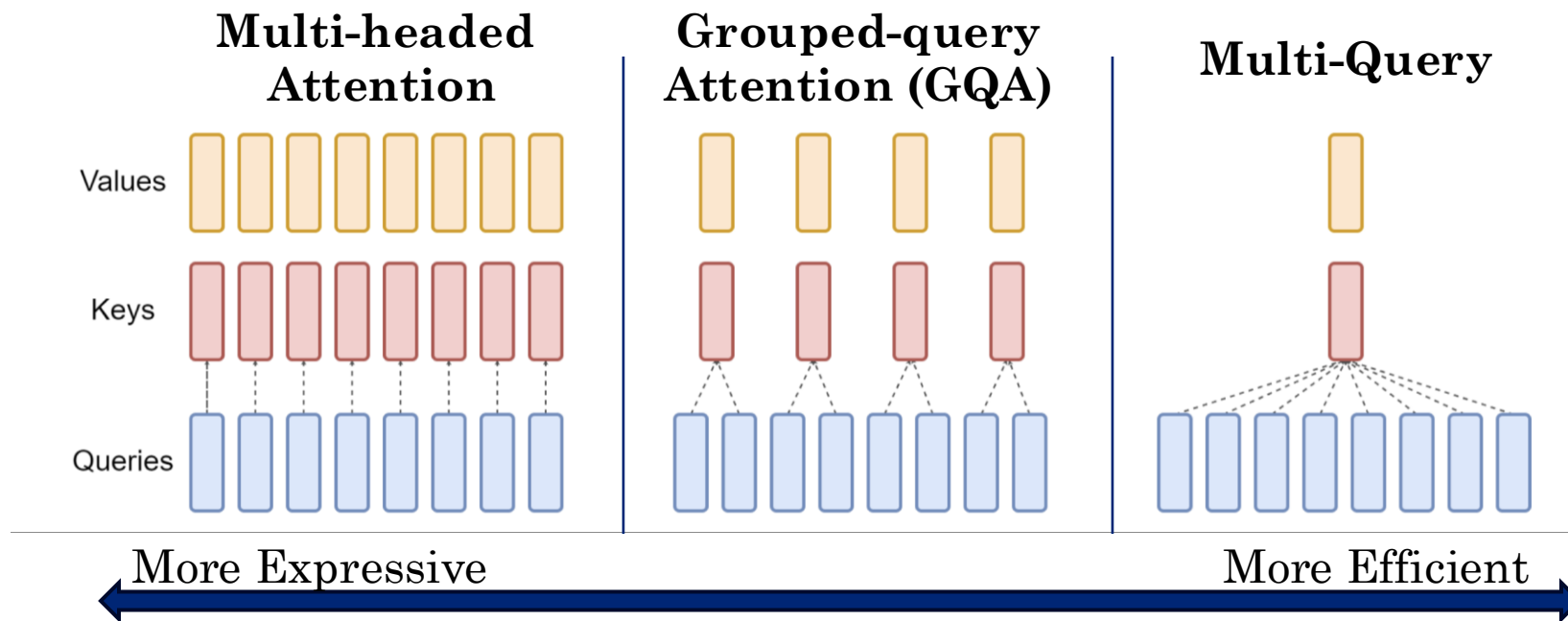
Multi-headed
Example: $H = 3$



Alternatives to Multi-Headed Attention

Creating (and storing) keys **K**, queries **Q**, and values **V** for each head is costly.

- Could we reuse the same **keys** and **values** with **multiple queries**?



Most models currently use **Grouped Query Attention (GQA)** which reuses **keys** and **values** across multiple **query** vectors.

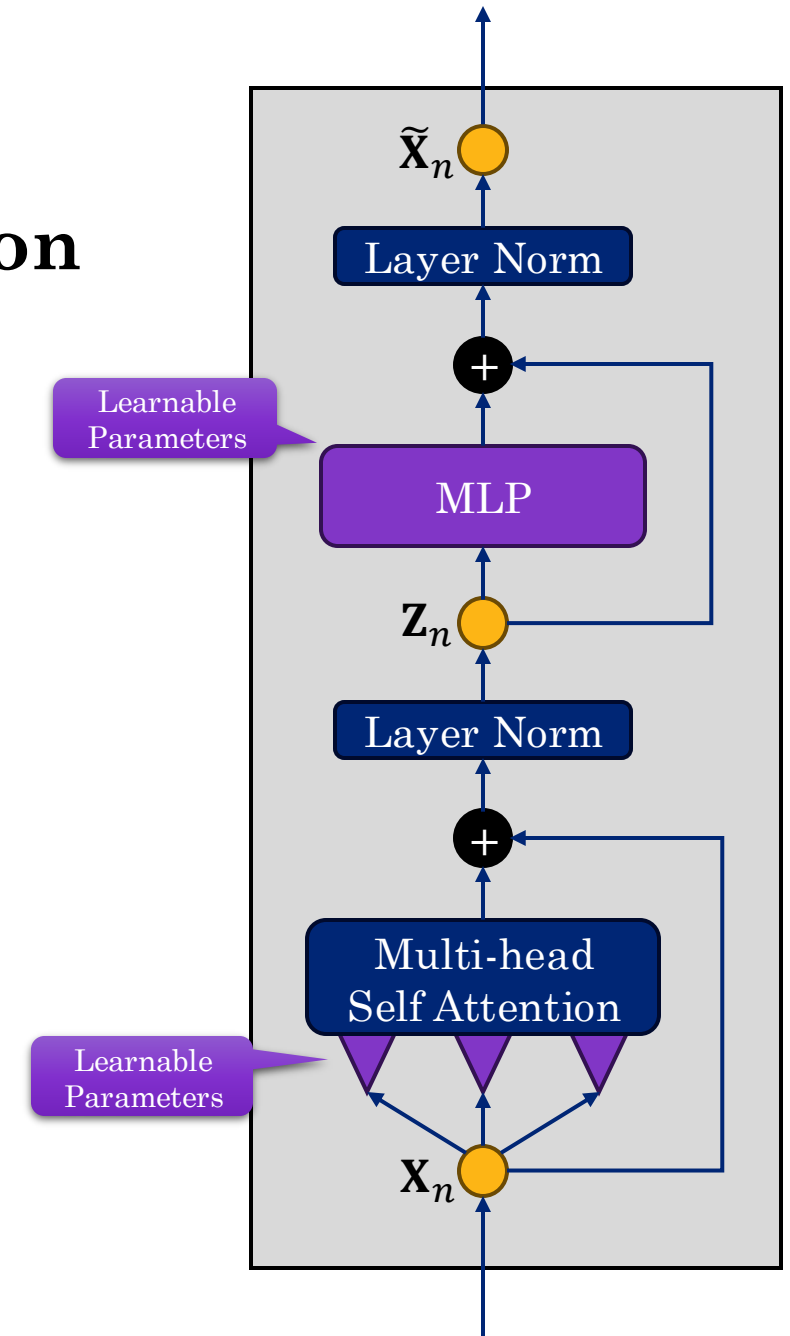
The Transformer Layer

A **transformer layer** combines **self-attention** with a token-level **neural network** (typically, a 2-layer MLP).

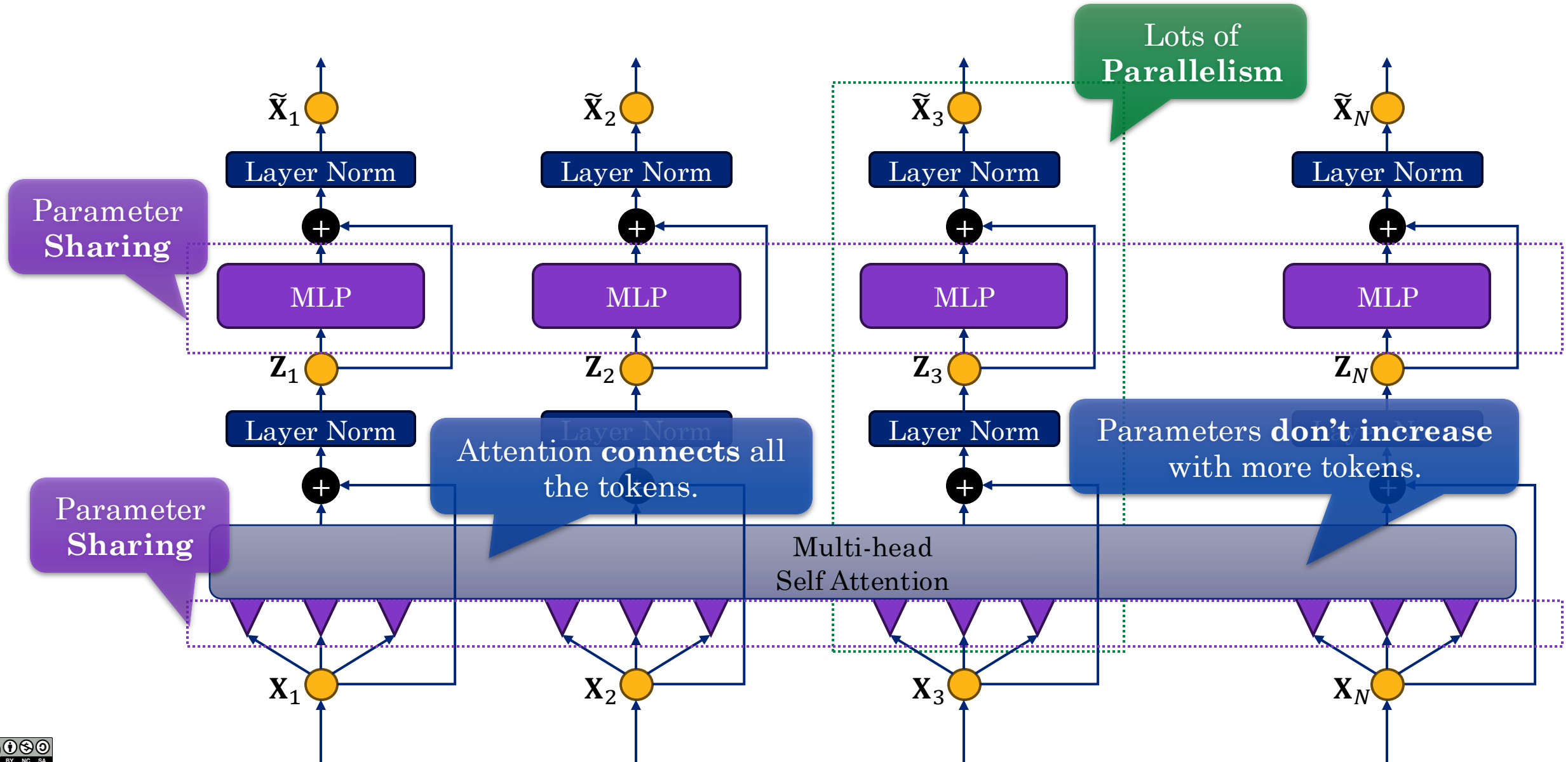
- Why is the the MLP is needed?
 - The output of attention is a **linear combination** (αV) of **linear transformations** ($V = X W^{(v)}$) on the input though α is a **non-linear function** of X .

Residual connections and **layer norm** are used to improve training

Transformer layers are repeated **many times** in a transformer model.



Unrolling the Transformer Layer

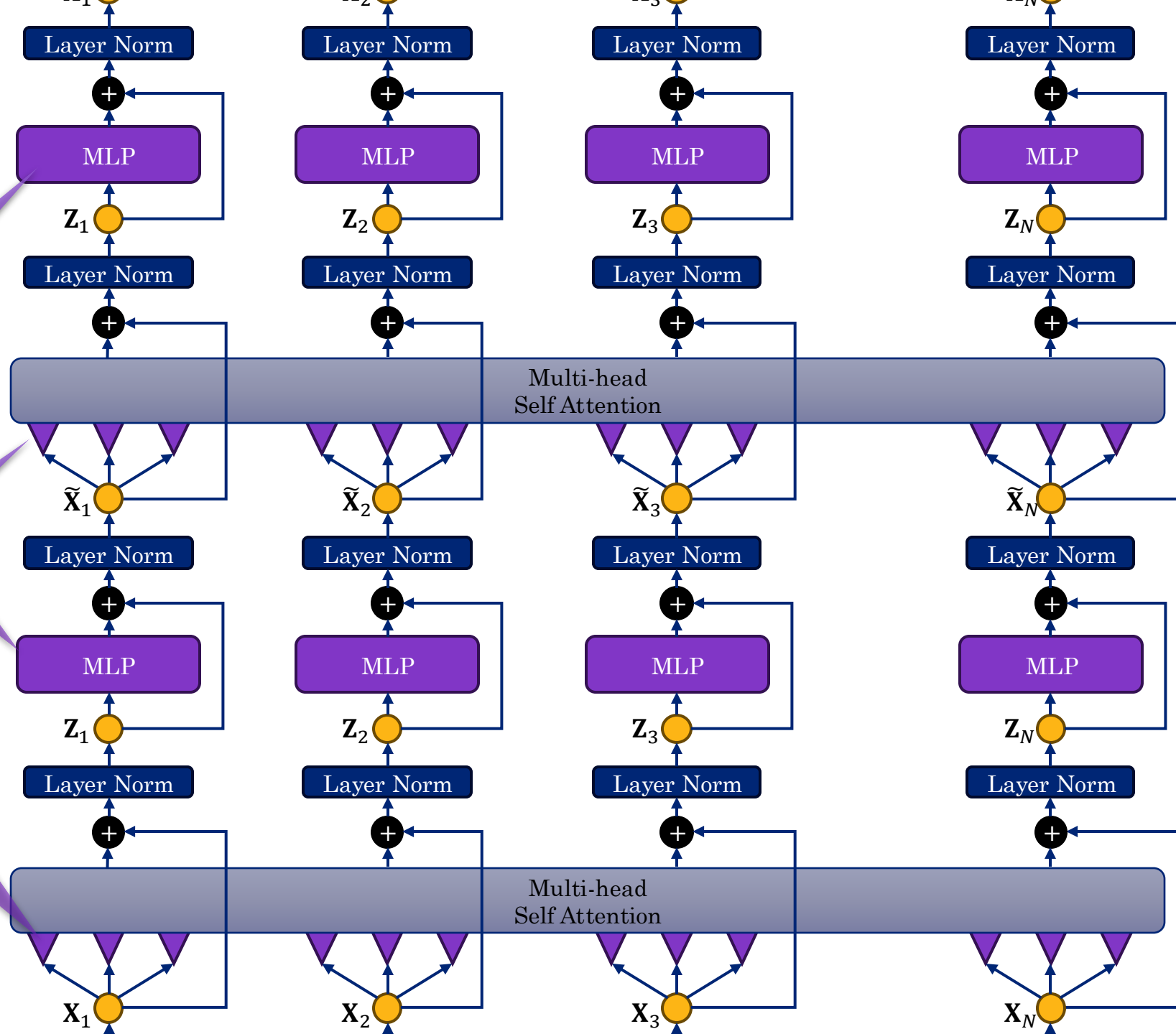


Stacking Transformer Layers

Different Weights!
(not shared)

Different Weights!
(not shared)

Each layer has
new weights.



Reading the Output of the Transformer

Suppose we want to apply the **transformer model** architecture to **image classification**.



Reading the Output of the Transformer

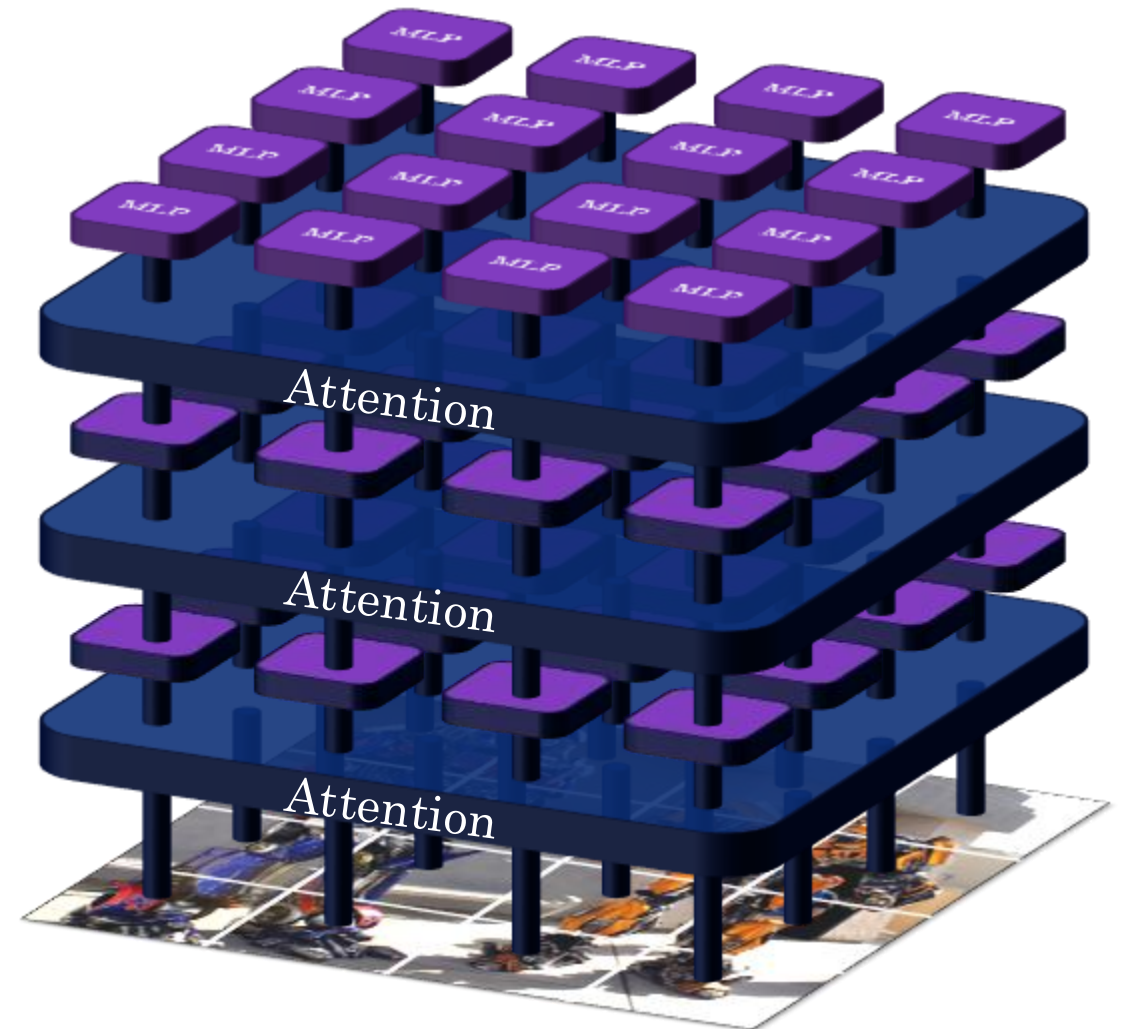
Suppose we want to apply the **transformer model** architecture to **image classification**.

The top layer now has many MLP outputs.

How do we combine them?

- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:



Reading the Output of the Transformer

Suppose we want to apply the **transformer model** architecture to **image classification**.

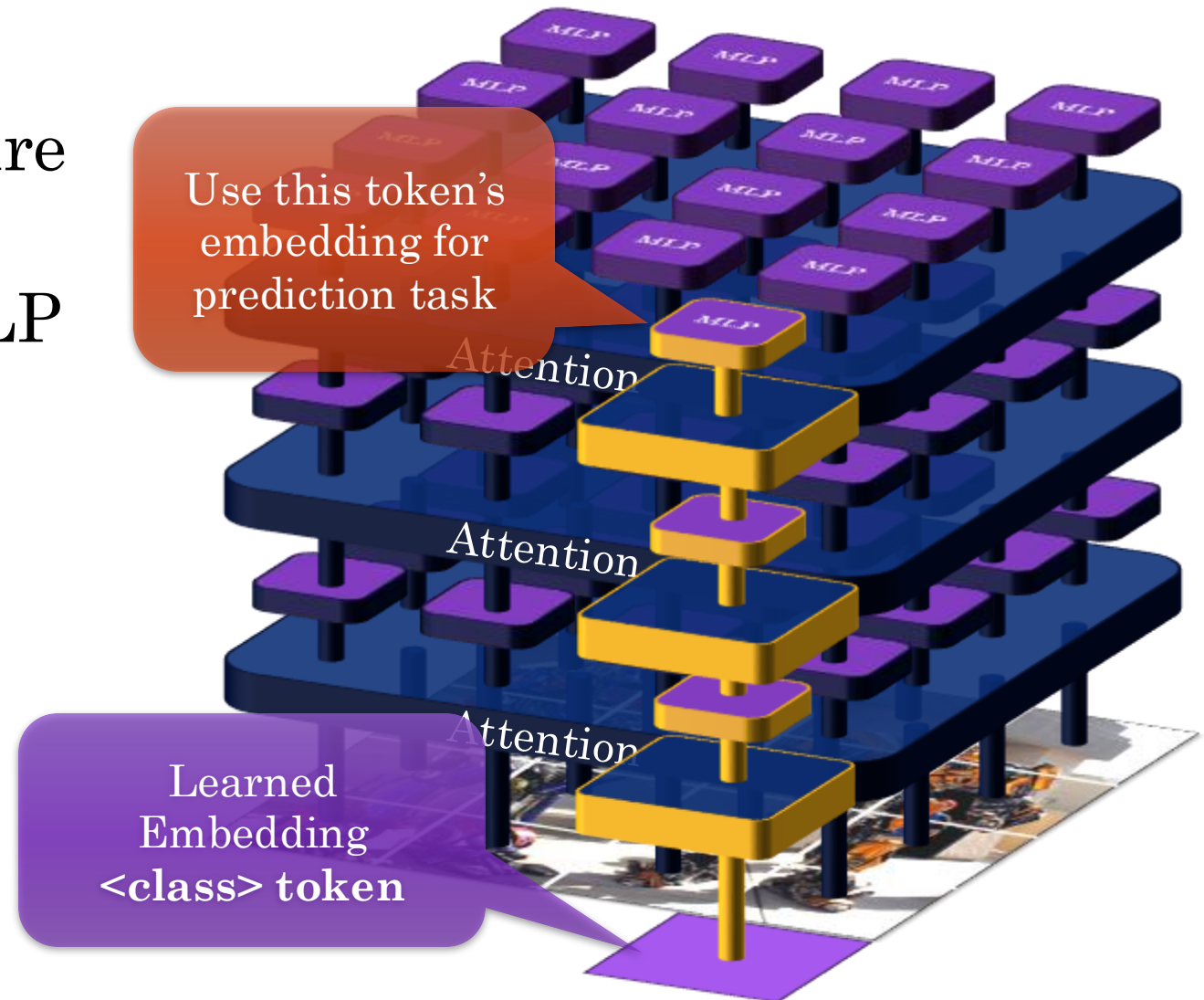
The top layer now has many MLP outputs.

How do we combine them?

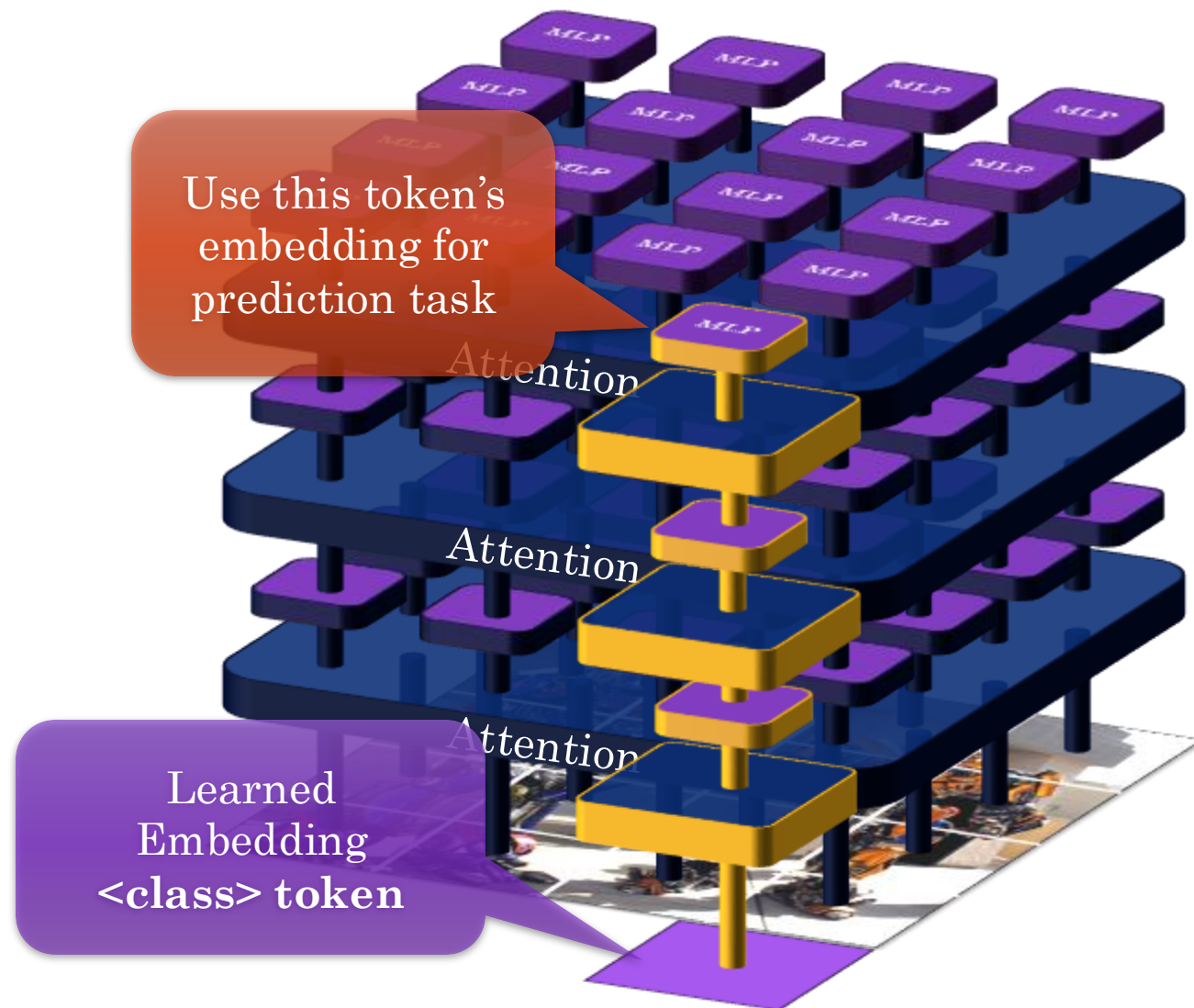
- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:

- **Add another token**



Reading the Output of the Transformer



Transformers and Token Ordering

Does the **order of tokens matter** in the transformer model?

If I **rearrange the tokens**, would I get a different output for **each token**?



Recall the attention operation is:

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = \sum_{i=1}^N A_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{Softmax} \left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

the **weighted averaging** which is **order invariant**.

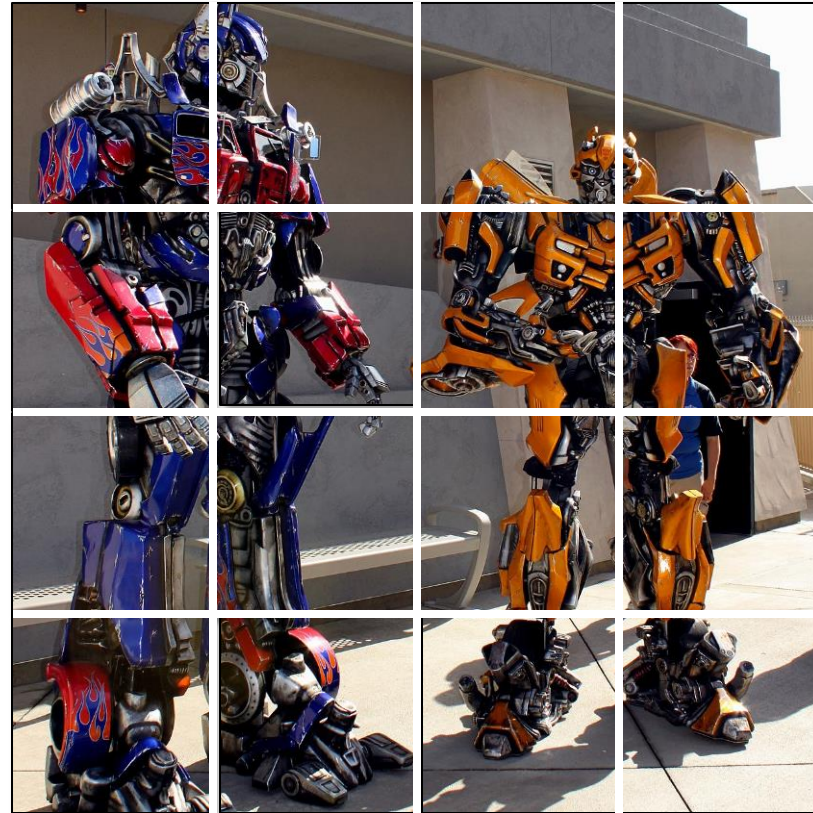
- Attention is **equivariant** – permuting the input tokens permutes the outputs.

Is this a problem?

- **No.** If you want a neural network for **sets of objects**.
- **Yes!** If you want to model **images** or **text**.

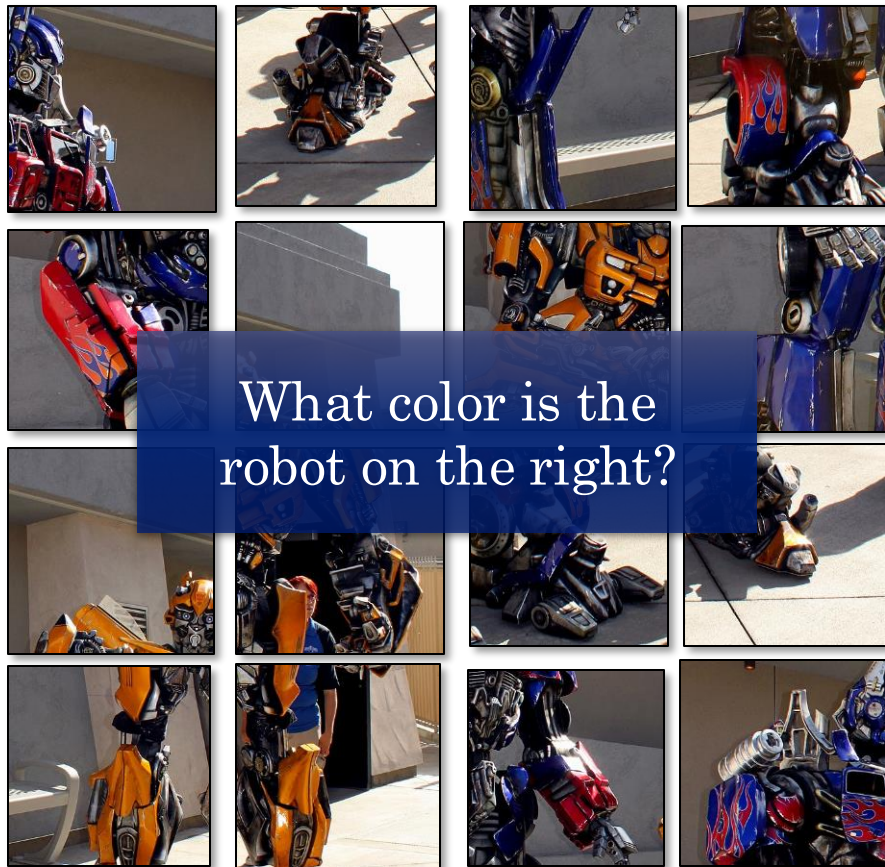
Token Position in Images and Text

Positional information is important for understanding images and text.



Token Position in Images and Text

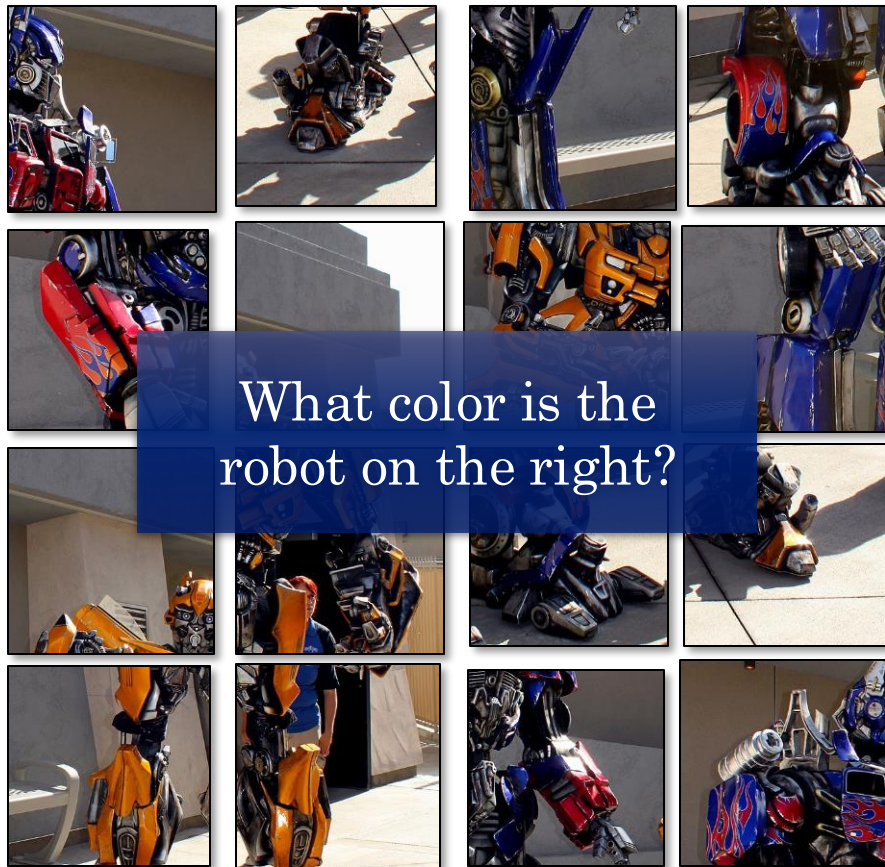
Positional information is important for understanding images and text.



The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

Token Position in Images and Text

Positional information is important for understanding images and text.



The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

The	food	was	bad	not	good	at	all!
-----	------	-----	-----	-----	------	----	------

A small permutation in the order changes the meaning!

Positional Encoding

Modify the raw features to contain information about the position of the token in the input. (**Feature Engineering**)

We add a **positional embedding** $\mathbf{r}_n \in \mathbb{R}^D$ **vector** encoding the position to the initial input embedding of each token $\tilde{\mathbf{x}}_n$:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

- Why not **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$?
 - Increases the dimensionality changing the model.
- Wouldn't this corrupt $\mathbf{x}_n$?
 - For high-dimensional vectors $\mathbf{x}_n$ and $\mathbf{r}_n$ are likely to be almost orthogonal.
 - Linear layers would end up adding $\mathbf{x}_n$ and $\mathbf{r}_n$ if we had used **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$
- How do we construct $\mathbf{r}_n$?

Designing the Position Embedding

An ideal positional encoding should:

1. Provide a **unique representation** $\mathbf{r}_n \in \mathbb{R}^D$ for each location.
2. Be **bounded** on the scale of the original features $\mathbf{x}_n$ (e.g., $[-1,1]$).
 - Needed for **stable training**.
3. Easily express **relative distances** between tokens
 - **Inductive Bias:** relative location is important for images and language.
4. Should work for **arbitrary number of tokens** N
 - Not all sentence or images are the same size.

Learned Positional Embeddings

The first GPT model (GPT-1) used **learned positional embeddings** in which each **positional embedding vector** $\mathbf{r}_n \in \mathbb{R}^D$ is learned through gradient descent.

- The vectors $\mathbf{r}_n$ were randomly initialized (small values)

Strengths:

- Expressive and easy to implement

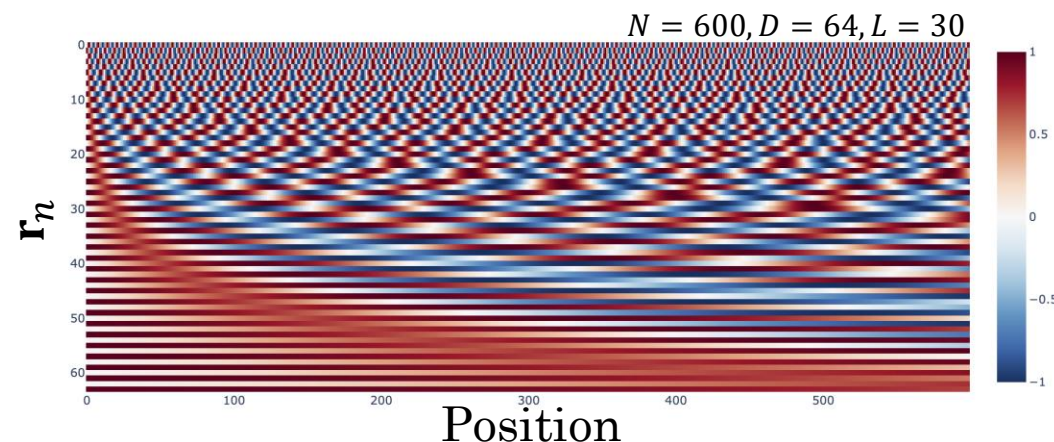
Weaknesses:

- The maximum N needs to be known in advances
- Relative distances can't be easily computed (needs to be learned)

Sinusoidal Positional Encoding

The original Transformer paper used sinusoidal embeddings to encode positions through a combination of waves.

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$



Like a “binary” encoding of the position but continuous:

Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
2	0	0	0	1	1	1	0	0	0	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
4	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

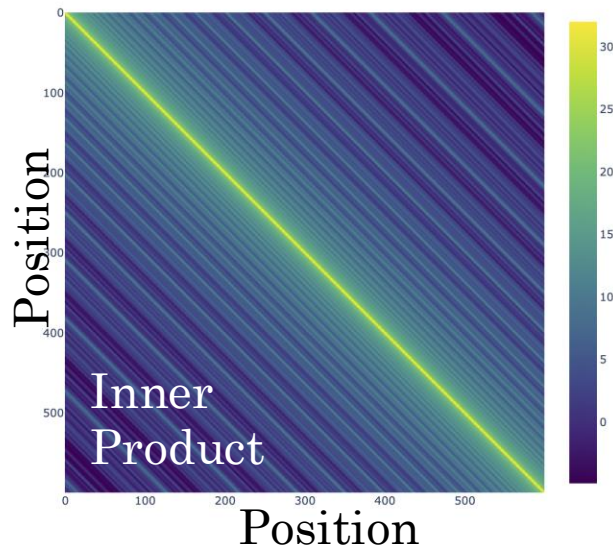
Querying a Relative Position

The sinusoidal position enables **linear layers** to query a fixed offset.

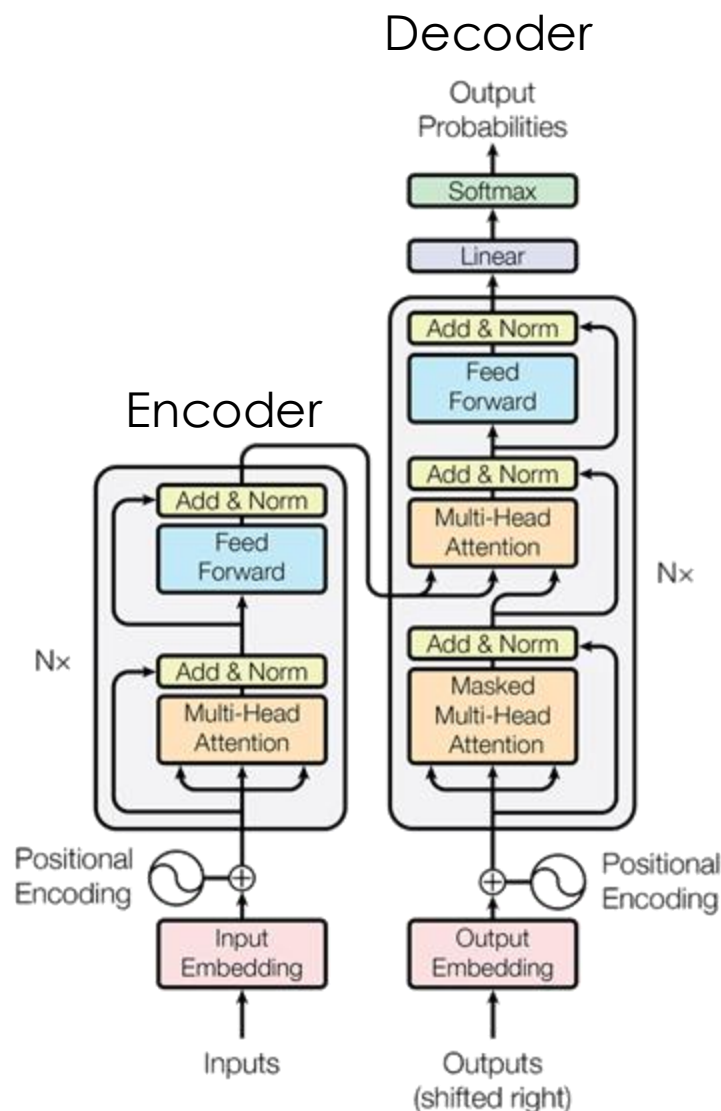
Suppose I have $\mathbf{r}_n$ and I want to query a key at $\mathbf{r}_{n-\Delta}$ then there exists a rotation matrix $\mathbf{R}_\Delta$ such that:

$$\mathbf{r}_{n-\Delta} = \mathbf{R}_\Delta \mathbf{r}_n$$

Inner-products of these encodings decrease with distance:



The Encoder Transformer



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

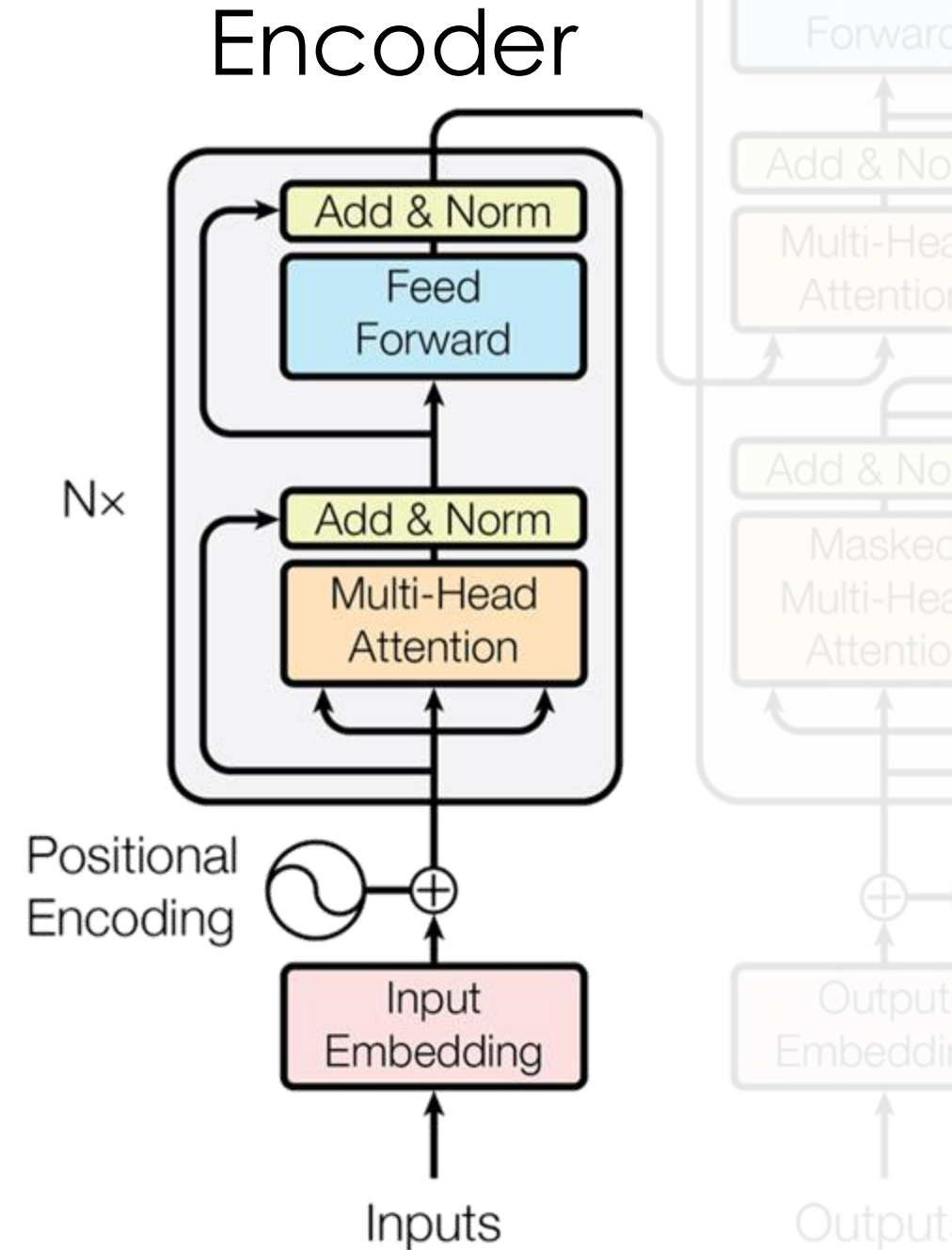
[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

The Encoder Transformer

The encoder transformer architecture is standard for **computer vision tasks** and **language embedding** tasks.

- Construct input embeddings
 - **Example:** Pixel patches
- Apply Positional Encoding
 - **Example:** Sinusoidal encoding
- Many repeated transformer blocks
- Use final output in downstream tasks
 - **Example:** Pooling or Special token



Lecture 21

Reading: *Chapter 12* in the Bishop Deep Learning Textbook